

Master's Thesis

**A Generative AI Framework for the
Design Optimization and
Performance Analysis of PCB
Power Delivery Networks**

M. Sc. Kovarthanam Muthusamy

Examiner: Prof. Dr.-Ing. Jürgen Götze

Supervisor: M. Sc. Nima Ghafarian Shoaee

External supervisor:

Submission Date:

Contents

List of Figures	v
List of Tables	viii
Mathematical Notation	ix
Acronyms	x
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 Research Objectives and Contributions	5
1.3 Structure of the Thesis	6
2 Theoretical Background	8
2.1 Power Delivery Networks and the Optimization Problem	8
2.1.1 Impedance Behavior and Decoupling Strategies	8
2.1.2 Power Integrity Simulation and Multi-modal Design Data	12
2.1.3 Formulation of the Decoupling Capacitor Placement Problem	14
2.2 Generative Modeling and Latent-Space Optimization	16
2.2.1 Variational Autoencoders and Multi-modal Learning	16
2.2.2 Physics-Informed and Surrogate Modeling	22
2.2.3 Optimization in Latent Representation Spaces	24
2.3 Related Work and Research Gap	25
2.3.1 Power Delivery Network Optimization Methods	25
2.3.2 Generative and Learning-Based Design Approaches	27
2.3.3 Research Gap and Motivation for the Proposed Framework	27
3 Design and Implementation	31
3.1 Data Preparation and Reprocessing	31
3.1.1 Raw Power-Integrity Simulation Outputs and File Structure	31
3.1.2 Multi-frequency Dataset Construction and Normalization	32

Contents

3.1.3	Data Splitting, Sampling, and Batch Construction	34
3.2	Multi-modal Variational Autoencoder as a Generative Surrogate	35
3.2.1	Architectural Design	35
3.2.2	Objective Functions and Physics-Informed Regularization	38
3.2.3	Training, Inference	41
3.2.4	Auxiliary Guidance Network for Latent-Space Optimization . .	43
3.3	Latent-Space Optimization of Decoupling Capacitor Placement	43
3.3.1	Problem Formulation	43
3.3.2	Optimization Procedure and Feasibility Assessment	45
3.4	Experimental Setup	46
3.5	Active Learning for scaling the model	47
4	Results and Evaluations	52
4.1	Performance of the Generative Surrogate Model	52
4.1.1	Reconstruction Performance	52
4.1.2	Validation of Generated PI Spatial Heatmaps	52
4.1.3	Evaluation of the Auxiliary Guidance Network	52
4.2	Latent-Space Design Optimization Results	52
4.2.1	Evaluation of Scalability and Computational Efficiency	52
4.2.2	Spatial placement and Capacitor-Type Evaluation of the Frame- work	52
4.3	Limitations	52
4.4	Ablations	52
5	Conclusion and Outlook	53
5.1	Summary of Contributions	53
5.2	Future Research Directions	53
	References	54
A	Power Integrity Simulation Environment and Dataset Specification	60
A.1	Simulation environment	60
A.1.1	Solver and invocation	60
A.1.2	Design under analysis and host directories	61
A.1.3	Solver settings not recorded in the repository	62
A.2	Batch file format and simulation phases	62
A.2.1	PEB structure	62
A.2.2	Two-phase dataset simulation	63
A.2.3	Pipeline configuration constants	63

Contents

A.2.4	Active-learning batches	64
A.3	From solver output to tensors	64
A.3.1	Decap slots and occupancy vectors	64
A.3.2	Impedance spectra	65
A.3.3	PI-distribution heatmaps	65
A.4	Dataset layout and metadata	65
A.4.1	Directory structure	65
A.4.2	<code>dataset_meta.json</code> fields	67
A.4.3	Dataset inventory and version control	67
A.5	Dataset statistics	68
A.5.1	Layout pool	68
A.5.2	Raw training set	68
A.5.3	Frequency anchors	69
A.5.4	Normalized training set	70
A.5.5	$K \leq 5$ subset	70
A.5.6	Active-learning overlay (exp059)	71
A.6	Normalization statistics	72
A.6.1	File structure	72
A.6.2	Heatmap normalization	72
A.6.3	Impedance and frequency normalization	73
A.7	Multi-type decap campaign (exp060) — in flux	73
A.7.1	Type catalog and combination list	74
A.7.2	Simulation pipeline configuration	74
A.7.3	Campaign state	75
A.7.4	exp060 configuration	76
A.8	Recorded inconsistencies between artifacts	76
A.9	Reproducibility notes	76
B	Hyperparameter Configuration for Training and Optimisation	78
B.1	Scope and how to read these tables	78
B.2	Surrogate training configuration: <code>exp059_capacity_freq</code>	79
B.2.1	Latent structure and architecture keys	79
B.2.2	Training schedule, optimiser and learning-rate scheduler	79
B.2.3	Data splitting and class balancing	80
B.2.4	Dataset, overlay and checkpoint paths	80
B.2.5	Logging and validation cadence	80
B.2.6	Heatmap loss weights	82
B.2.7	Impedance loss weights and peak curriculum	84

Contents

B.2.8	Occupancy loss	84
B.2.9	KL and latent regularisation	84
B.2.10	Modality dropout and cross-modal terms	86
B.2.11	Cross-frequency and layout path	86
B.2.12	Distillation and physics terms	87
B.2.13	Off-anchor evaluation and fine-tune early stopping	88
B.2.14	Numerical stability	89
B.2.15	Hardware and runtime	89
B.3	Architecture switches taken from code defaults	90
B.4	Fine-tune override configurations	90
B.4.1	Active-learning fine-tune: file value versus value actually used	91
B.4.2	Occupancy-only warm-up configuration	91
B.4.3	Active-learning configuration blocks that reach the fine-tune	91
B.5	Conflicts between configuration files	96
B.6	Stage-2 latent-optimisation configuration	98
B.7	Values this appendix cannot supply	101
C	Supplementary Experimental Results	102
D	Implementation Index	103
D.1	Chapter 2 — Theoretical Background	103
D.2	Chapter 3 — Design and Implementation	114

List of Figures

List of Tables

A.1	Solver executable and command-line interface.	61
A.2	Design, power bus and host directories used by every simulation pipeline.	61
A.3	Simulation parameters that live in the eCADSTAR project on the Windows host and are not recoverable from the repository.	62
A.4	Elements emitted inside a PEB group.	63
A.5	Simulation phases and their harvested artifacts.	63
A.6	Single-type combinations simulation pipeline (<code>pipelines/dataset_sim/run_combinations</code>	
A.7	eCADSTAR settings in the primary AL configuration (<code>active_learning_pi/config/exp05</code>	
	key <code>ecadstar</code>).	64
A.8	Frequency axis and target curve stored under <code>configs/</code>	65
A.9	Heatmap construction from a PI-Distribution <code>.map</code> file.	66
A.10	Layout-store dataset structure.	66
A.11	Fields written by <code>libs/dataset_meta.py</code>	67
A.12	Dataset directories present on the working machine (all git-ignored). Paths are absolute.	68
A.13	<code>datasets/data_multifreq_train</code> (raw), as read on 2026-08-06.	69
A.14	<code>datasets/data_multifreq_train_norm_unbounded</code> (normalized; the exp059 training input).	70
A.15	<code>datasets/data_multifreq_train_K5</code> and its dry-run report.	71
A.16	<code>datasets/data_multifreq_al_overlay_exp059</code> , read from disk on 2026- 08-05.	71
A.17	Global heatmap statistics, <code>datasets/data_multifreq_train_norm_unbounded/normaliza</code> key <code>Heatmap</code>	72
A.18	Per-MHz heatmap statistics, first four anchors.	72
A.19	Impedance and <code>PI_freq</code> statistics blocks of the normalized training set.	73
A.20	Multi-type decap catalog (<code>scrap/generation/generate_peb_multitype.py</code> , <code>TYPE_CATALOG</code>).	74

A.21 Multi-type simulation state. The progress-file rows were re-read on 2026-08-06; the folder counts were taken on 2026-08-05 after the final progress update.	75
A.22 Conflicts between artifacts. Both sides are named; none is resolved here.	77
B.1 Latent and architecture keys set in <code>config.yaml</code> (5 keys).	79
B.2 Training schedule and scheduler keys (13 keys).	80
B.3 Data split and balancing keys (7 keys).	80
B.4 Dataset, overlay and checkpointing keys (8 keys).	81
B.5 Logging and validation cadence keys (3 keys).	81
B.6 Heatmap reconstruction and structure loss weights (15 keys).	82
B.7 Heatmap peak- and valley-region loss terms (11 keys).	82
B.8 Heatmap focus phase and low- K emphasis (9 keys).	83
B.9 Heatmap value range and clipping (4 keys).	83
B.10 Impedance loss weights (11 keys).	84
B.11 Impedance peak curriculum (6 keys).	84
B.12 Occupancy loss keys (11 keys).	85
B.13 KL, β -annealing and latent regularisation keys (17 keys).	85
B.14 Modality dropout and cross-modal keys (8 keys).	86
B.15 Cross-frequency and layout-path keys (11 keys).	86
B.16 Latent- and output-distillation keys (4 keys).	87
B.17 Physics-loss keys (10 keys).	87
B.18 Off-anchor evaluation keys (4 keys). 250.0 is a training anchor; see the text above.	88
B.19 Active-learning fine-tune early-stopping keys present in the base <code>config.yaml</code> (4 keys).	88
B.20 Numerical-stability keys (10 keys).	89
B.21 Hardware and runtime keys (12 keys).	89
B.22 Defaults in <code>vae_model_kwargs()</code> and whether <code>config.yaml</code> overrides them. “Code default” means the key does not appear in any exp059 configuration file.	90
B.23 Active-learning fine-tune configuration: hand-written <code>config_al_finetune.yaml</code> (29 keys) against the generated <code>config_al_finetune.runtime.yaml</code> (29 keys). Rows in which the two disagree are discussed in section B.5.	92
B.24 <code>config_occ_only_finetune.yaml</code> , the occupancy-only warm-up override (28 keys).	93
B.25 <code>finetune</code> block of the primary active-learning configuration <code>active_learning_pi/config/</code> (lines 111–122).	94

List of Tables

B.26 <code>pre_al_occ_only_warmup</code> block of <code>active_learning_pi/config/exp059_gp_error.json</code> (lines 106–110).	94
B.27 <code>gp_error</code> block of <code>active_learning_pi/config/exp059_gp_error.json</code> (lines 140–155), 14 keys.	95
B.28 <code>optimize.py</code> model and checkpoint selection (lines 45–54).	98
B.29 <code>optimize.py</code> search setup (lines 56–69).	98
B.30 <code>optimize.py</code> latent-optimiser settings and objective (lines 71–77). . . .	99
B.31 <code>optimize.py</code> latent-prior and spectrum-shape regularisation weights (lines 79–86).	99
B.32 <code>optimize.py</code> peak-aware spectrum terms (lines 88–94). Each is read from the named environment variable with the default shown.	99
B.33 <code>optimize.py</code> diversity, occupancy-confidence and feasibility terms (lines 96–103).	100
B.34 <code>optimize.py</code> paths, device and output naming (lines 105–120).	100

Mathematical Notation

Acronyms

Chapter 1

Introduction

1.1 Motivation and Problem Statement

A printed circuit board’s power delivery network (PDN) has to supply the integrated circuits on the board with a stable voltage while they draw current that changes quickly. The usual way to express this requirement in the frequency domain is the target impedance rule: the impedance $Z(f)$ seen looking into the PDN from the load must stay below a frequency-dependent limit $Z_{\text{target}}(f)$ across the band of interest [1]. Decoupling capacitors (decaps) are the main tool the designer has for shaping $Z(f)$, and the design task is to decide how many of them to place and where.

In this thesis the band of interest is 1–600 MHz, sampled at 231 discrete frequency bins. The bin locations are fixed by the file `configs/Frequency_data_hz.npy`, which holds 231 values running from 1000000.0 Hz to 600000000.0 Hz. The grid is not uniformly spaced: the spacing between successive bins takes the three values 0.1 MHz, 1 MHz and 10 MHz. **[FILL IN: state why the frequency grid uses three different step sizes and who chose the breakpoints – the ECADSTAR sweep setup or export script that produced `Frequency_data_hz.npy` would settle it]**

The impedance limit itself is stored as an array of shape $(231, 1)$ in `configs/target_impedance.n`, so there is one limit per bin. It is not a constant: the first element is 0.8 and the last is 50 (stored as 49.99999999999999 in `float64`), and the values vary with frequency in between. Stage 2 of the framework loads this file, casts it to `float32` and squeezes it to a length-231 vector before use. **[FILL IN: confirm the unit of `configs/target_impedance.npy`. The rest of this thesis reads the mask as ohms because the Stage-2 code compares it against a tensor named `imp_ohm` and its default selection metric is named `max_ohm`, but that is an inference from variable naming, not a unit recorded with the array; a design note or the ECADSTAR export settings would settle it] [FILL IN: record the prove-**

nance of configs/target_impedance.npy – who specified this mask, and from which load-current or ripple budget it was derived]

The placement problem

The board studied here exposes a fixed set of 52 candidate decap sites. These are the valid cells of a 7×8 physical grid: the code sets `OCC_GRID_H`, `OCC_GRID_W` = 7, 8 and marks four cells invalid, namely (0, 3), (0, 4), (6, 3) and (6, 4), leaving 52 slots labelled C1–C52. A design decision is therefore a binary placement vector

$$\mathbf{b} \in \{0, 1\}^{52}, \quad K = \|\mathbf{b}\|_0, \quad (1.1)$$

where $b_i = 1$ means a decap is mounted in slot i and the budget K is the number of decaps used. K matters because decaps cost money and board area, so a placement that meets the mask with fewer parts is preferable to one that needs more.

Two properties make this problem hard. First, the map $\mathbf{b} \mapsto Z(\cdot)$ is only available through an expensive field solver, so every candidate placement that is to be judged exactly has to be simulated. Second, the discrete space has 2^{52} elements, which is stated in the project documentation as intractable to enumerate. The combination means that neither exhaustive search nor a simulation-in-the-loop search that evaluates a large number of candidates is practical.

The conventional design loop

The conventional workflow this thesis targets is iterative. The engineer places a set of decaps, simulates the resulting impedance spectrum, and checks it against the mask. If peaks violate the mask, a spatial power-integrity distribution is computed at the offending frequency, decaps are moved towards the hotspots that this distribution reveals, and the cycle repeats. Every turn of that loop contains at least one full solver run.

[FILL IN: quantify the cost of this loop for the board under study – engineer hours per iteration and the number of iterations typically needed to converge; nothing in the repository records this, and the literature search found no freestanding paper that measures it] The claim that iterative and manual decap tuning is slow enough to motivate a learned replacement is asserted in the motivation sections of the machine-learning and genetic-algorithm decap literature rather than measured in a dedicated study [2]–[5].

Ground-truth impedance data for this thesis is produced with ECADSTAR, which runs on a Windows host addressed from WSL; the active-learning configuration car-

ries absolute Windows paths for the ERF, EMC and PEB locations, for example `C:\Users\muthusamy\Desktop\design\H-shape.emc\H-shape.erf`. The configuration allocates a wait timeout of 18000 seconds for a simulation batch and 900 seconds for batch start, polled every 40 seconds. These are configured timeouts and not measurements. **[FILL IN: measure and report the wall-clock time of one ECADSTAR PI simulation, per layout and per batch; a timing log from an AL cycle’s simulate step would settle it] [FILL IN: report surrogate inference time on the same hardware and the resulting speed-up factor over simulation – no artifact in the repository contains this comparison]**

Learned approaches to decap placement

Learning-based decap placement is not new. Deep reinforcement learning was applied to decap selection from 2019 onwards [6], extended with an impedance-matrix reduction method and double Q-learning to cut optimisation time [3], and applied to silicon-interposer 2.5-D/3-D IC decap design in the same period [7]. The same line later moved to transformer-based policies [8], [9], and separate strands combine graph neural networks with genetic algorithms [5] or use learned models purely as fast impedance predictors [10]. Chapter 2 treats this work in detail and states where the present framework differs.

Approach taken in this thesis

The framework documented in this repository replaces the conventional loop with four elements. A learned differentiable surrogate of one fixed board, built as a multi-input variational autoencoder, stands in for the field solver. Gradient-based inverse design in the latent space of that surrogate searches for placements under a budget K . An active-learning loop optionally refines the surrogate on layouts where its error or uncertainty is high, simulating only a selected batch rather than every candidate. Finally, a performance-analysis layer decodes a frequency-conditioned spatial heatmap at the peak frequencies of a predicted spectrum. The project documentation gives the purpose of that layer as hotspot interpretation, that is, the intent is to inspect a predicted violation spatially instead of only in the spectrum. Whether the decoded heatmaps are accurate enough for that use is evaluated in Section 4.1.2; nothing in this chapter asserts that they are.

The surrogate models three modalities jointly for a single board: decap occupancy as the binary vector of Equation (1.1), the PI spectrum magnitude over the 231 frequency bins, and a 64×64 spatial PI heatmap at selected MHz anchors. Conditioning inputs are the budget K and the inspection frequency. In the current model track,

`exp059_capacity_freq`, the three modalities carry the shapes $(B, 1, 64, 64)$ for the heatmap, $(B, 52)$ for occupancy and $(B, 1, 231)$ for the impedance vector, where B is the batch size. These shapes are the ones documented in the docstring of the model’s `encode` method in `experiments/exp059_capacity_freq/codes/vae_multi_input_simple.py`, which lists the encoder’s three modality inputs. K is defined in code as the number of ones in the occupancy vector, so $K \in \{0, \dots, 52\}$. **[FILL IN: state the exact return signature of the `exp059` forward pass and the form of its occupancy output – in particular whether that output is decoder logits that are thresholded before use, or an already-binarised vector. The shapes above come from the `encode` docstring and are input shapes; the forward and decode definitions in the same file would settle what is returned]**

The inverse-design objective is stated per budget. For each K , a placement is feasible if the predicted spectrum stays below the mask with a safety margin m ,

$$\hat{Z}(f; \mathbf{b}) \leq Z_{\text{target}}(f) - m \quad \text{for all } f \text{ in the 231 bins,} \quad (1.2)$$

where $\hat{Z}$ denotes the surrogate’s predicted impedance as opposed to the simulated Z . Among feasible candidates the one with the lowest peak impedance $\max_f \hat{Z}(f; \mathbf{b})$ is selected; the implementation’s default selection metric is `max_ohm`, the maximum of `imp_ohm` over the frequency bins. If no candidate for a given K is feasible, the procedure records an explicit `no_solution.json` for that K instead of returning a best-effort placement. As the code stands, the Stage-2 script `pipelines/latent/optimize.py` loads its occupancy decoder and impedance surrogate from the legacy experiment `exp038_true_multi` rather than from `exp059_capacity_freq`, so the objective above is stated as method only and no inverse-design outcome is attributed to the current surrogate anywhere in this thesis.

One implementation detail of the search is load-bearing and is introduced here because it shapes the whole formulation. Reading a placement out of the decoder requires a hard top- K selection, which is a discrete operation with no useful gradient. The straight-through operator used in Stage 2 returns the hard top- K binary vector in the forward pass but passes the gradient of the soft occupancy probabilities backwards; its docstring states that without this, the top- K argsort severs the gradient and the latent vector never moves in response to the spectrum target.

Scope and validity boundary

The scope is deliberately narrow. The surrogate is trained for a single board and one design family, with no cross-board generalisation claimed; the decap type and the 52-slot set are fixed, so no multi-value or multi-type catalogue is covered; and the current

training filter keeps layouts with $K \leq 30$. The normalised training dataset records 24979 unique layouts and 582997 manifest rows.

Surrogate spectra and heatmaps are predictions, not sign-off. A proposed design has to be re-simulated with the ground-truth PI flow before it can be accepted. This boundary is stated explicitly in the project’s own validity notes and is repeated here because it constrains every result claim made later in the thesis.

[FILL IN: describe the board under study – layer stack, physical dimensions, IC port location and the power bus being decoupled; the AL configuration references an ERF named H-shape and a powerbus named Power_GND, but no board specification document exists in the repository] [FILL IN: state the capacitance value and part type assumed for the 52 slots in exp059; the only decap-profile artifact found belongs to a legacy experiment note and not to the exp059 track] [FILL IN: personal and industrial motivation for choosing this topic – not recorded anywhere in the repository, must be written by hand]

[FIGURE: Board slot map: the 7×8 grid with the four invalid cells removed, slots labelled C1–C52.]

[FIGURE: Conventional place–simulate–inspect–adjust loop next to the proposed surrogate-based loop.]

1.2 Research Objectives and Contributions

The overall aim follows from Section 1.1: to replace the simulate-in-the-loop decap placement search on one fixed board with a learned differentiable surrogate, to perform the search by gradient descent in that surrogate’s latent space under a budget constraint, and to add an analysis layer that localises impedance violations spatially. Active learning is included as a mechanism for spending a limited simulation budget on the layouts where the surrogate is least reliable.

Beyond that aim, the research questions and the contribution list are not fixed yet. No research questions and no enumerated contributions exist in the repository, in the documentation, or in the thesis vault, and they require supervisor agreement before they are written down.

[FILL IN: RQ1–RQn: formulate the research questions with the supervisor. Candidate themes visible in the artifacts are (i) whether a multi-modal VAE surrogate of one board is accurate enough to drive placement decisions, (ii) whether latent-space gradient search through a straight-through top- K read-out finds feasible placements at smaller K than the alternatives, and (iii) whether active learning improves the surrogate where it matters un-

der a fixed simulation budget. These are candidates only and must not be treated as agreed.]

[FILL IN: Contribution list: enumerate the claimed contributions once the experiments in Chapter 4 are complete and agreed with the supervisor. This list must match Section 5.1 in the conclusion. Nothing in the repository currently enumerates contributions.]

[FILL IN: State explicitly which of the objectives are demonstrated end-to-end and which are not. As the code stands, the Stage-2 inverse-design script loads occupancy-decoder and impedance-surrogate checkpoints from the legacy experiment `exp038_true_multi`, while the current surrogate track is `exp059_capacity_freq` – so no end-to-end inverse-design result may be attributed to `exp059` until Stage 2 is rewired. Resolve before claiming any objective as met.]

[FILL IN: Confirm the delimitation sentence: what this thesis does not attempt (signal integrity sign-off, EM/EMC field solving, emissions, cross-board transfer). Wording to be agreed.]

1.3 Structure of the Thesis

The thesis has five chapters. Front matter provides a table of contents, a list of figures, a list of tables, an unnumbered chapter on mathematical notation and an unnumbered list of acronyms.

Chapter 2, *Theoretical Background*, sets out the theory the rest of the work rests on. Section 2.1 covers power delivery networks and the optimisation problem, with subsections on impedance and decoupling (2.1.1), PI simulation and its multimodal outputs (2.1.2), and the formal decap placement formulation (2.1.3). Section 2.2 covers generative modelling and latent-space optimisation, with subsections on multimodal variational autoencoders (2.2.1), physics surrogates (2.2.2) and latent-space optimisation (2.2.3). Section 2.3 reviews related work on PDN optimisation methods (2.3.1) and generative learning approaches (2.3.2) and closes with the research gap (2.3.3).

Chapter 3, *Design and Implementation*, describes what was built. Section 3.1 covers data preparation and reprocessing: the raw PI outputs (3.1.1), the multi-frequency dataset (3.1.2) and the splitting and sampling strategy (3.1.3). Section 3.2 describes the multi-modal variational autoencoder used as the generative surrogate, covering the architectural design (3.2.1), the training objective and physics regularisation (3.2.2), training and inference (3.2.3) and the auxiliary guidance network (3.2.4). Section 3.3 formulates the latent-space optimisation of decap placement (3.3.1) and the optimisation procedure with its feasibility test (3.3.2). Section 3.4 states the experimental

setup and Section 3.5 the active-learning loop used to scale the model. **[FILL IN: state the per-cycle active-learning budget in Chapter 3 and resolve the conflict between active_learning_pi/config/exp059_gp_error.json (candidates_per_k 1600, worst_per_k 80, total_ecad_per_cycle 640, K = 3..10) and docs/active-learning.md:110–112, which documents 8 x 400 candidates and $8 \times 10 = 80$ ECAD simulations for the older exp057 loop]**

Chapter 4, *Results and Evaluations*, reports the measurements. Section 4.1 evaluates the generative surrogate through reconstruction performance (4.1.1), validation of predicted PI heatmaps (4.1.2) and evaluation of the auxiliary guidance network (4.1.3). Section 4.2 reports the latent-space design optimisation results, covering scalability and efficiency (4.2.1) and the spatial and capacitor-type evaluation (4.2.2). Section 4.3 states the limitations and Section 4.4 the ablations.

Chapter 5, *Conclusion and Outlook*, summarises the contributions (5.1) and gives directions for future research (5.2).

Three appendices follow the bibliography. Appendix A documents the power integrity simulation environment and the dataset specification. Appendix B lists the hyperparameter configuration used for training and optimisation. Appendix C collects supplementary experimental results. **[FILL IN: confirm what Appendix C will contain; it is still declared inline in thesis_v3.tex with the note that it depends on experiments not yet run]**

Chapter 2

Theoretical Background

This chapter collects the material the rest of the thesis depends on. It first describes the power delivery network (PDN) design problem in the specific form in which this work attacks it, then the simulation flow that produces the training data and the formal statement of the decoupling capacitor (decap) placement problem. The second half covers the machine-learning background: variational autoencoders and multi-modal fusion, physics-informed and surrogate modelling, and gradient-based optimization in a learned latent space. The chapter closes with related work and the research gap that motivates the framework.

Throughout the thesis a placement is written as a binary vector $\mathbf{b} \in \{0, 1\}^{52}$ over the board’s decap slots, the budget is $K = \|\mathbf{b}\|_0$, the PDN input impedance seen at the observation port is $Z(f)$ sampled at 231 frequency bins spanning 1–600 MHz, and $Z_{\text{target}}(f)$ denotes the frequency-dependent target impedance mask the design must stay below.

Numerical settings quoted in this chapter are those of the implemented framework. Their source locations in the code base are recorded in the implementation index appendix rather than in the text, so that the discussion here stays with the concepts and the method.

2.1 Power Delivery Networks and the Optimization Problem

2.1.1 Impedance Behavior and Decoupling Strategies

A power delivery network has one job: hold the supply voltage at the load pins inside a tolerance while the current drawn by that load changes. A switching digital load does not draw a constant current. Each transition charges and discharges internal

capacitance, so the current at the power pins carries a large transient component whose energy is spread over a wide frequency range rather than concentrated at the clock rate. Write $I(f)$ for the spectrum of that current and $Z(f)$ for the impedance the network presents at the port where the load is attached. The voltage disturbance seen at that port follows the frequency-domain form of Ohm's law,

$$|V_{\text{ripple}}(f)| = |Z(f)| \cdot |I(f)|. \quad (2.1)$$

Equation 2.1 is what makes impedance the design variable. The requirement is stated on the ripple, but the current spectrum belongs to the silicon and not to the board, so the only factor the board designer controls is $Z(f)$.

The target-impedance methodology turns that observation into a specification. An allowed ripple ΔV_{max} and a worst-case transient current I_{max} are fixed, and their ratio

$$Z_{\text{target}} = \frac{\Delta V_{\text{max}}}{I_{\text{max}}} \quad (2.2)$$

becomes the impedance the network must stay below. The approach and the associated capacitor-selection procedure for CMOS power distribution are due to [1]. Because the load has spectral content across a broad band, a single scalar is not enough in practice and the bound is imposed as a mask over frequency; [11] develops this as the frequency-domain target impedance method (ch. 13.22). That is the form used throughout this thesis: $Z_{\text{target}}(f)$ over the 231 bins spanning 1–600 MHz, with the mask itself described below.

The decoupling capacitor as a series RLC circuit

At the frequencies of interest a decoupling capacitor is not well described by its capacitance alone. The standard lumped model is a series RLC branch combining the capacitance C , an equivalent series resistance ESR and an equivalent series inductance ESL:

$$Z_{\text{cap}}(\omega) = \text{ESR} + j\omega \text{ESL} + \frac{1}{j\omega C}, \quad |Z_{\text{cap}}| = \sqrt{\text{ESR}^2 + \left(\omega \text{ESL} - \frac{1}{\omega C}\right)^2}, \quad (2.3)$$

with $\omega = 2\pi f$. The magnitude has a minimum at the series self-resonant frequency, where the inductive and capacitive reactances cancel:

$$f_{\text{SRF}} = \frac{1}{2\pi\sqrt{\text{ESL} \cdot C}}. \quad (2.4)$$

At f_{SRF} the impedance of the branch equals its ESR. Below it the branch is capacitive and $|Z_{\text{cap}}|$ falls with frequency as $1/(\omega C)$; above it the branch is inductive and $|Z_{\text{cap}}|$ rises as ωESL . The consequence is that one capacitor value is only useful over a limited band around its own self-resonance, which is why a real PDN uses several values and several parts rather than one large capacitor. The multilayer ceramic capacitor model, the equivalent series inductance and the loop-inductance approximation behind it are treated in [11] (ch. 13.12–13.14).

Anti-resonance in parallel capacitor banks

Connecting capacitors in parallel does not lower the impedance everywhere. Consider two branches at a frequency at which one is already above its self-resonance, and therefore inductive, while the other is still below its own, and therefore capacitive. The parallel combination of an inductive and a capacitive branch has a parallel resonance, and at that frequency the two susceptances cancel in the opposite sense to Equation 2.4: the combination presents an impedance *peak* rather than a dip, limited only by the loss in the loop. The same mechanism acts between a capacitor bank and the inductance of the package and of the plane path that feeds it. Anti-resonant peaks, not the resonant dips, are what violate $Z_{\text{target}}(f)$, and they are the reason adding a capacitor can make a design worse at some frequencies while improving it at others.

The mitigations are all ways of either moving a peak, damping it, or lowering the inductance that creates it: spreading capacitor values so that the individual self-resonances and the peaks between them are staggered rather than aligned; choosing parts whose ESR is large enough to damp the parallel resonance instead of leaving it sharp; adding more capacitors, which [11] treats explicitly as a way of engineering a reduced parallel-resonant peak (ch. 13.16–13.17); and placing the capacitors so that the inductance between them and the load is small, which is taken up next. **[FILL IN: Pin the value-spreading and ESR-damping strategies to a specific passage in [11] or replace them. Ch. 13.16–13.17 is confirmed from the publisher’s table of contents to cover combining capacitors in parallel and reducing the parallel-resonant peak by adding more capacitors; the other two strategies are stated here from general design practice and were not read from a verified page.]**

Mounting inductance and the spatial nature of the problem

The ESL quoted for a part is only one contribution to the inductance of the current loop it forms with the load. The capacitor pads, the mounting vias down to the power and ground layers, and the path through the planes back to the point being decoupled

all add to that loop. Above self-resonance the branch impedance is set by the total loop inductance L_{loop} rather than by ESL alone, so $|Z| \approx \omega L_{\text{loop}}$ and the penalty for a large loop grows linearly with frequency. Two identical capacitors mounted differently, or sitting at different distances from the IC power pin, therefore do not decouple equally well. Optimizing the mounting of a capacitor is the subject of [11] ch. 13.15, with the loop-inductance approximation in ch. 13.14.

This is the physical reason the placement problem is spatial rather than a counting problem. The vector $\mathbf{b} \in \{0, 1\}^{52}$ selects both how many capacitors are fitted and where they sit relative to the observation port, and two placements with the same budget K do not in general produce the same $Z(f)$. It is also why the occupancy modality is encoded on a graph whose edges follow board adjacency rather than being treated as an unordered set of 52 bits (Section 2.2.1).

At the upper end of the band the plane pair the capacitors connect to stops behaving as a single lumped node and becomes a distributed structure with resonances of its own, which also enter $Z(f)$ [12]. **[FILL IN: Confirm against a physical copy of Novak and Miller that the book covers plane-pair cavity resonance, and cite the specific chapter. The table of contents could not be retrieved for this draft, so no statement about cavity modes, their frequencies or their spatial pattern is made here, and this sentence should either be substantiated or removed.]**

Anti-resonance as a training prior

The framework does not carry a circuit model of anti-resonance. What it carries is a shape prior on the predicted spectrum: every impedance peak must be preceded by a resonance dip. The penalty is evaluated on the raw log-impedance channel as the mean of $\text{peak_score} \cdot \exp(-10 \cdot \text{prior_dip})$. From bin index 190 upwards the spectrum is monotonically increasing by construction, so the term acts mainly in the low-frequency region. It is therefore a training regularizer with a physical motivation, and this chapter presents it as such rather than as a derived circuit relation.

Two design levers follow from the physics above. The first is which slots carry a capacitor, which is the discrete variable this thesis optimizes. The second is which part is placed in a slot. **[FILL IN: State explicitly that the decap type is fixed in the main study, and give the capacitance, ESR, ESL and part number actually used in the ECADSTAR simulations. No parameter table exists: the occupancy modality is binary presence only, and although the exploratory fork `exp060_multitype_occ` names capacitor types it does not record their electrical parameters. Settled by the ECADSTAR component library or the PEB project.]**

The frequency-dependent target mask

The impedance requirement enters the framework as a fixed mask over the 231 frequency bins, running from $0.8\ \Omega$ at the low-frequency end to $49.999999999999999\ \Omega$ at the high-frequency end. Only the mask values are supplied; no analytic form is fitted to them.

[FILL IN: Derivation and justification of this mask: why $0.8\ \Omega$ at the low end rising to $50\ \Omega$ at the high end, which load current and ripple budget it corresponds to, and who specified it. Nothing in the repository records how the array was produced – this is a gap the author must close from the original design brief.]

[FIGURE: Target impedance mask $Z_{\text{target}}(f)$ over the 231 bins, on log–log axes.]

2.1.2 Power Integrity Simulation and Multi-modal Design Data

Simulation flow and dataset generation

Training data is produced by an end-to-end flow in six steps: sample and merge placement combination tables; build a PEB project in ECADSTAR script order; simulate the impedance spectrum and the PI-distribution heatmaps on Windows through a headless batch invocation of the field solver; append the result to the raw multi-frequency dataset; normalize, by a robust per-MHz $\log(1+x)$ transform with an optional $K \leq 30$ filter; and then train, evaluate or active-learn. Row i of a combination table corresponds to the ECADSTAR object PI- $(i+1)$ in output order.

The conventional manual design loop that the framework is meant to replace runs as follows: place capacitors, simulate the spectrum, and if peaks violate the mask, run a spatial PI distribution, move capacitors nearer the hotspots, and repeat. This description is taken from the project documentation and not from a measured study of designer behaviour.

[FILL IN: PI solver configuration: solver type, meshing, frequency-sweep setup and convergence criteria. The documentation covers only the batch invocation and file paths, so this must come from the ECADSTAR project settings or the tool documentation.]

[FILL IN: Wall-clock cost and compute cost of one ECADSTAR PI simulation. This number is the quantitative motivation for building a surrogate at all, and no artifact in the repository records it. A single timed batch run would settle it.]

[FILL IN: Board description: layer stackup, layer count, dielectric, physical dimensions, plane geometry and the definition of the observation port.]

The active-learning configuration names only the port IC1_Port1 and the powerbus Power_GND; no geometry document exists.]

The three design modalities

The framework consumes one design as three coupled views.

Occupancy. The board carries 52 decap slots laid out on a 7×8 grid with four invalid cells at (0, 3), (0, 4), (6, 3) and (6, 4). The slots are labelled C1–C52 in a fixed order, and index i of the placement vector $\mathbf{b}$ refers to the i -th label in that order. The budget of a sample is recovered from the vector itself as $K = \sum_i b_i$.

Impedance spectrum. The spectrum has exactly 231 points, and the length is enforced at ingestion: a raw file of any other length is rejected. Inside the model the spectrum carries an explicit channel dimension, so a batch of shape $(B, 231)$ is expanded to $(B, 1, 231)$ and the impedance decoder returns $(B, 1, 231)$.

The frequency grid itself is a stored array of 231 values running from 1000000.0 Hz through 1100000.0 Hz to 600000000.0 Hz. The spacing is not uniform in $\log_{10}$: the first $\log_{10}$ step is 0.0414 and the last is 0.0073. **[FILL IN: Record who defined the 231-bin sweep and on what basis, and state whether it matches the ECADSTAR sweep settings exactly. No specification document defines the grid; it exists only as the stored array.]**

PI-distribution heatmap. The spatial view is interpolated onto a 64×64 grid. At creation it is a two-channel stack of the masked impedance field and the board mask itself. The normalized arrays that reach training are single-channel and of shape $(1, 64, 64)$, which is also the input and output width of the heatmap encoder and decoder.

[FIGURE: The three modalities for one design: 52-slot occupancy grid, 231-bin spectrum, 64x64 PI heatmap.]

Frequency anchors and conditioning

The spatial heatmap is frequency-dependent, so the dataset is built at a discrete set of inspection frequencies called anchors. The current normalized training set declares 24 PI frequency anchors in MHz: 10.0, 63.0, 80.0, 100.0, 120.0, 150.0, 170.0, 180.0, 200.0, 230.0, 250.0, 270.0, 280.0, 300.0, 330.0, 350.0, 370.0, 390.0, 400.0, 420.0, 430.0, 450.0, 470.0 and 500.0. The same list appears in the normalization statistics of that dataset.

The dataset metadata is the authoritative anchor definition and is resolved first at runtime. Two legacy definitions in the code base disagree with it – a built-in default tuple (10, 63, 80, 130, 150, 200, 250, 270, 330, 400, 500) and a configuration file listing 20 entries – and both are reached only as fall-backs, so a run that silently fell through to one of them would train against a different anchor set.

Frequency enters the model as a scalar conditioning variable, normalized in $\log_{10}$ between 1×10^6 Hz and 600×10^6 Hz and clipped to $[0, 1]$:

$$\text{PI_freq}(f) = \text{clip}\left(\frac{\log_{10} f - \log_{10}(1 \times 10^6)}{\log_{10}(600 \times 10^6) - \log_{10}(1 \times 10^6)}, 0, 1\right). \quad (2.5)$$

For the current dataset the normalization constants are a $\log_{10}$ minimum of 6.0 and a $\log_{10}$ range of 2.778151250383644. When a batch carries no frequency, the normalized value for 200.0 MHz is substituted.

Dataset scale and normalization

The normalized training dataset contains 582997 manifest rows over 24979 unique layouts, held as a layout store, and was built with a maximum-budget filter that drops layouts with $K > 30$ before statistics and outputs are computed. Per-anchor sample counts are 24979 for every anchor except 80 MHz, which has 8480.

Heatmap normalization for this dataset is a robust per-MHz $\log(1 + x)$ transform in its unbounded variant, using per-MHz median and interquartile-range statistics, a background value of -3.1792 , and recorded clip bounds of -1.6791870594024658 and 6.275787830352783 . The same three values appear in the training configuration of `exp059_capacity_freq`; the unbounded loader path applies no clip. Impedance normalization is a logarithmic z -score with $\mu_{\log} = -1.039238452911377$ and $\sigma_{\log} = 1.7831873893737793$, fitted over 24979 layouts.

Training uses a layout-level holdout: all frequencies of one design identifier go to train or to validation but never both, with a train split of 0.9 and seed 42. **[FILL IN: State why layout-level rather than row-level splitting matters here in one sentence, with the leakage argument written out.]**

2.1.3 Formulation of the Decoupling Capacitor Placement Problem

The engineering problem is to keep $Z(f)$ below a frequency-dependent target mask over 1–600 MHz sampled at 231 bins, with the design lever being the binary placement $\mathbf{b} \in \{0, 1\}^{52}$ and $K = \|\mathbf{b}\|_0$. Exhaustive search over the 2^{52} placements is intractable.

Feasibility with a safety margin

The framework does not test feasibility against the mask directly. It tests against the mask shifted inward by a constant safety margin. Writing $\hat{Z}_j$ for the predicted impedance at bin j and $Z_{\text{target},j}$ for the mask value at that bin, both expressed in the

comparison space, the search counts a candidate feasible if and only if

$$\hat{Z}_j \leq Z_{\text{target},j} - \delta \quad \text{for all } j = 1, \dots, 231, \quad (2.6)$$

with a margin of $\delta = 0.1$.

Equation 2.6 is a surrogate-predicted feasibility test, not a verified one. The quantity $\hat{Z}_j$ is the model’s prediction, so a candidate the search flags as feasible has not been simulated at that point: ground-truth confirmation requires exporting the placement and re-running the ECADSTAR PI simulation. Wherever this thesis calls a candidate feasible without saying otherwise, the word refers to Equation 2.6 on predicted impedance and is not a sign-off.

The comparison space matters for interpreting δ . The search runs by default in logarithmic space, in which $\hat{Z}$ and Z_{target} are the z -scored log-impedance, so $\delta = 0.1$ is in normalized units and is not 0.1Ω . In that mode the mask is converted by $\log Z_{\text{target}}$ followed by z -scoring with the dataset impedance log mean and standard deviation.

[FILL IN: How the value 0.1 was chosen, and what it corresponds to in ohms at a representative frequency. Nothing in the repository records the rationale, and its physical size depends on the z -scoring constants above.]

The per-budget objective

The search is run per budget K . Let $Z_j^{\text{bnd}} = Z_{\text{target},j} - \delta$ be the shifted boundary, and define the per-bin violation

$$v_j = [\max(0, \hat{Z}_j - Z_j^{\text{bnd}})]^p, \quad p = 2.0. \quad (2.7)$$

The violation is aggregated as the sum of its mean and its maximum over bins, and a reward is granted for headroom below the boundary:

$$\text{exceed} = \frac{1}{231} \sum_j v_j + \max_j v_j, \quad \text{gap} = \frac{1}{231} \sum_j \text{below}_j, \quad (2.8)$$

where below_j is the headroom of bin j below the boundary. The base objective in the default mode is

$$\mathcal{L}_{\text{base}} = -w_{\text{gap}} \cdot \text{gap} + w_{\text{exc}} \cdot \text{exceed}, \quad w_{\text{gap}} = 1.0, \quad w_{\text{exc}} = 25.0. \quad (2.9)$$

The default objective mode is the gap-and-maximum form written above; in the alternative feasibility-mapping mode the base term reduces to $w_{\text{exc}} \cdot \text{exceed}$ only.

Among the candidates that satisfy Equation 2.6 on predicted impedance, the default selection metric is the maximum predicted impedance in ohms: the candidate with the

lowest maximum is kept, and the best candidate is only ever updated for seeds flagged feasible. If no seed is feasible for a budget, the run records explicitly that no solution was found for that K rather than returning the least-bad infeasible design.

Scope of the implemented formulation

The formulation above is the one implemented in the Stage-2 latent-optimization script, which loads the VAE and the impedance surrogate of `exp038_true_multi` for both roles, and whose normalization-statistics path refers to a dataset directory that is not present in the current checkout – the live dataset is the unbounded normalized multi-frequency set described in Section 2.1.2. There is no Stage-2 entry point wired to `exp059_capacity_freq`. The formulation is therefore presented here as implemented, and no inverse-design result may be attributed to the current surrogate on this basis. **[FILL IN: Either port Stage 2 to `exp059_capacity_freq` and update the normalization-statistics path, or state in the thesis that Stage 2 is demonstrated on `exp038_true_multi` only. Also write the per- K objective against the `exp059` model if that port happens.]**

2.2 Generative Modeling and Latent-Space Optimization

2.2.1 Variational Autoencoders and Multi-modal Learning

The variational autoencoder

The variational autoencoder [13] learns a latent variable model by jointly training an encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ and a decoder $p_\theta(\mathbf{x} \mid \mathbf{z})$ against the evidence lower bound, trading reconstruction quality against a Kullback–Leibler term that pulls the approximate posterior toward a prior.

Written out, for an observation $\mathbf{x}$ and a latent vector $\mathbf{z} \in \mathbb{R}^{d_{\text{latent}}}$ with prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, the marginal likelihood $\log p_\theta(\mathbf{x})$ is intractable and training maximizes the evidence lower bound (ELBO) instead:

$$\log p_\theta(\mathbf{x}) \geq \mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{z} \mid \mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})), \quad (2.10)$$

where θ are the decoder parameters, ϕ the encoder parameters and D_{KL} the Kullback–Leibler divergence. The first term is the reconstruction term: it rewards latents from which the decoder can reproduce the input. The second is a regularizer that pulls the approximate posterior $q_\phi(\mathbf{z} \mid \mathbf{x})$ toward the prior. The gap between $\log p_\theta(\mathbf{x})$ and the

bound is $D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p_\theta(\mathbf{z} \mid \mathbf{x}))$, which is non-negative and vanishes only when the approximate posterior equals the true one [13].

The encoder emits a mean $\boldsymbol{\mu}_\phi(\mathbf{x})$ and a diagonal variance $\boldsymbol{\sigma}_\phi^2(\mathbf{x})$, and the sampling step is made differentiable by the reparameterization trick: instead of sampling $\mathbf{z}$ from q_ϕ directly, one draws $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and sets

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad (2.11)$$

with $\odot$ the elementwise product, so that the gradient of the first term of Equation 2.10 with respect to ϕ passes through the sample [13]. Equations 2.10 and 2.11 are what make the latent usable as a search domain later: the KL term keeps posteriors from spreading arbitrarily far from the prior, so the region of $\mathbb{R}^{d_{\text{latent}}}$ that carries prior mass is the region the decoder has been trained on. Section 2.2.3 relies on this, and the Stage-2 objective carries an explicit prior term and a posterior-boundary term for that reason.

The weighting of the KL term against reconstruction is the β -VAE formulation [14], and conditioning a VAE on side information follows the conditional generative framework of [15].

In `exp059_capacity_freq` the KL term is configured with `free_bits` = 0.12, β annealing disabled at a final β of 0.03, a per-expert KL weight of 0.02, and a σ -regularization weight of 1.5 toward a target of 0.45.

The first of those settings addresses a known failure mode of Equation 2.10. The KL term is minimized outright by making $q_\phi(\mathbf{z} \mid \mathbf{x})$ equal to the prior for every input. In that regime the encoder carries no information about $\mathbf{x}$ and the decoder ignores the latent altogether; the approximate posterior has collapsed onto the prior, which is the failure mode named and analysed as posterior collapse in [16]. For this framework the consequence is specific rather than merely cosmetic: Stage 2 optimizes $\mathbf{z}$ against the target mask, and a latent the decoder ignores cannot steer the decoded placement.

The countermeasure used here is free bits, or KL thresholding, introduced alongside the inverse-autoregressive-flow work of [17]. The idea is to stop penalizing the KL contribution of a latent dimension once it has fallen below a fixed threshold, so that each dimension may retain a minimum amount of information at no cost and the optimizer has no incentive to drive it to zero. The threshold used here is the value 0.12 quoted above. **[FILL IN: Confirm the exact hinge form of the free-bits objective and the units of the threshold – nats or bits, per latent dimension or per group of dimensions – directly from the PDF of [17], and check it against the exp059 implementation, before writing the equation. The formulation could not be read from the paper’s own full text for this draft,**

so only the qualitative description above is given.] [FILL IN: Explain what the σ -regularizer (weight 1.5 toward a target of 0.45) is doing here, and why the free-bits threshold 0.12, the final β of 0.03 and the per-expert KL weight 0.02 were chosen at those particular values; the configuration records the values but no note records the reasoning.]

Product-of-Experts fusion

With several modalities describing one design, the joint posterior must be formed from per-modality posteriors. The Product-of-Experts multimodal VAE [18] multiplies Gaussian experts, which for Gaussians is a precision-weighted average; the mixture-of-experts alternative is developed in [19], and [20] discusses the trade-off between the two.

The implementation follows the PoE form directly. Given experts with means μ_i and variances σ_i^2 ,

$$\sigma_{\text{poe}}^{-2} = \sum_i \sigma_i^{-2}, \quad \mu_{\text{poe}} = \sigma_{\text{poe}}^2 \sum_i \frac{\mu_i}{\sigma_i^2}, \quad (2.12)$$

and a unit-Gaussian prior expert with $\mu = 0$, $\log \sigma^2 = 0$ is always appended as an extra expert; $\epsilon = 10^{-8}$ is added to each variance and to the combined precision. Expert log-variances are clamped to $[-6.0, 1.0]$ at each expert head and the fused log-variance is clamped to $[-4.0, 2.0]$ before reparameterization.

[FILL IN: Recorded rationale for choosing PoE over MoE or another fusion scheme for this problem. Nothing beyond a description of what PoE does was found in the repository, so this is the author’s own design argument to write.]

Structured latent and expert wiring

The latent is partitioned rather than shared uniformly. Writing $\mathbf{z} = [\mathbf{z}_{\text{shared}} \mid \mathbf{z}_{\text{peak}} \mid \mathbf{z}_{\text{spatial}}]$, the widths are

$$d_{\text{shared}} = d_{\text{latent}} - d_{\text{hm priv}} - d_{\text{imp peak}}, \quad (2.13)$$

$$d_{\text{layout}} = d_{\text{shared}} + d_{\text{imp peak}}, \quad (2.14)$$

where $d_{\text{hm priv}}$ is the number of heatmap-private dimensions and $d_{\text{imp peak}}$ the number of impedance-peak dimensions. For `exp059_capacity_freq` the total latent width is $d_{\text{latent}} = 128$, with $d_{\text{hm priv}} = 40$ and a conditioning width $d_{\text{cond}} = 32$. The peak width is not set in that configuration and takes its default of $d_{\text{imp peak}} = 8$, alongside default graph settings of hidden width 128, three layers and dropout 0.1. Equations 2.13–2.14 then give $d_{\text{shared}} = 80$ and $d_{\text{layout}} = 88$.

The experts are wired to different slices of that latent. The heatmap expert predicts the full latent vector; the occupancy expert predicts only the shared dimensions; the impedance expert predicts the shared plus peak dimensions; and dimensions an expert does not own are padded with the prior ($\mu = 0$, $\log \sigma^2 = 0$) before the product is taken. The heatmap encoder input width is $64 + 2d_{\text{cond}}$ because it sees both K and the normalized frequency, while the occupancy and impedance heads take $64 + d_{\text{cond}}$ and see K only. A dedicated frequency expert maps the frequency embedding to the heatmap-private dimensions alone, with its shared and layout dimensions padded by the prior; that expert is enabled by default and is not disabled in the exp059 configuration.

[FIGURE: Latent partition and expert-to-latent wiring: which expert writes which dimensions.]

Graph encoders over slots and frequency bins

Two of the three modalities are encoded with graph neural networks in the spirit of [21] and the message-passing formulation of [22]. The occupancy GNN operates on a fixed slot graph: the 52 slots are nodes, edges connect four-neighbours on the 7×8 grid, self-loops are added, and the adjacency is symmetrically degree-normalized as $D^{-1/2}AD^{-1/2}$. Node features are the occupancy value (1), a learned slot embedding (half the hidden width) and the normalized row and column position (2), with positions normalized by 6.0 and 7.0 respectively; the graph is pooled by concatenating the mean and the max over nodes and projected to an output width of 64.

The spectrum is treated as a chain graph over its 231 bins, with optional self-loops and the same symmetric degree normalization. Its node features are the bin value (1), a learned bin embedding (half the hidden width) and a normalized bin position (1) taken from an evenly spaced grid on $[0, 1]$, pooled the same way to width 64.

Each graph convolution layer computes a linear transform of the node features plus a linear transform of the neighbour aggregate $A\mathbf{x}$, followed by LayerNorm, a LeakyReLU with slope 0.1 and optional dropout, implemented in plain PyTorch with no external graph-learning dependency. Before the latent heads, the two 64-wide features are concatenated and projected from width 128 to width 64 with LayerNorm and LeakyReLU into a joint feature that feeds both the occupancy and the impedance expert head.

Both discrete modalities are decoded by graph decoders as well: a global latent and conditioning vector is broadcast to every node, concatenated with node embeddings and positions, passed through GNN layers and read out per node – 52 logits for occupancy, 231 values for the spectrum.

Conditioning on budget and frequency

The budget is embedded with a learned table of 53 entries, covering $K \in \{0, \dots, 52\}$, and the conditioning vector handed to the heatmap path is the concatenation of the K embedding and the frequency embedding, of width $2d_{\text{cond}}$.

Frequency is embedded by a Fourier-feature MLP. The input is $[f, \sin(2\pi fb), \cos(2\pi fb)]$ for $b = 1, \dots, B_{\text{four}}$, giving an input width of $1+2B_{\text{four}}$, mapped by Linear–SiLU–Linear–SiLU to d_{cond} . In `exp059_capacity_freq` the number of Fourier features is $B_{\text{four}} = 16$, where `exp057_structured_graph` and `exp058_asymmetric_kl` use 8.

Conditioning is injected into the heatmap decoder by feature-wise linear modulation [23]. The implementation applies $x \mapsto x \cdot (1 + \gamma) + \beta$ with γ and β produced by a Linear layer from the conditioning vector, whose weights and bias are zero-initialized except that the bias entries for the γ half are filled with 1.0. **[FILL IN: Reconcile the FiLM initialization: the forward pass adds 1.0 to γ while the bias for the γ half is also set to 1.0, so the layer does not start exactly at identity. Confirm the intended behaviour against the exp043 frequency-conditioning module and state it.]**

Multi-scale conditioning is enabled, so FiLM is instantiated at four decoder scales: the 8×8 bottleneck on 128 channels, 16×16 on 64 channels, 32×32 on 32 channels, and 64×64 on 16 channels, each conditioned on the $2d_{\text{cond}}$ vector; a further FiLM layer on 32 channels sits in the heatmap encoder. The corresponding switch is not present in the `exp059` configuration file and defaults to enabled, whereas the `exp057_structured_graph` model builder has no such key at all.

The `exp059` heatmap decoder has no U-Net skip connections: it is driven by the latent and by FiLM conditioning only, which is the form Stage 2 and unconditional generation require. The `exp057_structured_graph` builder still exposes a heatmap skip-connection switch, enabled by default; the `exp059` builder has no such key.

Modality dropout and the occupancy-only path

At inference time, when a placement is being proposed, the spectrum and the heatmap are exactly the quantities that are not yet known. The encoder must therefore work from occupancy alone. Modality dropout is included so that the encoder is trained on subsets of the modalities, which is the condition the occupancy-only path faces at inference. During training each of the three data experts is independently kept with probability $1 - p_{\text{drop}}$ and otherwise replaced by the prior; if all three are dropped, one is forced back on. The frequency expert is appended after the dropout step and is never dropped, and an optional protection flag forces the heatmap expert on. In `exp059_capacity_freq` the dropout probability is $p_{\text{drop}} = 0.06$, both initially and as

the start value of the schedule, heatmap protection switches on at epoch 25, corresponding to a training fraction of 0.325, and the focus phase uses $p_{\text{drop}} = 0.03$. **[FILL IN: Quantify the effect of modality dropout on the occupancy-only path: occupancy-only reconstruction and heatmap error with and without modality dropout, from a matched pair of runs. No such ablation exists in the repository, so this draft states the design intent behind the mechanism and makes no causal claim that the path would be unusable without it.]**

The occupancy-only encode is also available as an explicit inference path: the impedance input is set to zero, the joint layout encoder is run, and the posterior is formed from the occupancy expert together with the PoE prior expert alone. This is the same path the active-learning loop uses for scoring and the occupancy-only fine-tune uses for training.

During training the decode latent is sometimes taken from a layout encode instead of the full posterior: with probability `layout_train_prob` the layout branch is used, and within that branch with probability `occ_only_encode_prob` the occupancy-only encode is used. The full encode is always run for the KL term; only the decode latent is swapped.

The live `exp059_capacity_freq` configuration sets `layout_train_prob` = 0.65 and `occ_only_encode_prob` = 0.4, while the active-learning fine-tune override sets 0.9 and 0.7. The experiment notes for the capacity study record both probabilities at 0 (full-encode path only, active learning disabled) and both distillation weights at 0, whereas the current configuration sets the latent and output distillation weights to 2.5; the notes and the configuration also disagree on the occupancy-only evaluation flag, the overlay data directory, the learning rate, the number of epochs and the resume checkpoint. **[FILL IN: Resolve the disagreement between the exp059 notes and the live exp059 configuration by stating, per reported run, which configuration was actually in force – the checkpoint’s own recorded config would settle it.]**

The layout-to-private head is a two-hidden-layer MLP over the concatenation of the 64-wide occupancy feature, the 64-wide impedance feature and the $2d_{\text{cond}}$ conditioning vector. It predicts the heatmap-private latent dimensions, optionally FiLM-modulated by the frequency embedding and squashed as $4 \tanh(x/4)$, and acts as the distillation student for the heatmap encode as teacher. Both the private head and its frequency modulation are enabled by default and neither is disabled in the exp059 configuration. The configured distillation weights are 2.5 for the latent term, a multiplier of 3.0 on the private dimensions, 0.3 for the log-variance term and 2.5 for the output term – subject to the disagreement recorded above.

Model capacity across the experiment lineage

`exp057_structured_graph` and `exp058_asymmetric_kl` used a latent width of 65, a heatmap-private width of 15, a conditioning width of 8 and 8 Fourier features; `exp059_capacity_freq` raises these to 128, 40, 32 and 16. `exp058` differs from `exp057` in the layout-path training mix and the KL settings: `layout_train_prob` 0.9 and `occ_only_encode_prob` 0.7 against 0.65 and 0.4, a free-bits threshold of 0.12 against 0.08, and a final β of 0.03 against 0.05. `exp060_multitype_occ` is a fork of `exp059` that replaces binary occupancy with a multi-type one-hot schema $\text{occ} \in \{0, 1\}^{52 \times T}$, an all-zero row meaning empty, default $T = 2$, decoded with sigmoid and focal binary cross-entropy rather than softmax cross-entropy; that description comes from hand-written notes, not from a run artifact, and `exp060` carries the same four capacity values as `exp059`.

2.2.2 Physics-Informed and Surrogate Modeling

Surrogate modelling for signal and power integrity is an established direction; [2] and [24] survey the field for packaging and for EDA more broadly. For PDN impedance specifically, [10] trains a deep network to predict impedance for arbitrary board shape, stackup, IC location and decap placement, and [25] models power-ground plane impedance with deep networks under an adaptive sampling process. Worst-case power integrity has been predicted with convolutional networks [26].

[FILL IN: One paragraph on what “physics-informed” means for this thesis specifically: which constraints are imposed as loss terms versus which are imposed structurally (hard top- K , board mask), and where that sits relative to the physics-informed learning literature. This is an interpretive framing the author should write.]

Three physics terms are implemented in `exp059_capacity_freq`. The anti-resonance shape prior of Section 2.1.1 is one. The second and third come from a physics critic, a CNN that maps the 7×8 decap occupancy grid together with the normalized inspection frequency to a spatial effectiveness map in $[0, 1]$ at 64×64 . It is supervised by inverting and min-max normalizing the heatmap foreground, and it supplies a radius-influence penalty on reconstructed heatmap pixels that are cold where decap coverage is weak. The critic is detached during the VAE step. The configured weights are radius-influence 1.0, critic supervision 2.0 and anti-resonance 0.5.

Before the heatmap is decoded, occupancy is binarized to a hard top- K zero/one vector, described in the source as CAD-aligned, optionally with a straight-through estimator. The top- K operation is vectorized through a descending-sort threshold so that a per-row variable K needs no Python loop. `exp059_capacity_freq` enables the

binary occupancy decode and disables the straight-through variant during training.

Decoder input widths follow from the latent partition: the heatmap decoder receives the full latent plus $2d_{\text{cond}}$ conditioning; the occupancy decoder receives $\mathbf{z}_{\text{shared}}$ plus the K embedding; and the impedance decoder receives $\mathbf{z}_{\text{layout}}$ plus the K embedding plus a 64-wide occupancy graph context, which is the passed occupancy if one is given and otherwise the sigmoid of the reconstructed occupancy logits.

A frequency-weighted two-dimensional FFT loss on the heatmap is also implemented, computing an L_1 distance between $\log(1 + x)$ magnitudes of the real FFT of the reconstruction and of the target, with an optional Gaussian bump in MHz. It is not active in `exp059_capacity_freq`: its weight key is absent from the configuration and from the configuration dataclass, so the lookup falls back to 0.0 and the term is skipped. The experiment notes state the opposite, namely a weight of 1.5, a mid-boost of 1.5, a centre of 200 MHz and a width of 80 MHz. The configuration is taken as authoritative, and this thesis does not describe the spectral loss as an active training term.

Off-anchor evaluation

Because heatmaps exist only at the 24 anchors, generalization between anchors has to be measured explicitly. `exp059_capacity_freq` declares evaluation frequencies of 155.0, 250.0 and 265.0 MHz with weights 2.5, 3.0 and 2.5 under the key `eval_off_anchor_mhz`. That key name is not literally accurate: 250.0 MHz is itself one of the 24 training anchors, so only 155 and 265 MHz are genuinely off-anchor. Any evaluation reported over the three-frequency set therefore mixes one in-distribution anchor with two held-out frequencies, and Chapter 4 and Chapter 5 must state which of the two readings is meant.

Decoding at a non-anchor frequency is also available in blended form, by log-blending the decodes at the two bracketing training anchors, $\hat{h} = (1 - w)\hat{h}_{\text{lo}} + w\hat{h}_{\text{hi}}$ with the bracketing pair and the weight w determined by the requested frequency. That blend is a separate method; the training and evaluation path decodes directly at the requested normalized frequency.

Active learning as a data-efficiency mechanism

Because each label costs an ECADSTAR simulation, the framework retrains on selected new layouts rather than on uniformly sampled ones. The general framing is pool-based active learning [27]; uncertainty for a deep model can be obtained from MC dropout [28] or from ensembles [29], and batch-mode selection for regression without an ensemble is treated in [30]. Gaussian-process bandit optimization [31] is the reference point for the

residual-GP acquisition used in this work. The exp059 candidate generator searches budgets K from 3 to 10, with 1600 candidates per K , 80 worst per K , 640 total ECAD simulations per cycle, a heatmap grid size of 64, and an evaluation frequency grid of [120, 150, 155, 170, 180, 200, 230, 250, 265, 270, 280, 300] in MHz. The loop itself is described in Chapter 3, not here.

2.2.3 Optimization in Latent Representation Spaces

The idea of optimizing a design by moving through a learned continuous latent space and decoding the result was established outside electronics by [32], which searches a VAE latent space of molecules and decodes candidates back to discrete structures.

The argument for doing this is that it replaces a search over discrete objects with a search over a continuous vector. The set $\{0, 1\}^{52}$ of placements supports no gradient: an objective evaluated on it can only be probed by flipping bits according to some hand-chosen move set, and there is no direction in which to step. Encoding designs into $\mathbb{R}^{d_{\text{latent}}}$ and decoding them back yields a domain in which a candidate can be moved by a differentiable update instead. [32] states this property directly for molecules: continuous representations “allow the use of powerful gradient-based optimization to efficiently guide the search for optimized functional compounds”. This thesis applies the same argument to decap placement, subject to the qualification in the next paragraph.

The obstacle for PDN placement is that the decoded object is discrete. A placement must be a 0/1 vector with exactly K ones before it can be handed to CAD, and the top- K selection that produces it is not differentiable. The standard remedy is the straight-through estimator [33]; continuous relaxations of discrete variables such as Gumbel-Softmax [34] and the Concrete distribution [35] are the alternative family.

The implementation uses a straight-through top- K read-out. With $\mathbf{p}$ the soft occupancy probabilities and $\mathbf{b}_{\text{hard}}$ the K -hot vector obtained by selecting the K largest entries,

$$\tilde{\mathbf{b}} = \mathbf{b}_{\text{hard}} + \mathbf{p} - \text{sg}(\mathbf{p}), \quad (2.15)$$

where $\text{sg}(\cdot)$ denotes the stop-gradient operator. The forward pass therefore sees the hard K -hot vector while the backward pass passes identity gradient to the probabilities. Without it the sorting operation behind the top- K selection severs the gradient and $\mathbf{z}$ never moves in response to the spectrum target. This is the structural reason the estimator is required here, and Section 2.3.3 returns to it.

With the decoder frozen, the search runs Adam over $\mathbf{z}$ per budget K with parallel seeds. The configured settings are budgets $K = 1, \dots, 25$, 32 candidate seeds, 1200 steps, a learning rate of 5×10^{-2} and gradient clipping at 5.0. The total objective adds to the base term of Equation 2.9 a latent prior term, a spectrum shape regularizer,

a posterior-boundary term, a track term (MSE to target), the anti-resonance physics term and a peak / dual-top- K alignment term; with more than one seed it further adds a diversity term and an occupancy-confidence term:

$$\begin{aligned}\mathcal{L} = \overline{\mathcal{L}_{\text{base}}} &+ 0.05 \mathcal{L}_{z\ell_2} + 2.0 \mathcal{L}_{\text{shape}} + 10.0 \mathcal{L}_{\text{post}} + 2.0 \mathcal{L}_{\text{track}} \\ &+ 0.5 \mathcal{L}_{\text{AR}} + w_{\text{peak}} \mathcal{L}_{\text{peak}} (+ 0.35 \mathcal{L}_{\text{div}} + 0.5 \mathcal{L}_{\text{conf}}),\end{aligned}\tag{2.16}$$

with the posterior-boundary term limiting $\mathbf{z}$ to 2.5 posterior standard deviations. **[FILL IN: Insert the configured peak-loss weight w_{peak} from the Stage-2 optimization script, and give the functional form of the shape, track, posterior-boundary, peak, diversity and occupancy-confidence terms. Only the weights were extracted, not the formulas.]**

Two implementation details qualify what Equation 2.16 optimizes against. The impedance path defaults to a separate occupancy-to-spectrum surrogate rather than the VAE impedance head; that surrogate is a direct map from an occupancy vector of length 52 to a log- z spectrum of shape (1, 231) with hidden widths (256, 512, 512, 512, 256), and it does not use the normalized inspection frequency at all. And, as recorded in Section 2.1.3, the whole script is wired to `exp038_true_multi`.

[FIGURE: Stage-2 loop: $\mathbf{z}$ init, decode to soft occupancy, STE top- K , surrogate spectrum, loss against the mask, Adam step.]

2.3 Related Work and Research Gap

2.3.1 Power Delivery Network Optimization Methods

Target impedance and heuristic search

Target-impedance-based methodology and capacitor selection for CMOS PDNs is described in [1]. Board-level decap placement has been attacked repeatedly with heuristic and evolutionary search. The titles record a nature-inspired algorithm [36], a genetic algorithm minimizing the number of capacitors [37], and an iterative genetic algorithm combined with machine learning [38]; simulation-driven module generation for decap placement is reported in [4]. Only the bibliographic titles of those four could be verified for this draft, so nothing is asserted here about the evaluator each one queries. **[FILL IN: Confirm from each of electronics2019nature, ga2020minimum, electronics2020iterative and ars2023ai what the evaluator is (a field solver or a learned model) and whether it is queried for a scalar score without a gradient being taken through it. Only then can a shared black-box-search structure be claimed for this group.]**

The one work in this group whose search mechanism could be verified is [5], and it is also the closest complete instance to the present framework: a graph-convolutional impedance surrogate wrapped in a genetic algorithm that queries the surrogate’s scalar value, with no gradient taken through the surrogate. The distinction to hold onto is not the surrogate, which both approaches have, but what is done with it – a GA samples and scores, whereas this thesis differentiates through the surrogate and moves a latent vector.

Impedance surrogates

[10] predicts PDN impedance for arbitrary board shape, stackup, IC location and decap placement, trained on more than one million boundary-element-method boards. Its output is an impedance prediction; the placement is an input to it, not something the model proposes. [25] models power-ground plane impedance with deep networks and an adaptive sampling process, and [26] predicts worst-case power integrity with a CNN. Those two are cited here from their titles only. **[FILL IN: Confirm from goay2022adaptive and todaes2022worst whether either proposes a placement rather than only predicting a quantity, or drop the joint reading of this group as prediction-only surrogates.]**

Reinforcement learning for decap placement

A distinct line, largely from KAIST, learns a placement *policy* over discrete actions. [7] applies deep RL to decap design on silicon interposers for 2.5D/3D ICs, meeting a target impedance with minimum area. [39] trains a pointer-network policy with REINFORCE as a sequence-to-sequence model over candidate decap slots, handling arbitrary probing ports and keep-out regions. [8] uses an attention-based RL policy that parameterizes decap choice at multiple ports of an HBM PDN, and reports minutes against more than a month for a full search. [9] generalizes the earlier pointer-network work of the same first author into a symmetric transformer with permutation-symmetry and relative-position inductive biases, and reports more than 30% fewer decaps than prior art on real hardware. Those two figures are as reported by the respective papers; they were obtained on those authors’ own boards and are not comparable to any number in this thesis.

Outside that group, [40] reports hierarchical decap optimization for the PDN of 2.5D ICs based on deep reinforcement learning. **[FILL IN: Verify duan2024hierarchical against the paper itself. Only its bibliographic title was available for this draft, so no statement is made here about its action space, reward, evaluator or how the frequency- and time-domain co-analysis enters the method.]**

2.3.2 Generative and Learning-Based Design Approaches

Predicting a spatial field of the power network as an image is now a small literature of its own. [41] uses encoder-decoder networks to produce on-chip static and dynamic IR-drop maps conditioned on layout, which legitimizes decoding a PI field as an image rather than as a scalar. [42] combines a GNN over the PDN with a CNN for dynamic IR-drop prediction, which is independent external evidence that the graph structure of the network carries information a purely spatial view does not. [43] is architecturally the closest neighbour of this thesis’s heatmap decoder: a conditional multimodal generative framework fusing image and graph features into a conditional diffusion process for spatial IR-drop maps. Two differences should be stated precisely rather than glossed: it is a diffusion model where this thesis uses a VAE decoder, and it conditions on layout where this thesis conditions on inspection frequency. All three operate on on-chip IR drop, whereas the modality here is a board-level PI distribution at a given frequency.

Learning-based EDA more broadly is surveyed in [24] and, for packaging-level SI/PI, in [2]. Placement as a learned decision problem at chip level appears in [44], and graph/transformer models over interconnect in [45].

2.3.3 Research Gap and Motivation for the Proposed Framework

Across the works surveyed above, no paper was found that combines a multi-modal generative surrogate over occupancy, spectrum and spatial PI distribution with decoder-mediated latent-space gradient optimization through a discrete top- K read-out for PDN decap placement. That statement is a claim about the literature searched for this thesis, not a claim of superiority over any of it: none of the comparisons below is a performance comparison, and no number in this thesis is benchmarked against a number in those papers.

Distinction from the differentiable-surrogate line

The methodologically closest work is the differentiable-surrogate line from the examiner’s own group. [46] performs amortized neural optimization for pre-layout signal-integrity design-space exploration, mapping a context to near-optimal design parameters in a single deterministic forward pass through a differentiable surrogate. [47] builds buffer-parameterized surrogate models over a 44-parameter design space and benchmarks neural, Gaussian-process and tree-based surrogates for cross-technology SI analysis and optimization. [48] takes a deliberately non-gradient-based route: a deterministic decision-tree filter followed by an Earth Mover’s Distance ranking over

continuous design-parameter predictions, explicitly avoiding both inverse modelling and stochastic search.

The distinction rests on a targeted reading of all three, checking each for any mention of a latent or embedding space, of discrete design variables, of top- K or cardinality-constrained selection, and of a straight-through or relaxed gradient estimator. None of the three contains any of them: the optimization variable in each case is a vector of continuous parameters in the original design-parameter space – equalizer and buffer parameters, PCB parameters, interconnect design variables. This thesis instead optimizes a latent vector $\mathbf{z}$ that is decoded to a placement, and the read-out from the decoder to the placement is a hard top- K selection over 52 discrete slots. That is precisely why the straight-through estimator of Equation 2.15 is structurally necessary here and is absent there: with a continuous parameter vector there is nothing between the optimizer and the surrogate to break the gradient, whereas here the top- K selection would sever it.

Two further differences are worth stating without inflating them. Those works address signal integrity; this one addresses power integrity, which the thesis scope restricts it to. And the amortized formulation of [46] produces a design in one forward pass, whereas the Stage-2 formulation here runs an iterative search per budget K (Section 2.2.3). **[FILL IN: Say explicitly whether the amortized single-pass property is a disadvantage of this thesis’s approach in inference cost, and if so by how much. That needs a timing measurement of the Stage-2 loop, which has not been recorded.]**

Distinction from the reinforcement-learning paradigm

The KAIST line – [7] on interposers, [39] with a pointer network and REINFORCE, [8] on HBM, and [9], which generalizes the first of those two given the shared first author – is a third paradigm, distinct from both the differentiable-surrogate line and from this thesis. Those methods learn a placement policy over discrete actions or action sequences. There is no gradient search through a differentiable surrogate, and there is no generative latent space to search in at all: the model outputs actions, not a representation that is decoded. The practical consequence is a difference in what has to be redone when the requirement changes. For [9] the literature survey for this thesis records a policy that must be retrained per target scenario class. On the other side, the framework here retargets to a new $Z_{\text{target}}(f)$ by re-running the latent search with the decoder frozen, as implemented in the Stage-2 script against `exp038_true_multi`. That retargeting has not been exercised against the `exp059` surrogate, for the wiring reason given in Section 2.1.3, so it is a property of the implemented Stage-2 script and not a reported result. **[FILL IN: Confirm from park2020drldcap, kim2021epeps**

and park2022transformer whether a change of target impedance mask requires policy retraining, and cite the specific statement in each. Only kim2023devformer is recorded as requiring per-scenario retraining, so the re-targeting contrast is deliberately not generalized to the whole reinforcement-learning line here.]

The closest use of latent representations in SI/PI

The nearest hit in the SI/PI literature for the phrase “latent representations” combined with optimization is [49]. It learns latent representations of DRAM equalizer eye diagrams as a fast evaluation feature, then runs an advantage actor–critic agent that optimizes the raw equalizer parameters using that latent representation as a cheap fitness signal. The latent space there is a frozen evaluation feature consumed by a policy over raw parameters. It is never itself the optimization variable, and nothing is decoded back through it. That is the exact distinction this thesis rests on, and it holds even for the closest neighbour found.

Distinction from surrogate-plus-search and from spatial-map models

Against [5], the difference is the use made of the surrogate: a genetic algorithm queries it for a scalar and searches around it, whereas here the gradient is taken through it. Against [10], the difference is in the data regime: it generalizes across many boards using cheap boundary-element labels, whereas this thesis trains one board deeply on more expensive ECADSTAR labels, which is the direct motivation for the active-learning loop rather than for broader sampling. Against the IR-drop spatial-map line [41]–[43], the difference is both the quantity modelled – board-level PI distribution at a chosen inspection frequency, against on-chip IR drop – and the role of the model: those works predict a field, whereas here the spatial decoder is one branch of a joint generative model whose latent is subsequently optimized.

Absence of a shared benchmark

No shared benchmark exists across any of these PDN decap-placement works. The Götze-group papers, the KAIST RL line, DevFormer, [5] and [10] each use their own board, port definition and target mask. None of their reported numbers is therefore directly comparable to any result in this thesis, and no such comparison is made. A fair quantitative comparison would require re-implementing at least one of those baselines on the 52-slot board and the 231-bin mask used here. **[FILL IN: Decide and state which single baseline, if any, will be re-run on this board for Chapter 5, and**

what the equal-budget protocol would be. As of this draft no baseline has been re-run.]

From the gap to the research questions

[FILL IN: The author's own statement of the research questions and hypotheses that follow from this gap, agreed with the supervisor. This is deliberately not drafted: the questions the thesis commits to answering are not recorded anywhere in the repository.]

[FILL IN: One paragraph connecting the gap to the three contributions claimed in Chapter 1, once those are finalized, with a cross-reference to the corresponding chapter for each.]

Chapter 3

Design and Implementation

This chapter describes the framework as it was built and configured. It covers the construction and normalization of the multi-frequency dataset, the multi-modal variational autoencoder that serves as the power-integrity surrogate, the latent-space optimization stage that searches for decoupling capacitor placements, the experimental setup, and the active-learning loop that extends the training set with newly simulated layouts. The subject is the method and its implementation; results are reported in the following chapter and are not anticipated here.

Notation is fixed as follows. A placement is a binary vector $\mathbf{b} \in \{0, 1\}^{52}$ over the 52 decap slots of the board, the decap budget is $K = \sum_i b_i$, and the power-integrity response is the input impedance magnitude $Z(f)$ sampled on 231 frequency bins spanning 1–600 MHz. The target impedance mask against which a placement is judged is $Z_{\text{target}}(f)$. The spatial power-integrity distribution at a single inspection frequency is a heatmap H on a 64×64 grid. **[FILL IN: describe the board under study (H-shape test vehicle) in one paragraph here: outline, stackup, plane pair and port definition. The active-learning configuration names the ECADSTAR design file H-shape.erf with powerbus Power_GND and port IC1_Port1, but no committed artifact documents the geometry itself.]**

3.1 Data Preparation and Reprocessing

3.1.1 Raw Power-Integrity Simulation Outputs and File Structure

Ground truth is produced by ECADSTAR power-integrity simulation. A processing stage converts the raw export into a layout store made of three parallel trees — per-design layout arrays, per-sample heatmaps and per-sample spectra — together with a manifest index (`manifest.csv`).

Per design the store holds an occupancy vector of shape (52,) and an impedance spectrum of shape (231,). These two shapes are the fixed interface of the whole framework: the occupancy graph encoder is built over 52 slots and the impedance graph encoder over 231 bins. A manifest row is one (design, inspection frequency) pair, so a layout appears once per anchor frequency, each time with its own spatial heatmap, while occupancy and the full spectrum are properties of the layout alone.

[FILL IN: describe the ECADSTAR simulation setup behind the raw export: solver settings, excitation, meshing, and how the spatial heatmap is exported. Only the active-learning batch invocation of the simulator is documented; the original bulk data generation is not described in any committed artifact.]

[FILL IN: confirm the stored heatmap resolution as a dataset property. The active-learning configuration sets a heatmap grid size of 64 and the decoder’s final conditioning stage operates at 64×64 , but the dataset metadata records no heatmap shape field, so the resolution is inferred from the model and the pipeline configuration rather than read from the dataset.]

[FIGURE: Data flow from the ECADSTAR export through the processing stage to the layout store (layout arrays, heatmaps, spectra, manifest).]

3.1.2 Multi-frequency Dataset Construction and Normalization

The normalized training set used throughout this work is `datasets/data_multifreq_train_norm_un`. Its metadata records the stage as `normalized`, names `pipelines/normalize/multifreq.py` as the generating script, and carries the generation timestamp `2026-07-06T13:39:03.768907+00:00`. The set contains 582997 manifest rows over 24979 unique layouts and 24979 layout directories, with 582997 heatmap files and 582997 spectrum files.

The dataset is multi-frequency in the sense that each layout is simulated at a set of anchor inspection frequencies. Twenty-four anchors are recorded, in MHz: 10.0, 63.0, 80.0, 100.0, 120.0, 150.0, 170.0, 180.0, 200.0, 230.0, 250.0, 270.0, 280.0, 300.0, 330.0, 350.0, 370.0, 390.0, 400.0, 420.0, 430.0, 450.0, 470.0 and 500.0. Coverage is not uniform. Every anchor carries 24979 samples except 80 MHz, which carries 8480.

[FILL IN: state whether the reduced 80 MHz coverage (8480 versus 24979) is deliberate or the residue of an interrupted simulation batch.] The anchor set is reported inconsistently across the project’s internal documentation, which names sets of sixteen and of six anchors; the twenty-four values above are those recorded in the metadata of the dataset actually consumed at training time, and are the ones used here. **[FILL IN: reconcile the anchor sets recorded in `dataset_meta.json`**

(24), docs/dataset.md (16) and docs/normalization-and-losses.md (six), and correct the two documents, which predate exp059.]

Before statistics are fitted, the build applies a decap-budget filter at $K \leq 30$. The budget is recovered per design as the number of occupancy entries above 0.5, and layouts exceeding the limit are dropped both from the statistics fit and from the written output.

Three modalities are normalized separately.

Heatmaps. The normalization mode is robust, per-MHz, applied in $\log(1+x)$ space and unbounded. The forward and inverse transforms are

$$z = \frac{\log_{10}(\Omega) - \text{median}_{\text{MHz}}}{\text{IQR}_{\text{MHz}}}, \quad \Omega = \expm1(z \cdot \text{IQR}_{\text{MHz}} + \text{median}_{\text{MHz}}), \quad (3.1)$$

where Ω is the physical impedance of a heatmap pixel in ohms and the median and interquartile range are fitted per anchor frequency. Two entries illustrate the spread: at 10.0 MHz the median is 0.05887173116207123 and the IQR is 0.03866304084658623, while at 500.0 MHz the median is 1.3578120470046997 and the IQR is 0.6529430150985718. The per-anchor statistics therefore differ by more than an order of magnitude across the anchor set. Global quality-control references are stored alongside them: a lower clip of -1.6791870594024658 , an upper clip of 6.275787830352783 , a maximum z of 11.23223876953125 and a background value of -3.1792 . In the unbounded mode these clip values serve as references and not as training bounds. The experiment configuration mirrors the same constants and does not clip reconstructions to that band.

Foreground and background are separated by a physical threshold of 0.05Ω . Robust clip percentiles of 0.5 and 99.5 are defined in the same normalization stage but take effect only in the bounded mode. Two build-time switches select between these behaviours: robust per-MHz statistics are enabled by default, the unbounded transform is not, so the build that produced this dataset selected the unbounded mode explicitly.

Impedance spectra. The 231-bin spectrum is normalized by a single global log z -score with mean -1.039238452911377 and standard deviation 1.7831873893737793 , fitted over 24979 layouts. The resulting range spans -2.7170498371124268 to 2.474900722503662 .

Inspection frequency. The scalar conditioning frequency is normalized in $\log_{10}$ space with an offset of 6.0 and a range of 2.778151250383644 . The stored band edges are 1000000.0 Hz and 600000000.0 Hz, which is the 1–600 MHz span quoted throughout this thesis.

[FILL IN: state why the heatmap uses per-MHz robust statistics while the impedance spectrum uses a single global log z -score. Both choices are

visible in normalization_stats.json; the reasoning is not recorded.]

3.1.3 Data Splitting, Sampling, and Batch Construction

The holdout is taken at layout level rather than at row level. Every row belonging to a design — that is, all of its anchor frequencies — is assigned entirely to either the training or the validation partition, in the ratio 0.9, and the assignment is deterministic given the split seed and the sorted design keys. The default split seed is 42.

The motivation is recorded in the implementation itself. An earlier version split frequencies within a design, which placed the same layout on both sides of the holdout and, in the words of that code, “leaked layout identity into val and is unsuitable for GP / AL residual evaluation”. The change is dated 2026-07-20 and required a fresh retrain from scratch, with the prior artifacts archived. The experiment notes give the resulting partition as approximately 22481 training and 2498 validation layouts; those two figures are stated with a tilde in the notes and are not emitted by any artifact. **[FILL IN: recompute the exact train/val layout counts for split seed 42 and a 0.9 split, and cite the emitting artifact rather than the approximate figures in notes.md.]**

Batch construction departs from what the configuration file appears to promise. A K -stratification flag is set, but in this code path it only causes the budget of each row to be precomputed; the weighted sampler is instantiated only when K -balancing or frequency-balancing is requested, and both of those are disabled. Neither stratification nor balancing is therefore active in exp059. **[FILL IN: decide whether the intended K -stratified sampling was ever exercised, or whether the flag should be removed; the configuration key and the dataloader behaviour disagree.]**

With no balancing sampler active and an epoch budget of 64000 rows, smaller than the training pool, each epoch draws its rows at random with replacement. Batches additionally carry cross-frequency pairs: a lookup table maps each training index to alternate-MHz rows of the same design, and the collate function attaches the paired row to the batch. This pairing is what the cross-frequency consistency loss of Section 3.2.2 consumes.

Loader settings are a batch size of 224 per GPU, a validation batch size of 320, four worker processes and a prefetch factor of 2, with neither persistent workers nor in-RAM caching.

3.2 Multi-modal Variational Autoencoder as a Generative Surrogate

3.2.1 Architectural Design

The surrogate is a variational autoencoder [13] whose input modalities are fused by a Product of Experts [18] and which is conditioned on the decap budget K and the inspection frequency in the manner of a conditional VAE [15]. The `exp059` model class, `MultiInputVAEPoeFreq`, extends the shared multi-input VAE with Product-of-Experts fusion, adds a frequency expert that writes the private latent dimensions, and removes the U-Net skip connections.

Structure of the latent space. The latent dimension is 128, of which 40 dimensions are private to the heatmap path. Conditioning vectors are 32-dimensional and the frequency encoder uses 16 Fourier bands. The impedance-peak block is not set in the experiment configuration and takes its constructor default of 8 dimensions. The latent is partitioned as $z = [z_{\text{shared}} \mid z_{\text{peak}} \mid z_{\text{spatial}}]$ with

$$d_{\text{shared}} = d_{\text{latent}} - d_{\text{hm-priv}} - d_{\text{imp-peak}}, \quad d_{\text{layout}} = d_{\text{shared}} + d_{\text{imp-peak}}, \quad (3.2)$$

which for these values gives a shared block of 80 dimensions and a layout block of 88.

Decoder input widths follow the partition. The heatmap decoder takes the full latent together with the concatenated 64-dimensional conditioning vector. The occupancy decoder takes only the 80-dimensional shared block plus a 32-dimensional conditioning vector. The impedance decoder takes the 88-dimensional layout block $[z_{\text{shared}} \mid z_{\text{peak}}]$, a 32-dimensional conditioning vector, and a 64-dimensional occupancy graph context. A placement is thus forced through the shared block alone, spectrum-peak structure is given a private sub-block, and frequency-specific spatial detail is carried in z_{spatial} , which only the heatmap decoder reads.

Product-of-Experts fusion. Each modality contributes a Gaussian expert $\mathcal{N}(\mu_m, \sigma_m^2)$ and the fused posterior takes the standard Product-of-Experts form: the combined precision is the sum of the expert precisions and the combined mean is the precision-weighted mean of the expert means [18]. A unit-Gaussian expert ($\mu = 0, \log \sigma^2 = 0$) is always appended as an anchor, so the product remains well defined when only one modality is present. Expert log-variances are clamped to $[-6.0, 1.0]$ and the fused log-variance to $[-4.0, 2.0]$. **[FILL IN: state why Product-of-Experts was chosen over mixture-of-experts fusion [19] for this problem; the trade-off is discussed in [20] but the decision is not recorded for exp059.]**

Modality encoders. The occupancy expert is a graph encoder over the 52 decap slots laid out on a 7×8 grid, with four-neighbour edges, optional self-loops and symmetric degree normalization, following the graph-convolution formulation of [21]; it is implemented in plain PyTorch without a graph library. The impedance expert is a chain-graph network over the 231 spectrum bins — edges between consecutive bins, self-loops, symmetric normalization — with a learned per-bin embedding, so neighbouring frequency bins exchange messages in the sense of [22]. The graph hyperparameters are not set in the experiment configuration; the applied defaults are a hidden width of 128, three layers and dropout 0.1. The 64-dimensional occupancy and impedance features are fused before the latent heads by a linear map from 128 to 64 followed by layer normalization and a leaky ReLU of slope 0.1. The heatmap expert is a convolutional encoder with an 8×8 spatial bottleneck and dropout 0.05.

Conditioning on budget and frequency. The budget is conditioned through a discrete embedding over the 53 possible values $K \in \{0, \dots, 52\}$ into the 32-dimensional conditioning space. The inspection frequency is conditioned through a `FreqConditioner`: sinusoidal Fourier features over 16 bands are concatenated with the raw normalized frequency, giving an input of dimension $1 + 2 \times 16$, and passed through a two-layer SiLU network to 32 dimensions. The vector handed to the heatmap path is the concatenation of the budget and frequency embeddings, of size 64. The FiLM and frequency-conditioning modules are imported from an earlier experiment fork (exp043) rather than defined within exp059, so the exp059 stack is not self-contained in the way the other forks are. **[FILL IN: list every cross-experiment import in the exp059 code directory (a grep for “experiments.exp0” over that directory settles it); an earlier draft claimed the frequency-conditioning import was the only one, which is not established.]**

Frequency modulation of the decoder uses FiLM [23], applied as

$$\text{FiLM}(x) = x \odot (1 + \gamma) + \beta, \quad (3.3)$$

where γ and β are produced from the conditioning vector by a projection whose weight and bias are zero-initialised except for bias[: C] = 1.0, so that the layer starts close to the identity. Heatmap FiLM is enabled in the configuration. Multi-scale FiLM is also active, but its switch does not appear in the experiment configuration and takes a code default of true — the architectural change that characterises exp059 is therefore governed by a default rather than by the experiment’s own configuration file. With it enabled, the heatmap decoder is modulated at four scales: 128 channels at the 8×8 bottleneck, 64 channels at 16×16 , a 32-channel stage, and 16 channels at 64×64 .

Decoder path and spatial injection. The U-Net skip connections were removed, and the reason is stated in the code: “No U-Net skips: decoder is latent+FiLM only (matches stage-2 / generation)”. The corresponding fusion modules were deleted. Spatial information about the placement instead enters through an occupancy spatial tower, which scatters the 52-dimensional occupancy onto the 7×8 grid and fuses occupancy features into the heatmap decoder at 16×16 , 32×32 and 64×64 .

Layout-private head. A layout-private head predicts the heatmap-private latent dimensions from occupancy, impedance and the conditioning vector, and acts as the distillation student for the full-encode teacher. Its hidden width is 384, a frequency FiLM is applied to the predicted mean, and the output is squashed as $\mu_{\text{priv}} \leftarrow 4.0 \tanh(\mu_{\text{priv}}/4.0)$.

Modality dropout. During training a Bernoulli keep mask is drawn over the three experts — heatmap, occupancy and impedance — and dropped experts are replaced by the prior. At least one expert is always kept, and the heatmap expert is force-kept once its protection stage is reached. The dropout probability is 0.06, annealed from a starting value of 0.06 over 18 epochs, with heatmap protection from epoch 25 and a reduced value of 0.03 during the heatmap-focus phase. The design intent is to keep the encoder usable when experts are missing at inference: occupancy is the only modality available for a placement that has been proposed but not yet simulated, and the active-learning loop scores such placements through the occupancy-only encode path. **[FILL IN: cite an ablation or per-path evaluation showing that the occupancy-only encode path degrades without modality dropout. No such artifact exists, so the enabling role of this term is an untested design assumption rather than a measured effect.]**

Discrete occupancy read-out. Occupancy is decoded to a hard top- K binary vector before the heatmap decode, so that the spatial prediction is conditioned on a CAD-realizable placement rather than on a soft one. The read-out is fully vectorised, using a descending-sort threshold that supports per-row variable K and avoids Python loops and device synchronisation on the decode path. A straight-through mode is available, in which the forward value is the hard top- K vector and the gradient flows through the soft occupancy [33]; in exp059 training that mode is off.

Encode paths available at inference. The model exposes several encode paths. Four matter here: encoding from all modalities; cross-modal encoding from the heatmap, the impedance, or occupancy and impedance together; encoding of the layout latent from occupancy and impedance with the private head; and `encode_occupancy_latent`,

which encodes from occupancy alone. The last is the path on which the active-learning loop and any forward design query depend.

[FIGURE: Block diagram: three modality experts, PoE fusion with prior anchor, structured latent split, FiLM-conditioned heatmap decoder with occupancy spatial tower.]

3.2.2 Objective Functions and Physics-Informed Regularization

Reconstruction terms. The reconstruction objective is a weighted sum over the three modalities plus an occupancy consistency term,

$$\mathcal{L}_{\text{recon}} = w_{\text{hm}}\mathcal{L}_{\text{hm}} + w_{\text{occ}}\mathcal{L}_{\text{occ}} + w_{\text{imp}}\mathcal{L}_{\text{imp}} + w_{\text{occK}}\mathcal{L}_{\text{occK}}, \quad (3.4)$$

with heatmap weight 5.0, occupancy weight 7.0, impedance weight 4.0 and occupancy- K consistency weight 2.0. A heatmap-focus phase beginning at epoch 25 overrides these to heatmap 7.0, impedance 4.0, cross-frequency 1.1 and dynamic range 3.0. **[FILL IN: give the design rationale for the numeric loss weights (5.0 / 7.0 / 4.0 / 2.0 and the focus-phase overrides). The experiment notes explain the capacity and FiLM changes of exp059 but not how these weights were arrived at; a weight-sweep artifact would settle it.]**

Occupancy reconstruction uses a focal binary cross-entropy with focusing parameter 1.5 and a positive-class weight $(52 - K)/K$ clamped to $[0.5, 10.0]$, which compensates for the strong class imbalance of a small- K binary target. The K -consistency term is a hinge that penalises $\text{relu}(0.5 - \ell)$ on target-on slots and $\text{relu}(\ell + 0.5)$ on target-off slots, where ℓ is the decoded occupancy logit.

The impedance loss is a composite over the 231 bins. Its components are a derivative term of weight 1.4; a top- K term over the 20 largest bins with weight 4.5; an under-prediction penalty of 2.8; a concavity term of 2.5; a frequency-weighting exponent of 2.0; a dual top- K term of 0.75; and peak-index and peak-magnitude terms of 1.5 and 2.0 over the 8 strongest peaks. The peak terms are ramped in rather than applied from the start, beginning at epoch 25 and reaching full weight over 22 epochs, with the peak-focus phase also starting at epoch 25.

Heatmap reconstruction adds several shape-sensitive terms to the pixel loss: a Pearson-correlation term of weight 0.75, a gradient-vector term of 2.0 and a gradient-direction term of 1.0, the latter two with a minimum gradient magnitude of 0.08 and a Huber delta of 0.5. Extreme percentiles are treated separately by top-region Huber terms at the 0.95 and 0.05 quantiles, each of weight 2.5 and Huber delta 0.5. A soft-argmax peak-location loss carries weight 2.5 at temperature 0.035; the corresponding

valley-location loss is disabled with weight 0.0.

Latent regularization. The KL divergence is computed per dimension as

$$\text{KL}_j = -\frac{1}{2} \left(1 + \log \sigma_j^2 - \mu_j^2 - \exp(\log \sigma_j^2) \right), \quad (3.5)$$

clamped from below by a free-bits floor of 0.12 before averaging, so that a dimension carrying less than that much information is not pushed further toward the prior. Beta annealing is disabled and β is fixed at 0.03, giving a fixed β -weighted KL in the sense of [14]. Three further latent regularizers enter the total: a hinge $w_{\text{hinge}} \text{relu}(|\mu| - \tau)^2$ with weight 0.1 and threshold 4.0; a batch-mean bias term $w_{\text{bias}} \bar{\mu}^2$ with weight 0.05; and a per-expert KL averaged over experts with weight 0.02. A sigma regularizer of weight 1.5 keeps the fused posterior standard deviation inside a band around a target of 0.45, penalising values below the target and above $1.3\times$ the target. **[FILL IN: explain the choice of sigma regularizer target 0.45 and free-bits floor 0.12; no artifact records how they were selected.]**

Cross-frequency consistency, distillation and jitter. Cross-frequency training decodes the latent at an alternate MHz value and compares the result with the paired alternate-MHz heatmap supplied by the pair lookup of Section 3.1.3. Its weight is 1.2 from epoch 20, with a layout-mixing probability of 0.45, and it re-uses the encoded latent. Distillation transfers the full-encode teacher onto the layout student with a latent term of weight 2.5 — the private dimensions multiplied by 3.0 and the log-variance component by 0.3 — and an output term of weight 2.5, implemented as a Huber loss between the layout decode and the detached teacher decode. Frequency jitter is applied on the decode side with probability 0.6 and a $\log_{10}$ standard deviation of 0.1.

Spectral anti-blur term, inactive in this experiment. A two-dimensional FFT log-magnitude L_1 anti-blur loss is implemented, together with an optional Gaussian bump that would emphasise a mid-band frequency window. It does not act in exp059. Its weight key is absent from the experiment configuration, so the weight lookup returns its default of 0.0, and the function returns nothing for any non-positive weight; the term never enters the total loss. This has to be stated, because the experiment notes present the frequency-weighted spectral loss as one of the three headline changes of exp059 and quote a weight of 1.5, a mid-band boost of 1.5, a centre of 200 MHz and a width of 80 MHz. Those keys do not exist in the configuration as it stands. **[FILL IN: reconcile the experiment notes, which list the spectral anti-blur loss as a headline exp059 change, with the configuration, in which the weight key is**

absent and the term inactive: either restore the keys and rerun, or correct the note.]

Physics regularizers and their absence from the objective. Three physics terms are implemented in a dedicated loss module. *Radius influence* penalises reconstructed foreground pixels that are cold where a frequency-conditioned critic judges decap coverage to be weak. *Critic supervision* trains that critic on the inverted min-max normalised foreground of the target heatmap; the critic is a dual-branch convolutional network taking the occupancy grid together with a 16-dimensional frequency embedding and producing a spatial effectiveness map, and it is detached inside the radius-influence term so that VAE gradients do not corrupt it. *Anti-resonance* encodes the requirement that every impedance peak be preceded in frequency by a resonance dip:

$$\begin{aligned} s^{\text{peak}} &= \min(\text{relu}(c - l), \text{relu}(c - r)), & s^{\text{dip}} &= \min(\text{relu}(l - c), \text{relu}(r - c)), \\ \mathcal{L}_{\text{ar}} &= \text{mean}(s^{\text{peak}} \cdot \exp(-10.0 \cdot d^{\text{prior}})), \end{aligned} \quad (3.6)$$

where c , l and r are the centre, left and right bin values of the impedance channel and d^{prior} is the running cumulative maximum of prior dip scores.

The experiment configuration assigns non-zero weights to all three: 1.0 for radius influence, 2.0 for critic supervision and 0.5 for anti-resonance, with a critic warm-up of 11 epochs, a slope anneal over 22 epochs and a foreground clip at -1.6791870594024658 . The physics module is instantiated, its parameters reach the optimizer, and they are stored in the checkpoints. None of the three terms nevertheless contributes to the training loss in this experiment. The exp059 trainer replaces the epoch function of the shared training core with a variant whose signature no longer carries the physics arguments and which calls the loss with the physics module unset; the loss guards the radius-influence, critic-supervision and anti-resonance terms on the presence of that module, so none of them is added.

The exp059 surrogate is consequently not physics-informed in its training objective, and this thesis does not describe it as such. The physics prior that is genuinely active in an objective sits in Stage 2, which as implemented runs on the legacy `exp038_true_multi` stack rather than on exp059 (Section 3.3.1). There, the latent optimizer applies an anti-resonance term of weight 0.5, overridable by the checkpoint configuration; see Section 3.3.2. **[FILL IN: state whether disabling the physics terms in the exp059 training loop was a deliberate ablation or an unintended consequence of the epoch-function replacement. Nothing in the repository explains the disconnect, and the answer changes how this subsection should be framed.]**

3.2.3 Training, Inference

Configuration and code organisation. Each experiment reads a configuration file from its own directory, and an environment override may be layered on top of it at runtime. That override is the mechanism by which the active-learning fine-tune injects its own settings (Section 3.5). The exp059 trainer specialises the shared training core by replacing several of its components: the configuration object, the model builder, the epoch function, the heatmap loss, the total loss, the statistics hook, the dataloader factory, the configuration loader and the impedance loss.

Optimizer and schedule. The optimizer is AdamW with weight decay 1×10^{-4} , selected as fused where available, then in its `foreach` form, and otherwise as a plain implementation. The scheduler reduces the learning rate on plateau in minimising mode, with a factor of 0.5, a patience of 15 epochs and a floor of 1×10^{-5} . A separate impedance learning rate can be enabled after the peak-focus epoch; it is enabled, with a multiplier of 1.0 and a peak-focus epoch of 25.

The configuration as it stands specifies a learning rate of 1.5×10^{-5} , 510 epochs, a learning-rate reset on resume, and a resume checkpoint pointing at the experiment’s last model. The experiment notes state that the exp059 training run used a learning rate of 2×10^{-4} , 500 epochs and no resume checkpoint. The two no longer agree. **[FILL IN: state which learning rate, epoch count and resume setting produced the checkpoint whose results are reported — the notes give $2e-4$ / 500 / null and the configuration gives $1.5e-05$ / 510 / `last_model.pt` — and cite the run log that settles it.]**

Numerical stability. The policy is deliberately conservative. Mixed precision, TF32 and graph compilation are all off, training is forced to fp32, non-finite batches are skipped, and gradients are clipped to a norm of 1.0. Log-variances are clamped to $[-10.0, 10.0]$, the reconstruction latent has a soft floor of -8.0 , and the loss is capped at 1000000.0 for finiteness. **[FILL IN: state why mixed precision and TF32 were disabled for exp059 — whether a specific instability was observed, and in which run.]**

Curriculum. The milestone epochs in the current configuration are: cross-frequency consistency from epoch 20; heatmap focus, heatmap protection under modality dropout, the impedance peak terms and the end of beta scheduling from epoch 25; and warm-ups of 11 epochs for the physics critic and for the penalty terms.

Validation and monitoring. Validation runs on the first epoch, on each checkpoint interval and on the final epoch, with a checkpoint interval of 5 epochs and no checkpoint pruning. The off-anchor evaluation runs every 5 epochs at 155.0, 250.0 and 265.0 MHz with weights 2.5, 3.0 and 2.5 respectively, over at most 20 batches, and evaluates the occupancy-only decode path. The name is only partly accurate for this frequency list: 155.0 and 265.0 are absent from the 24 training anchors of Section 3.1.2 and are genuinely held out, whereas 250.0 is a training anchor and carries the largest of the three weights, so it acts as an in-distribution reference inside the same metric. The experiment directory also retains a best-off-anchor checkpoint, so this metric doubles as a checkpoint-selection criterion rather than serving only as a monitor. **[FILL IN: state why 250 MHz, a training anchor, sits in the off-anchor evaluation set with the highest weight of the three, and confirm exactly which aggregate quantity best_off_anchor_model.pt is selected on.]** Each run writes per-epoch records of the total and component losses, the heatmap peak split, the impedance split, the off-anchor evaluation and per-epoch timing into the experiment’s metrics directory.

Distributed training. Distributed data-parallel training is enabled with a base batch size of 160 and linear learning-rate scaling, so the learning rate is multiplied by $(\text{batch size} \times \text{world size})/160$. Initialisation uses the NCCL backend with a default 30-minute timeout, and the model is wrapped with unused-parameter detection enabled. **[FILL IN: state why unused-parameter detection was enabled in the distributed wrapper. The setting carries no comment, note or commit message giving a reason, so the natural explanation — expert branches left unused by modality dropout — is not evidenced.]**

Encode-path mixing during training. The layout-mixing probability is 0.65 and the occupancy-only encode probability is 0.4, and this chapter takes these and the neighbouring keys from the configuration file. The experiment notes and the project handover document both describe the exp059 capacity phase as full-encode only, with both probabilities at zero. **[FILL IN: state which encode-path mixing setting produced the reported capacity results. The configuration gives layout_train_prob 0.65, occ_only_encode_prob 0.4, occupancy-only evaluation enabled and distillation weights of 2.5, whereas the notes and the handover document describe all of these as zero or disabled for the capacity phase.]**

[FILL IN: document the training hardware and software environment: GPU model and count, driver, CUDA and PyTorch versions, and wall-clock time per epoch. No committed artifact records this; the timing CSV gives

per-epoch durations but not the machine.]

[FILL IN: report the exp059 model parameter count. It is printed at runtime by the training core but not stored in any artifact.]

3.2.4 Auxiliary Guidance Network for Latent-Space Optimization

No module, class or function of this name exists in the current model track; the term appears only in the thesis outline. The nearest implemented component is a feed-forward impedance surrogate that maps an occupancy vector directly to an impedance spectrum, and which Stage 2 uses to score candidate placements without invoking the VAE impedance head. It exists only in the legacy experiment forks exp037, exp038, exp039, exp040 and exp041, and is absent from exp057 through exp060.

In the exp038 fork it maps occupancy (52,) to an impedance log- z spectrum (1, 231) and does not use the inspection frequency. Its hidden widths are (256, 512, 512, 512, 256) with dropout 0.1, and it is trained for 200 epochs at batch size 128 with a learning rate of 3×10^{-4} , plateau patience 15, factor 0.5 and floor 1×10^{-5} , weight decay 1×10^{-4} , a top- K term over 20 bins with weight 7.0, and an under-prediction penalty of 2.8. Stage 2 loads its checkpoint from the exp038 experiment directory whenever the surrogate is enabled, which is the default, and otherwise falls back to the VAE impedance decoder head.

[FILL IN: decide what this subsection is to contain. Either (a) rename the heading to “Impedance Surrogate for Latent-Space Optimization” and present the exp038 surrogate above as the component, stating that it belongs to the legacy Stage-2 stack and not to exp059; or (b) implement and describe an exp059-native guidance network. As it stands there is no exp059 component under this name.]

3.3 Latent-Space Optimization of Decoupling Capacitor Placement

3.3.1 Problem Formulation

Stage 2 inverts the surrogate. With all network weights frozen, the search variable is the latent vector z rather than the placement itself, following the latent-space inverse-design template of [32]. For a fixed budget K , the decoded occupancy probabilities

$p(z) \in [0, 1]^{52}$ are read out to a placement by a hard top- K selection,

$$\mathbf{b}(z, K) = \text{top}_K(p(z)) \in \{0, 1\}^{52}, \quad \sum_i b_i = K, \quad (3.7)$$

and the predicted spectrum $\hat{Z}(f)$ over the 231 bins is compared with the target mask $Z_{\text{target}}(f)$. A candidate is declared feasible only when the predicted curve lies at or below the feasibility boundary at *every* one of the 231 bins,

$$\hat{Z}(f_j) \leq Z_{\text{boundary}}(f_j) \quad \text{for all } j = 1, \dots, 231, \quad Z_{\text{boundary}} = Z_{\text{target}} - m, \quad (3.8)$$

with a boundary margin $m = 0.1$. The objective maximises the margin below the boundary, and the loss is evaluated in the normalized log space, so the base term rewards margin and penalises excursions above the boundary,

$$\mathcal{L}_{\text{base}} = -w_{\text{gap}} \cdot \text{gap} + w_{\text{exceed}} \cdot \text{exceed}, \quad w_{\text{gap}} = 1.0, \quad w_{\text{exceed}} = 25.0, \quad (3.9)$$

where the exceed component is a mean of $\text{relu}(\hat{Z} - Z_{\text{boundary}})$ raised to the power 2.0 over the bins, plus a maximum-excursion term. **[FILL IN: write out the exact form of the maximum-excursion component of the exceed term and of the gap term; the extraction recorded the weights and the mean component but not the full expression.]**

The target mask is loaded from a stored array, is required to have shape (231,) after squeezing, is interpreted in ohms, and is converted into the model’s normalized log space by $(\log Z_{\text{target}} - \mu_{\log})/\sigma_{\log}$. The stored array has shape (231, 1) and dtype float64, with minimum 0.8 and maximum 49.99999999999999. **[FILL IN: document the provenance and definition of the target impedance mask: who set the 0.8 ohm floor and the 49.99999999999999 ohm ceiling, against which specification, and with which script. The .npz file carries no unit metadata and no generating script was found.]**

Scope limitation. Stage 2 as implemented targets the legacy `exp038_true_multi` stack: it defaults to that experiment for both the VAE occupancy decoder and the impedance surrogate checkpoint, and imports its VAE class from that fork. It is not wired to `exp059`, a limitation also recorded in the project’s latent-optimization documentation. Everything described in this section is therefore the method as implemented against `exp038`, and no end-to-end inverse-design result may be attributed to the `exp059` surrogate. A second, smaller inconsistency points the same way: the Stage-2 default normalization statistics path names a dataset directory that does not exist in the current checkout, whereas the `exp059` training set is the unbounded set

of Section 3.1.2. **[FILL IN: record the work needed to wire Stage 2 to the exp059 checkpoint, including the correct normalization statistics path; the Stage-2 default points at datasets/data_multifreq_norm/, which is absent, while training uses datasets/data_multifreq_train_norm_unbounded.]**

3.3.2 Optimization Procedure and Feasibility Assessment

The search is gradient descent on z with everything else frozen. Budgets $K = 1 \dots 25$ are searched, each with 32 candidate seeds optimized as a single batch over 1200 steps at a learning rate of 5×10^{-2} , with gradients clipped at 5.0 and an initial shared-latent temperature of 0.8. Seeds may be drawn randomly per run and recorded in the run configuration, or fixed to $0 \dots N - 1$; the random mode is the default. **[FILL IN: reconcile the three budget ranges in the framework: Stage 2 searches $K = 1 \dots 25$, active learning acquires $K = 3 \dots 10$, and the dataset is filtered to $K \leq 30$. Nothing in the repository explains the relationship.]**

Straight-through top- K read-out. The read-out of Equation 3.7 is not differentiable, and this is the load-bearing detail of Stage 2. The implementation uses a straight-through estimator [33],

$$\mathbf{b} = \text{top}_K^{\text{hard}}(p) + p - \text{sg}[p], \quad (3.10)$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operator: the forward value is the hard top- K vector and the gradient is passed to p unchanged. The reason is stated in the code — “Without this the top-K argsort severs the gradient and z never moves on the spectrum target.” Relaxing the discrete selection with a Gumbel-softmax or concrete relaxation [34], [35] is an alternative that is not used here.

One qualification matters for interpreting Stage 2. The estimator is applied only on the surrogate branch: when the impedance surrogate is loaded, the forward pass evaluates the surrogate on the straight-through top- K occupancy, whereas without it the impedance comes from the VAE decoder head and the occupancy read-out is not in the gradient path at all. The surrogate is enabled by default, so the straight-through branch is the default, but the fallback branch does not exercise it.

Regularization of the search. Several terms keep the search inside the region where the decoder is trustworthy and keep the 32 seeds from collapsing onto one another. A latent prior term of weight 0.05 penalises $((z - \mu)/\sigma)^2$ against the aggregate posterior rather than against the unit prior, and a posterior-boundary hinge of weight 10.0 activates beyond a Mahalanobis-style radius of 2.5. Spectrum-shape terms comprise

a shape regularizer of weight 2.0 with a minimum log-spectrum standard deviation of 0.12, which penalises degenerate flat spectra, and a tracking term of weight 2.0 for mean-squared alignment with the target. Peak alignment is enforced by a peak loss of weight 1.0, a peak-index term of 3.0, a peak-magnitude term of 2.0 and a dual top- K term of 2.5, over the 8 strongest peaks and the 20 largest bins; all of these may be overridden by environment variables. Seed diversity is encouraged by penalising the mean pairwise cosine similarity across the parallel seeds with weight 0.35, and occupancy confidence with weight 0.5. Finally the anti-resonance physics prior of Equation 3.6 is applied with weight 0.5, overridable by the checkpoint configuration. This is the one physics term that is active in an optimization objective in this work, and it belongs to Stage 2 as implemented against exp038; as Section 3.2.2 records, no physics term enters the exp059 training objective.

Selection and reporting of solutions. Among the feasible candidates for a budget, the selection metric is the maximum predicted impedance in ohms across the spectrum: candidates are ranked by their peak and the lowest peak wins. Ranking by the mean is available as an alternative. If no candidate for a budget is feasible, the run writes an explicit no-solution record for that budget rather than emitting a best-effort placement, which keeps infeasibility distinguishable from a poor solution in the downstream reports.

Surrogate feasibility is a screening verdict and not a sign-off. A placement that Stage 2 declares feasible has been checked only against the surrogate’s own prediction, and requires ground-truth re-simulation in ECADSTAR before any claim about the physical board is made.

[FIGURE: Stage-2 loop: frozen decoder, gradient on $\mathbf{z}$, straight-through top-K read-out into the impedance surrogate, feasibility test against the target mask.]

3.4 Experimental Setup

The model track used for all training and active learning reported in this thesis is exp059_capacity_freq, referred to as exp059 throughout. exp057_structured_graph and exp058_asymmetric_kl are legacy, and exp060_multitype_occ is an exploratory fork. Each experiment directory is a self-contained copy of the training stack, so a change in one fork does not propagate to the others, and checkpoints are not cross-loadable between them.

Training data is the unbounded normalized set described in Section 3.1.2: 582997 manifest rows over 24979 layouts and 24 anchor frequencies, split by layout in the

ratio 0.9 with seed 42. Two configuration files govern the two phases of the experiment, `config.yaml` for the capacity phase and `config_al_finetune.yaml` for the active-learning fine-tune, the latter injected at runtime through the configuration-path override. A checkpoint trained under the capacity configuration alone is not directly usable for active-learning scoring, which needs the occupancy-only encode path.

[FILL IN: state the compute environment used for the reported runs: GPU model and count, host memory, CUDA and PyTorch versions, and total training wall-clock time. This is not recorded in any committed artifact and must not be taken from a live shell.]

Simulation of newly proposed layouts runs on ECADSTAR under Windows, driven from WSL. The active-learning configuration names the design file `H-shape.erf`, the powerbus `Power_GND` and the port `IC1_Port1`, requests heatmap-only PEB export, and allows a simulation wait of 18000 s. The simulation steps of the pipeline cannot be executed without that host.

The residual Gaussian process used for acquisition (Section 3.5) is implemented with GPyTorch [50] as an approximate GP trained with a variational evidence lower bound.

All numeric settings quoted in this chapter are taken from the experiment configurations, the dataset metadata and the active-learning configurations. Several of the project’s internal reference documents describe earlier experiment generations and disagree with those artifacts on anchor sets, loss weights and acquisition budgets; they are used for method description only and not as a source of numbers.

3.5 Active Learning for scaling the model

The surrogate is trained on a fixed pool of simulated layouts, and the layouts that matter for inverse design — the placements an optimizer wants to propose — are not necessarily well represented in that pool. The active-learning loop closes this gap in the standard pool-based way [27], [30]: generate a large pool of candidate layouts, score them, send only the highest-scoring batch to the expensive ECADSTAR simulator, ingest the labels, and fine-tune the surrogate on the enlarged set.

Cycle structure and control. A cycle is described here as eight steps: per- K acquisition, which covers candidate generation, inference and selection; PEB construction; ECADSTAR simulation; ingest with normalization and pre-fine-tune evaluation; overlay construction; fine-tune; and post-fine-tune evaluation with the decision report. The step count is computed at runtime and is eight only when fine-tuning and post-fine-tune evaluation are both enabled, and seven or six otherwise. A propose-only mode

runs generation and inference alone, with no simulation and no fine-tune, and is the mode used for dry runs. Three configurations are provided: a residual-GP acquisition (the default and primary run), a Monte-Carlo acquisition, and an equal-budget random control.

Candidate pool. For exp059 the pool spans budgets $K = 3$ to $K = 10$, sweeping all budgets in one pool, with 1600 candidates per budget, 80 selected per budget and a total ECAD budget of 640 simulations per cycle. Candidates are random K -hot occupancy vectors over the 52 slots. Generation is seeded deterministically per iteration and per budget as `seed = 42 + iteration × 10000 + K`, so a cycle can be reproduced exactly. Each candidate is also assigned an inspection frequency from the grid [120, 150, 155, 170, 180, 200, 230, 250, 265, 270, 280, 300] MHz under per- K strata quotas 120:30, 150:50, 155:100, 170:40, 180:50, 200:90, 230:90, 250:140, 265:130, 270:80, 280:70, 300:70. An exploration slice adds 160 further candidates per budget, drawn from a continuous 120–320 MHz range quantized to 5 MHz with band edges [120, 180, 230, 280, 320]. Selection applies the frequency quotas together with a minimum uncertainty percentile of 40, and where pool strata are given without matching ECAD-side strata, the latter are auto-scaled from the pool quotas to that budget’s selection count. **[FILL IN: give the rationale for the frequency strata quotas — why 250 MHz receives 140 candidates per K and 155 MHz receives 100. No artifact states it.]**

These budgets differ from the written active-learning documentation, which is exp057-era and states 400 candidates per budget, 10 selected per budget, 80 ECAD simulations per cycle and off-anchor frequencies of 90, 265, 435 and 510 MHz. The values above are those of the exp059 configuration. **[FILL IN: update docs/active-learning.md, whose acquisition budgets and off-anchor frequencies predate the exp059 configuration used here.]**

Acquisition by a residual Gaussian process. The primary exp059 run acquires by predicted error. The acquisition dispatch supports three modes — residual GP, random and Monte-Carlo — and defaults to Monte-Carlo when unset. The residual-GP mode does not ask the VAE how confident it is: it fits a separate Gaussian process whose regression target is the surrogate’s own error on labelled data, and ranks candidates by that predicted error. The configured settings use the predicted-heatmap feature path, the 99th-percentile absolute error over the physical foreground as target, a sparse variational GP regressor, $\kappa = 0.5$, 4000 fitting points, 256 inducing points, 80 fitting epochs, seed 0, a novelty weight of 0.0, and a score mode of posterior mean. Other residual targets are available in the implementation: peak-pixel error in three

variants, a structural target, mean absolute and mean squared error, and impedance mean squared error.

The acquisition function implemented is the upper confidence bound

$$a(x) = \mu_e(x) + \kappa \sigma_e(x), \quad (3.11)$$

in the form of [31], where μ_e and σ_e are the posterior mean and standard deviation of the error GP. Under the configured score mode the acquisition returns μ_e alone, and the novelty term enters only under the combined UCB-plus-novelty mode. The production exp059 run therefore ranks purely by predicted error and does not use σ_e . One implementation docstring describes the mode as ranking by the full upper confidence bound, which does not match the resolved behaviour. **[FILL IN: correct the inference-pool docstring, which states that residual-GP candidates are ranked by $\mu_e + \kappa \sigma_e$, whereas the configured score mode resolves to the mean alone with sigma unused; docs/gp-error-surrogate.md states the mean default explicitly.]**

The GP inputs come from a feature path that requires no simulation: the model encodes the candidate occupancy through the occupancy-only path, decodes it, re-encodes the model’s *own* predicted heatmap, and uses the resulting posterior mean as the GP input. This is what allows candidates to be scored before any ECAD label exists. The fitted GP is cleared at the start of every per- K acquisition, so each cycle performs a fresh fit rather than re-using a cached one. **[FILL IN: confirm the reason for clearing the cached GP fit each cycle — the obvious candidate is that fine-tuning shifts the latent space in which the GP is fitted, but only the clearing itself is recorded, with no comment or note stating why.]**

This construction is assembled from three separate literatures rather than taken from one canonical precedent. The sparse variational GP that makes it tractable at candidate-pool scale comes from the inducing-point variational formulation of [51] and its stochastic extension [52], implemented here in GPyTorch [50]. The acquisition rule is GP-UCB [31]. Modelling a separate discrepancy term rather than trusting the primary model’s own uncertainty is the model-discrepancy decomposition of [53]. No source found in the literature review combines them as a unit for active learning of a neural surrogate in engineering design, and this chapter does not claim one. Modelling error with a second model is also a departure from asking the network itself, for example by Monte-Carlo dropout [28], which is the mechanism the Monte-Carlo mode implements. **[FILL IN: state the design rationale for choosing the residual GP over MC-dropout uncertainty in the author’s own words; the framework document motivates it, but the decision is not recorded as a comparison in any artifact.]**

Controls. The random control assigns uniform random scores from a generator seeded by the run seed plus the iteration index, preserving the ECAD budget exactly so that the comparison is equal-budget. The Monte-Carlo mode retains its parameters in the exp059 configuration: 4 forward passes, 4 latent passes and 8 decoder passes with decoder dropout enabled, an automatic worst-candidate score metric, and score weights of 1.0 for decoder-dropout reconstruction error, 1.0 for latent heatmap mean squared error, 1.0 for the variance of the 99th percentile, 0.5 for its range, 0.5 for peak-location spread, 0.25 for the peak coefficient of variation and 0.35 for prior tension.

No comparative claim is made here about the three acquisition modes. The exp059 acquisition A/B remains an open task and the project’s claim ledger records the GP-versus-Monte-Carlo comparison as uncertain, so this chapter describes the mechanisms and leaves their relative merit to be established.

Scoring configuration. Scoring runs in layout inference mode at a reference frequency of 200.0 MHz, with a shared-latent temperature of 1.5 and a heatmap grid of 64, using a 64×64 boolean board mask and the normalization statistics of the unbounded training set, so that scoring uses the same normalization the model was trained under.

Simulation, ingest and fine-tune. Selected layouts are exported as PEB batches and simulated by ECADSTAR, as described in Section 3.4. Labels are ingested and robustly normalized, then merged into an overlay dataset (`datasets/data_multifreq_al_overlay_exp`) which is mixed into training with a per-sample weight of 128.0 on the overlay rows. **[FILL IN: state the reason for the 128.0 overlay sample weight and how the value was chosen; both the AL configuration and the fine-tune configuration set it, but no artifact records the intent or a comparison against the training code’s own default.]** The fine-tune resumes from the last checkpoint and trains for 45 epochs beyond the checkpoint’s own epoch count. The merged runtime configuration is written out and passed to the trainer through the configuration-path override.

The fine-tune’s encode-path mixing probabilities differ between the two configurations that define them. The experiment’s fine-tune configuration sets a layout-mixing probability of 0.9 and an occupancy-only encode probability of 0.7, together with latent and output distillation weights of 2.5, occupancy-only evaluation, and early stopping with patience 3, minimum delta 0.01 on the layout-cross criterion. The active-learning configuration sets 0.65 and 0.4, and the runtime merge copies the active-learning values over the experiment file, so the executed values are 0.65 and 0.4. **[FILL IN: state whether the executed fine-tune mixing values of 0.65 / 0.4 were intended;**

the experiment’s fine-tune configuration, the project handover document and CLAUDE.md all give 0.9 / 0.7, and the runtime merge silently overrides them.]

Reporting. Decision reporting is gated on minimum sample sizes of 30 ranked candidates and 20 ECAD labels, and re-scores only the newly ECAD-labelled layouts. Every cycle writes `scored_candidates.json`, `acquisition_rank_quality.json`, pre- and post-fine-tune off-anchor evaluations, a cycle summary, a cycle evaluation report, a decision report and a decision ledger. The results chapter draws its active-learning numbers from these files.

The loop has been executed on exp059. The run `al_exp059_gp_error_001` contains four iterations, and the summary of the fourth records residual-GP acquisition over budgets `[3, 4, 5, 6, 7, 8, 9, 10]` with 1600 candidates and 80 selections per budget, 12800 candidates in total and 640 selected. This is stated only as evidence that the pipeline described above runs end to end; no performance figure from those runs belongs in this chapter.

[FIGURE: Active-learning cycle: candidate generation, residual-GP scoring, PEB export, ECADSTAR simulation, overlay build, fine-tune, decision report.]

Chapter 4

Results and Evaluations

4.1 Performance of the Generative Surrogate Model

4.1.1 Reconstruction Performance

4.1.2 Validation of Generated PI Spatial Heatmaps

4.1.3 Evaluation of the Auxiliary Guidance Network

4.2 Latent-Space Design Optimization Results

4.2.1 Evaluation of Scalability and Computational Efficiency

4.2.2 Spatial placement and Capacitor-Type Evaluation of the Framework

4.3 Limitations

4.4 Ablations

Chapter 5

Conclusion and Outlook

5.1 Summary of Contributions

5.2 Future Research Directions

References

- [1] L. D. Smith, R. E. Anderson, D. W. Forehand, T. J. Pelc, and T. Roy, “Power distribution system design methodology and capacitor selection for modern cmos technology,” *IEEE Trans. Advanced Packaging* 22(3), 284–291, 1999. [Online]. Available: <https://doi.org/10.1109/6040.784476>.
- [2] M. Swaminathan, H. M. Torun, H. Yu, J. A. Hejase, and W. D. Becker, “Demystifying machine learning for signal and power integrity problems in packaging,” *IEEE Trans. Components, Packaging and Manufacturing Technology* 10(8), 1276–1295, 2020. [Online]. Available: <https://doi.org/10.1109/TCPMT.2020.3011910>.
- [3] L. Zhang, W. Huang, J. Juang, H. Lin, B. C. Tseng, and C. Hwang, “An enhanced deep reinforcement learning algorithm for decoupling capacitor selection in power distribution network design,” in *IEEE Int. Symp. Electromagnetic Compatibility and Signal/Power Integrity (EMCSI)*, pp. 245–250, 2020. [Online]. Available: https://scholarsmine.mst.edu/ele_comeng_facwork/6954/.
- [4] N. G. Shoaee, Z. Nezhi, W. John, R. Brüning, and J. Götze, “Generating ai modules for decoupling capacitor placement using simulation,” *Advances in Radio Science* 21, 49–55, 2023. [Online]. Available: <https://doi.org/10.5194/ars-21-49-2023>.
- [5] H. Manoharan, J. Juang, L. Zhang, *et al.*, “Graph convolutional neural network assisted genetic algorithm for pdn decap optimization,” in *IEEE Int. Symp. Electromagnetic Compatibility, Signal & Power Integrity (EMC+SIPI)*, 2024. [Online]. Available: <https://doi.org/10.1109/EMCSIPI49824.2024.10705608>.
- [6] L. Zhang, Z. Zhang, C. Huang, *et al.*, “Decoupling capacitor selection algorithm for pdn based on deep reinforcement learning,” in *IEEE Int. Symp. Electromagnetic Compatibility, Signal & Power Integrity (EMC+SIPI)*, pp. 616–620, VERIFY: publisher page returned 403; fields confirmed from a co-author’s publication list, page numbers unconfirmed, 2019. [Online]. Available: <https://ieeexplore.ieee.org/document/8825249/>.

References

- [7] H. Park, S. Kim, Y. Kim, *et al.*, “Deep reinforcement learning-based optimal decoupling capacitor design method for silicon interposer-based 2.5-d/3-d ics,” *IEEE Trans. Components, Packaging and Manufacturing Technology* 10(3), 467–478, 2020. [Online]. Available: <https://pure.kaist.ac.kr/en/publications/deep-reinforcement-learning-based-optimal-decoupling-capacitor-de/>.
- [8] H. Park, M. Kim, S. Kim, J. Kim, *et al.*, “Transformer network-based reinforcement learning method for power distribution network (pdn) optimization of high bandwidth memory (hbm),” *arXiv / IEEE TMTT*, 2022. [Online]. Available: <https://arxiv.org/abs/2203.15722>.
- [9] H. Kim, M. Kim, F. Berto, J. Kim, and J. Park, “Devformer: A symmetric transformer for context-aware device placement,” in *ICML 2023*, 2023. [Online]. Available: <https://arxiv.org/abs/2205.13225>.
- [10] L. Zhang, J. Juang, Z. Kiguradze, *et al.*, “Fast pdn impedance prediction using deep learning,” *arXiv:2106.10693 / Int. J. Numerical Modelling 2022*, 2021. [Online]. Available: <https://arxiv.org/abs/2106.10693>.
- [11] E. Bogatin, *Signal and Power Integrity – Simplified*, 3rd ed. Pearson / Prentice Hall PTR, 2017, VERIFY: two ISBNs appear across retailers for this edition – confirm the one on the copy actually used. [Online]. Available: <https://www.informit.com/store/signal-and-power-integrity-simplified-9780134512204>.
- [12] I. Novak and J. R. Miller, *Frequency-Domain Characterization of Power Distribution Networks*. Artech House, 2007, VERIFY: bibliographic data confirmed, but the table of contents could not be fetched – confirm the plane-cavity-resonance coverage against a physical copy before citing it for that claim.
- [13] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” in *ICLR 2014 / arXiv:1312.6114*, 2013. [Online]. Available: <https://arxiv.org/abs/1312.6114>.
- [14] I. Higgins, L. Matthey, A. Pal, *et al.*, “Beta-vae: Learning basic visual concepts with a constrained variational framework,” in *ICLR 2017*, 2017. [Online]. Available: <https://openreview.net/forum?id=Sy2fzU9gl>.
- [15] K. Sohn, H. Lee, and X. Yan, “Learning structured output representation using deep conditional generative models,” in *NeurIPS 2015*, 2015. [Online]. Available: <https://papers.nips.cc/paper/5775-learning-structured-output-representation-using-deep-conditional-generative-models>.

References

- [16] J. He, D. Spokoyny, G. Neubig, and T. Berg-Kirkpatrick, “Lagging inference networks and posterior collapse in variational autoencoders,” in *ICLR 2019 / arXiv:1901.05534*, 2019. [Online]. Available: <https://arxiv.org/abs/1901.05534>.
- [17] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling, *Improving variational inference with inverse autoregressive flow*, NeurIPS 2016 / arXiv:1606.04934, VERIFY: title/authors/year confirmed from arXiv, but the free-bits formulation was not readable from the full text – confirm the equation before reproducing it, 2016. [Online]. Available: <https://arxiv.org/abs/1606.04934>.
- [18] M. Wu and N. Goodman, “Multimodal generative models for scalable weakly-supervised learning,” in *NeurIPS 2018 / arXiv:1802.05335*, 2018. [Online]. Available: <https://arxiv.org/abs/1802.05335>.
- [19] Y. Shi, N. Siddharth, B. Paige, and P. H. S. Torr, “Variational mixture-of-experts autoencoders for multi-modal deep generative models,” in *NeurIPS 2019*, 2019. [Online]. Available: <https://arxiv.org/abs/1911.03393>.
- [20] S. Kutuzova, O. Krause, D. McCloskey, M. Nielsen, and C. Igel, *Multimodal variational autoencoders for semi-supervised learning: In defense of product-of-experts*, arXiv:2101.07240, 2021. [Online]. Available: <https://arxiv.org/abs/2101.07240>.
- [21] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *ICLR 2017 / arXiv:1609.02907*, 2016. [Online]. Available: <https://arxiv.org/abs/1609.02907>.
- [22] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *ICML 2017 / arXiv:1704.01212*, 2017. [Online]. Available: <https://arxiv.org/abs/1704.01212>.
- [23] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “Film: Visual reasoning with a general conditioning layer,” in *AAAI 2018 / arXiv:1709.07871*, 2018. [Online]. Available: <https://arxiv.org/abs/1709.07871>.
- [24] G. Huang, J. Hu, Y. He, J. Liu, *et al.*, “Machine learning for electronic design automation: A survey,” *ACM TODAES / arXiv:2102.03357*, 2021. [Online]. Available: <https://arxiv.org/abs/2102.03357>.
- [25] C. H. Goay, Z. Q. Cheong, C. E. Low, N. S. Ahmad, and P. Goh, “Power-ground plane impedance modeling using deep neural networks and an adaptive sampling process,” *International Journal of Electronics and Telecommunications*

References

- 68(4), 2022. [Online]. Available: <https://ijet.pl/index.php/ijet/article/view/10.24425-ijet.2022.143887>.
- [26] X. Dong, Y. Chen, J. Chen, *et al.*, “Worst-case power integrity prediction using convolutional neural network,” *ACM Trans. Design Automation of Electronic Systems* 28(4), 1–19, 2023. [Online]. Available: <https://doi.org/10.1145/3564932>.
- [27] B. Settles, “Active learning literature survey,” *Univ. Wisconsin-Madison, Dept. of Computer Sciences, Technical Report TR1648*, 2009. [Online]. Available: <https://minds.wisc.edu/handle/1793/60660>.
- [28] Y. Gal and Z. Ghahramani, “Dropout as a bayesian approximation: Representing model uncertainty in deep learning,” in *ICML 2016 / arXiv:1506.02142*, 2016. [Online]. Available: <https://arxiv.org/abs/1506.02142>.
- [29] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” in *NeurIPS 2017 / arXiv:1612.01474*, 2016. [Online]. Available: <https://arxiv.org/abs/1612.01474>.
- [30] S. Kapoor, H. Vardhan, U. Timalisina, S. Kumar, P. Volgyesi, and J. Sztipanovits, *Data-efficient surrogate modeling for engineering design: Ensemble-free batch-mode deep active learning for regression*, arXiv:2211.10360, 2022. [Online]. Available: <https://arxiv.org/abs/2211.10360>.
- [31] N. Srinivas, A. Krause, S. Kakade, and M. Seeger, “Gaussian process optimization in the bandit setting: No regret and experimental design,” in *ICML 2010 / arXiv:0912.3995*, 2010. [Online]. Available: <https://arxiv.org/abs/0912.3995>.
- [32] R. Gómez-Bombarelli, J. N. Wei, D. Duvenaud, J. M. Hernández-Lobato, B. Sánchez-Lengeling, *et al.*, *Automatic chemical design using a data-driven continuous representation of molecules*, ACS Central Science 4(2) / arXiv:1610.02415, 2018. [Online]. Available: <https://arxiv.org/abs/1610.02415>.
- [33] Y. Bengio, N. Léonard, and A. Courville, *Estimating or propagating gradients through stochastic neurons for conditional computation*, arXiv:1308.3432, 2013. [Online]. Available: <https://arxiv.org/abs/1308.3432>.
- [34] E. Jang, S. Gu, and B. Poole, “Categorical reparameterization with gumbel-softmax,” in *ICLR 2017 / arXiv:1611.01144*, 2016. [Online]. Available: <https://arxiv.org/abs/1611.01144>.
- [35] C. J. Maddison, A. Mnih, and Y. W. Teh, “The concrete distribution: A continuous relaxation of discrete random variables,” in *ICLR 2017*, 2016. [Online]. Available: <https://arxiv.org/abs/1611.00712>.

References

- [36] S. Piersanti, R. Cecchetti, C. Olivieri, F. de Paulis, A. Orlandi, and M. Buecker, “Decoupling capacitors placement at board level adopting a nature-inspired algorithm,” *Electronics* 8(7), 737, 2019. [Online]. Available: <https://doi.org/10.3390/electronics8070737>.
- [37] F. de Paulis, R. Cecchetti, C. Olivieri, and M. Buecker, “Genetic algorithm pdn optimization based on minimum number of decoupling capacitors applied to arbitrary target impedance,” in *IEEE Int. Symp. Electromagnetic Compatibility and Signal/Power Integrity (EMCSI)*, 2020. [Online]. Available: <https://doi.org/10.1109/EMCSI38923.2020.9191458>.
- [38] R. Cecchetti, F. de Paulis, C. Olivieri, A. Orlandi, and M. Buecker, “Effective pcb decoupling optimization by combining an iterative genetic algorithm and machine learning,” *Electronics* 9(8), 1243, 2020. [Online]. Available: <https://doi.org/10.3390/electronics9081243>.
- [39] H. Kim, H. Park, M. Kim, *et al.*, “Deep reinforcement learning framework for optimal decoupling capacitor placement on general pdn with an arbitrary probing port,” in *IEEE 30th Conf. Electrical Performance of Electronic Packaging and Systems (EPEPS)*, VERIFY: publisher page not readable; fields confirmed from the KAIST EE lab project page, 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/9609194/>.
- [40] Y. Duan, H. Feng, Z. Yu, H. Wu, L. Shao, and X. Zhu, *Hierarchical decoupling capacitor optimization for power distribution network of 2.5d ics with co-analysis of frequency and time domains based on deep reinforcement learning*, arXiv:2407.04737 (rev. 2025), 2024. [Online]. Available: <https://arxiv.org/abs/2407.04737>.
- [41] V. A. Chhabria, V. Ahuja, A. Prabhu, N. Patil, P. Jain, and S. S. Sapatnekar, “Encoder-decoder networks for analyzing thermal and power delivery networks,” *ACM TODAES 2022 / arXiv:2110.14197*, 2021. [Online]. Available: <https://arxiv.org/abs/2110.14197>.
- [42] Y. Zhao, Z. Chai, X. Jiang, Y. Lin, R. Wang, and R. Huang, “Pdnnet: Pdn-aware gnn-cnn heterogeneous network for dynamic ir drop prediction,” *IEEE Trans. CAD (TCAD)*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.18569>.
- [43] K. Thorat, N. Meng, M. Karami, C. Ding, Y. Lao, and Z. J. Shi, *Gif: A conditional multimodal generative framework for ir drop imaging in chip layouts*, arXiv:2604.09999 (Apr 2026), 2026. [Online]. Available: <https://arxiv.org/abs/2604.09999>.

References

- [44] A. Mirhoseini, A. Goldie, M. Yazgan, *et al.*, “A graph placement methodology for fast chip design,” *Nature* 594(7862), 207–212, 2021. [Online]. Available: <https://doi.org/10.1038/s41586-021-03544-w>.
- [45] D. Kim, J. Park, Y. Oh, and B. Hwang, “Traceformer: S-parameter prediction framework for pcb traces based on graph transformer,” in *Proc. 61st ACM/IEEE Design Automation Conference (DAC '24)*, San Francisco, CA, USA, pp. 1–6, 2024. [Online]. Available: <https://doi.org/10.1145/3649329.3658484>.
- [46] J. Withöft, W. John, E. Ecik, R. Brüning, and J. Götze, *Amortized neural optimization for pre-layout signal integrity design space exploration using differentiable surrogates*, arXiv:2606.07463 (Jun 2026), 2026. [Online]. Available: <https://arxiv.org/abs/2606.07463>.
- [47] J. Withöft, W. John, E. Ecik, R. Brüning, and J. Götze, *Buffer-parameterized machine learning surrogate models for cross-technology signal integrity analysis and optimization*, arXiv:2605.18170 (May 2026), 2026. [Online]. Available: <https://arxiv.org/abs/2605.18170>.
- [48] E. Ecik, W. John, J. Withöft, R. Brüning, and J. Götze, *Surrogate-assisted framework for si-compliant interconnect design optimization using the earth mover’s distance*, arXiv:2606.15234 (Jun 2026), 2026. [Online]. Available: <https://arxiv.org/abs/2606.15234>.
- [49] M. Usama and D. E. Chang, *Deep reinforcement learning-based dram equalizer parameter optimization using latent representations*, arXiv:2507.02365, 2025. [Online]. Available: <https://arxiv.org/abs/2507.02365>.
- [50] J. R. Gardner, G. Pleiss, D. Bindel, K. Q. Weinberger, and A. G. Wilson, “Gpytorch: Blackbox matrix-matrix gaussian process inference with gpu acceleration,” in *NeurIPS 2018 / arXiv:1809.11165*, 2018. [Online]. Available: <https://arxiv.org/abs/1809.11165>.
- [51] M. K. Titsias, “Variational learning of inducing variables in sparse gaussian processes,” in *AISTATS 2009, PMLR vol. 5*, pp. 567–574, 2009. [Online]. Available: <https://proceedings.mlr.press/v5/titsias09a.html>.
- [52] J. Hensman, N. Fusi, and N. D. Lawrence, “Gaussian processes for big data,” in *UAI 2013 / arXiv:1309.6835*, 2013. [Online]. Available: <https://arxiv.org/abs/1309.6835>.
- [53] M. C. Kennedy and A. O’Hagan, “Bayesian calibration of computer models,” *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 63(3), 425–464, 2001. [Online]. Available: <https://academic.oup.com/jrsssb/article/63/3/425/7083367>.

Appendix A

Power Integrity Simulation Environment and Dataset Specification

This appendix is reference material. It records the simulation environment that produced the ground-truth labels, the file formats that carry them, and the on-disk specification of the datasets used for training and for active learning. Tables are preferred to prose throughout; each table is preceded by one short orienting paragraph. Everything stated here is taken from repository artifacts (source code, configuration files, run reports, dataset metadata) as read on 2026-08-05, except Section [A.7](#), whose figures are read from the Windows simulation host — a progress file, per-band output folders and a running process — and carry the observation timestamps given with them. Quantities that exist only inside the eCADSTAR project on the Windows host — solver settings, stack-up, port geometry, board dimensions, capacitor models for the binary dataset — are marked as gaps rather than reconstructed.

A.1 Simulation environment

A.1.1 Solver and invocation

All power-integrity labels are produced by the eCADSTAR PI/EMI analysis engine, driven in batch mode from WSL. There is no GUI automation path: the AutoHotkey driver was removed and the headless CLI is the only route to the engine, with no fallback and no toggle to switch back (`docs/ecadstar_headless_cli.md`; `active_learning_pi/al/ecadstar`). One residual key remains in the primary active-learning configuration, a null `ahk_exe` entry under `ecadstar`, which is no longer read by any driver.

The launcher (`pipelines/dataset_sim/ecadstar.py`, function `run_ecadstar_batch_headless`)

Table A.1: Solver executable and command-line interface.

Item	Value
Engine executable	C:\Program Files\eCADSTAR\eCADSTAR 2023.0\Analysis\bin\engineer.exe
Argument line	"<erf>" --batch "<peb>" --batch-auto-exit, optionally extended with --impulse-port <n>
-b / --batch <file>	execute the given batch file after the design loads
-be / --batch-auto-exit	quit the application when the batch finishes
-p / --impulse-port <n>	IMPULSE model-library server port
Positional argument	the design riffle (.erf / .rif)

starts the engine through PowerShell `Start-Process -PassThru -NoNewWindow` and then blocks on `WaitForExit(timeout_ms)`. Return codes are 0 for a clean self-exit, 124 when the timeout expires and the process is killed, and -1 when PowerShell is unavailable. The default timeout is 172_800 seconds. Windows paths held in the configuration files are converted from WSL form by `resolve_windows_path / windows_path_str`.

A.1.2 Design under analysis and host directories

A single board design is simulated throughout. All of the paths below are absolute Windows paths; the simulation half of this work is therefore reproducible only on that host.

Table A.2: Design, power bus and host directories used by every simulation pipeline.

Item	Value
Design file	C:\Users\muthusamy\Desktop\design\H-shape.emc\H-shape.erf
EMC output directory	C:\Users\muthusamy\Desktop\design\H-shape.emc (engine writes PI-1 ... PI-N)
PEB staging directory	C:\Users\muthusamy\Desktop\design\PEB
Raw output root	C:\Users\muthusamy\Desktop\Raw (RAW_ROOT_WIN)
PEB build directory	C:\Users\muthusamy\Desktop\Raw\peb (PEB_DIR_WIN)
Analysed power bus	Power_GND
Spectrum export port(s)	IC1_Port1, IC2_Port2 (combinations pipeline); IC1_Port1 (multi-type pipeline and the AL PEB builder)

The two port strings are a configuration difference between pipelines, not a defect found in an artifact. **[FILL IN: state which port convention applies to the 231-bin spectra actually stored in datasets/data_multifreq_train; settle it by inspecting the harvested CSV filenames for a legacy PI folder against peb_components in the pipeline that produced it]**

A.1.3 Solver settings not recorded in the repository

The following belong to the eCADSTAR project itself and were not found in any tracked file. They are listed explicitly so that the appendix does not imply knowledge it does not have.

Table A.3: Simulation parameters that live in the eCADSTAR project on the Windows host and are not recoverable from the repository.

Parameter group	Status
Frequency sweep definition, number of sweep points	[FILL IN: export the PI analysis setup from <code>H-shape.erf</code> and record the sweep definition; the only repository trace is the 231-point axis in <code>configs/Frequency_data_hz.npy</code> and the <code>EXPECTED_IMP_LENGTH = 231</code> check]
Mesh / plane-pair settings	[FILL IN: record from the eCADSTAR PI analysis setup dialog]
Port definitions and locations (IC1_Port1, IC2_Port2)	[FILL IN: record port placement and reference from the design]
Layer stack-up, dielectric and copper material parameters	[FILL IN: export stack-up table from <code>H-shape.erf</code>]
Board outline dimensions	[FILL IN: record board size; needed to give the 64×64 heatmap grid a physical scale]
Decap C / ESR / ESL for the single-type (binary) dataset	[FILL IN: record the capacitor model attached to slots C1–C52 in the binary campaign; the <code>TYPE_CATALOG</code> in <code>scrap/generation/generate_peb_multitype.py</code> covers only the multi-type pipeline]

[FIGURE: Annotated board view of `H-shape.erf` showing the 52 decap slots, the IC port(s) and the power-bus outline.]

A.2 Batch file format and simulation phases

A.2.1 PEB structure

A PEB file is XML. Its root element is `<Batch Analysis="PI" Version="2" PowerBus="{powerbus` and it contains an `<Iterate>` list of `<Group>` blocks, one group per layout row. Each group first writes the 52 component enable lines, then the analysis command for that group.

For the multi-type campaign the per-component block grows from one line to four: Component/Name, Component/Value (capacitance), PI Decap/ESR and PI Decap/ESL. Empty slots write `false` and a value of 0.

Table A.4: Elements emitted inside a PEB group.

Purpose	Emitted line
Slot occupancy (52 lines)	<code><Edit Table="Component" Name="C{i+1}" Column="Name" Value="true false"/></code> for C1...C52
PI-Distribution group	<code><Loop Analysis="PI-Distribution" Count="1"><EditPIDistribution Frequency="{freq}"/></Loop></code>
PI-Spectrum group	<code><CreatePISpectrum Component="{components}" Format="ARV,CSV" /></code>
Frequency string format	<code>f"{int(round(float(mhz)))}e6" (Hz)</code>

A.2.2 Two-phase dataset simulation

Dataset simulation is split into an impedance phase and a per-band distribution phase. Both phases iterate over the same layout rows, and each produces one PI- n output folder per row.

Table A.5: Simulation phases and their harvested artifacts.

Phase	PEB builder	Harvested file
Impedance	<code>build_impedance_peb,</code> spectrum only (<code>include_distribution=False</code>), one PEB over all rows	first *.csv in the PI folder whose name contains ic1 (case-insensitive)
Distribution	<code>build_distribution_peb,</code> distribution only (<code>include_spectrum=False</code>), one PEB per MHz band	<code><PI-n>/Power_GND/Z_{MHz:04d}.000MHz.map</code>

After a batch, `wait_for_pi_outputs` polls the EMC directory until PI-1 through PI-N are present, checking for .map files when the mode is `distribution` and for .csv files when the mode is `impedance`. CSV row i of the layout list maps to PI- $(i + 1)$ in eCADSTAR script order. In the merged combination list the legacy rows keep the low indices, PI-1...PI-19499, and the newer combinations follow at PI-19500...PI-29499.

A.2.3 Pipeline configuration constants

The runnable pipelines take no command-line arguments; their behaviour is fixed by a configuration block of module-level constants. The values below are those present in the block at the time of reading. **[FILL IN: establish whether SWEEP_MHZ was edited between campaigns and which band sets were actually simulated; logs/append_queue/done.jsonl records the bands that completed]**

Appendix A. Power Integrity Simulation Environment and Dataset Specification

Table A.6: Single-type combinations simulation pipeline (pipelines/dataset_sim/run_combinations_sim_pipeline.py).

Constant	Value
SWEEP_MHZ	[170, 180, 280, 430, 470, 490, 550, 590]
Input CSV	data/heatmaps/peb_with_29k/all_combinations_merged
ECADSTAR_WAIT_TIMEOUT_SEC	172_800
ECADSTAR_WAIT_POLL_SEC	120

[FILL IN: note whether the 490, 550 and 590 MHz bands in SWEEP_MHZ ever reached the dataset — they are absent from the 24 anchors in datasets/data_multifreq; the append queue in logs/append_queue/done.jsonl would settle it]

A.2.4 Active-learning batches

The active-learning loop exports all selected candidates of a cycle into one combined PEB (active_learning_pi/al/peb_batch.py, build_peb_from_selection), carrying a per-sample frequency formatted as "{int(round(mhz))}e6". The builder sets include_distribution=True and include_spectrum = not heatmap_only. In the primary configuration both heatmap_only_peb and heatmap_only_labels are true, so AL batches request PI-Distribution output only.

Table A.7: eCADSTAR settings in the primary AL configuration (active_learning_pi/config/exp059_gp_error.json, key ecadstar).

Key	Value
clear_lock_file	true
wait_for_pi	true
wait_timeout_sec	18000
wait_batch_start_timeout_sec	900
wait_poll_sec	40

Label ingestion (active_learning_pi/al/ingest_labels.py) reads PI-1 through PI-N from the EMC directory, copies the .map file for each candidate's frequency into labels_dir/sample_{i:03d}/, and writes occupancy.npy, pi_freq_mhz.npy and heatmap.npy, the last produced by create_Heatmaps at grid size 64.

A.3 From solver output to tensors

A.3.1 Decap slots and occupancy vectors

Decap slots are arranged on a 7×8 board grid. Four cells are declared invalid — (0, 3), (0, 4), (6, 3) and (6, 4) — leaving $7 \times 8 - 4 = 52$ valid positions, labelled C1 through C52.

Appendix A. Power Integrity Simulation Environment and Dataset Specification

A placement is therefore the binary vector $\mathbf{b} \in \{0, 1\}^{52}$, with budget $K = \sum_i b_i$. On disk an occupancy vector is a length-52 float32 array; it is binarised with a threshold of 0.5.

[FIGURE: The 7x8 slot grid with the four invalid cells shaded and the C1–C52 labelling order.]

A.3.2 Impedance spectra

The exported PI-Spectrum CSV is read with delimiter `;`, `skiprows=4` and `usecols=[1]`. A vector is rejected unless its length is exactly 231 (`EXPECTED_IMP_LENGTH = 231`). The impedance $Z(f)$ used throughout the thesis is this 231-bin vector.

Table A.8: Frequency axis and target curve stored under `configs/`.

Artifact	Contents
<code>configs/Frequency_data_hz.npy</code>	shape (231,), minimum 1000000.0 Hz, maximum 600000000.0 Hz; first values 1000000, 1100000, 1200000, 1300000, 1400000; last values 5.8e+08, 5.9e+08, 6.0e+08
<code>configs/target_impedance.npy</code>	shape (231, 1), minimum 0.8, maximum 49.99999999999999

Neither file carries metadata about its own construction. **[FILL IN: document how the 231-point frequency grid was generated (the `.npy` has no provenance); the eCADSTAR sweep definition in `H-shape.erf` would settle it] [FILL IN: document how $Z_{\text{target}}(f)$ in `configs/target_impedance.npy` was specified — per-band values, derivation, and the design rule behind the 0.8 and 50 limits]**

A.3.3 PI-distribution heatmaps

Each `.map` file is parsed and interpolated onto a fixed 64×64 grid. The steps implemented in `libs/data_creation/heatmap.py` are listed below in the order applied.

A.4 Dataset layout and metadata

A.4.1 Directory structure

Datasets use a layout-store organisation: per-layout quantities are written once under a design directory, and per-sample quantities (one per layout–frequency pair) are written under flat sample directories.

Table A.9: Heatmap construction from a PI-Distribution .map file.

Step	Setting
Coordinate handling	shifted to the minimum, scaled by $1e-5$
Value floor	z values floored at 0.01
Duplicate vertices	averaged
Interpolation	<code>RBFInterpolator</code> , kernel <code>thin_plate_spline</code> , <code>smoothing=0</code>
Fallback	cubic <code>griddata</code> with nearest-neighbour NaN fill
Grid size	64
Stored tensor	<code>np.stack([Zi * mask_board, mask_board], axis=0)</code> : channel 0 is the masked interpolated impedance, channel 1 is the mask
Board mask	<code>configs/binary_mask.npy</code> , (64, 64) boolean, 3432 True pixels

Table A.10: Layout-store dataset structure.

Path	Contents
<code>layouts/{design_id}/occ.npy</code>	occupancy vector, shape (52,)
<code>layouts/{design_id}/imp.npy</code>	impedance, reshaped to (-1, 1), i.e. (231, 1) in the raw store
<code>heatmap/sample_N.npy</code>	the 2-channel 64×64 stack
<code>PI_freq/sample_N.npy</code>	the sample's frequency in Hz, <code>float64</code>
<code>manifest.csv</code>	one row per sample
<code>dataset_meta.json</code>	dataset-level metadata

Appendix A. Power Integrity Simulation Environment and Dataset Specification

The manifest columns are `sample_name`, `design_id`, `freq_label`, `freq_mhz`, `freq_hz`, `source_folder`, `append_tag`, `pi_number` and `decap_index` — nine columns in total.

A.4.2 `dataset_meta.json` fields

Table A.11: Fields written by `libs/dataset_meta.py`.

Field	Notes
<code>schema_version</code>	integer
<code>stage</code>	raw or normalized
<code>source_script</code>	the script that produced the dataset
<code>generated_at_utc</code>	ISO-8601 UTC timestamp
<code>dataset_root</code>	absolute path to the dataset root, as written by <code>libs/dataset_meta.py</code> line 172
<code>storage</code>	layout_store
<code>counts</code>	manifest_rows, unique_layouts, heatmap_files, pi_freq_files, layout_directories
<code>size</code>	total_bytes, total_mb, by_subdirectory_mb
<code>pi_frequencies_mhz</code>	the anchor list
<code>samples_per_mhz</code> , <code>samples_per_freq_label</code>	per-band coverage
<i>Additional fields on normalized sets:</i>	
<code>normalization</code>	stats_file, has_heatmap_stats
<code>mode</code>	e.g. full
<code>source_dataset</code>	the raw set it derives from; also written as an absolute path
<code>heatmap_norm</code>	normalization mode name
<code>max_k_filter</code>	budget cut-off applied

A.4.3 Dataset inventory and version control

None of the dataset directories are under version control. The repository `.gitignore` ignores `datasets/`, `data_multi_norm_robust/`, `checkpoints/`, `**/checkpoints/`, `*.pt` and `*.pth`; `git check-ignore` confirms that `datasets/data_multifreq_train/manifest.csv` and `datasets/data_multifreq_train_norm_unbounded/normalization_stats.json` are matched by `.gitignore` line 20 (`datasets/`). Reproducing any result in this thesis therefore requires rebuilding the datasets from the raw eCADSTAR output on the Windows host, or obtaining the directories separately.

`datasets/data_multifreq_al_overlay_exp060` is referenced by `experiments/exp060_multity` line 23 but does not exist on disk.

The current model track, `exp059_capacity_freq`, trains from `data_dir datasets/data_multifreq`. The primary AL configuration uses the same `data_dir`, `normalization_stats_json`

Appendix A. Power Integrity Simulation Environment and Dataset Specification

Table A.12: Dataset directories present on the working machine (all git-ignored). Paths are absolute.

Path	Role
/home/ubuntu/genai_pdn/datasets/data_multifreq_train	raw training set
/home/ubuntu/genai_pdn/datasets/data_multifreq_train_norm_unbounded	normalized training, imported for exp059
/home/ubuntu/genai_pdn/datasets/data_multifreq_train_norm_robust	normalized training, earlier experiments
/home/ubuntu/genai_pdn/datasets/data_multifreq_train_K5	freq 5 train K5
/home/ubuntu/genai_pdn/datasets/data_multifreq_overlay_exp057	Alec_vallayver(overlay)exp057
/home/ubuntu/genai_pdn/datasets/data_multifreq_overlay_exp058	Alec_vallayver(overlay)exp058
/home/ubuntu/genai_pdn/datasets/data_multifreq_overlay_exp059	Alec_vallayver(overlay)exp059
/home/ubuntu/genai_pdn/data_multi_norm_robust	older backup at the repository root

datasets/data_multifreq_train_norm_unbounded/normalization_stats.json and overlay_data_dir datasets/data_multifreq_al_overlay_exp059.

[FILL IN: state the train/validation/test split definition for the normalized dataset — no split artifact was located in the dataset directories read for this appendix; the split logic in the exp059 dataloader would settle it]

A.5 Dataset statistics

A.5.1 Layout pool

The layout pool is a merged combination list. The merge report (data/heatmaps/merged_combinations generated 2026-07-01T13:13:39.245630+00:00) records `n_old` 19499 rows from `all_combinations` `n_new` 10000 rows from `combinations.csv`, `overlap_rows` 0, giving 29499 merged rows. The merged CSV read by the simulation pipeline is `data/heatmaps/peb_with_29k/all_combinations.csv` which has 29499 lines. The output path recorded in the merge report itself (`data/heatmaps/all_combinations.csv`) no longer exists at that location; see Section A.8.

A.5.2 Raw training set

This dataset root is written to by the append worker, which was still being triggered while this appendix was assembled; the values below are the state at the read time given in the table and must be re-read before submission. The counts have not changed across the re-reads made for this appendix, but `generated_at_utc` has.

[FILL IN: explain why the 80 MHz band covers only 10000 of the 29499 layouts — no artifact was located that records the reason; the per-band append log in logs/append_queue/done.jsonl and the 80 MHz raw folder]

Table A.13: `datasets/data_multifreq_train` (raw), as read on 2026-08-06.

Field	Value
<code>schema_version</code>	1
<code>stage</code>	raw
<code>source_script</code>	<code>pipelines/data/append_merged_combinations_multifre</code>
<code>generated_at_utc</code>	2026-08-05T19:28:42.489387+00:00
<code>dataset_root</code>	<code>/home/ubuntu/genai_pdn/ datasets/data_multifreq_train</code>
<code>storage</code>	<code>layout_store</code>
<code>manifest_rows</code>	688477
<code>unique_layouts</code>	29499
<code>heatmap_files</code>	688477
<code>pi_freq_files</code>	688477
<code>layout_directories</code>	29499
<code>total_mb</code>	43307.27
<code>heatmap</code>	43113.86
<code>layouts</code>	39.05
<code>PI_freq</code>	89.3
<code>samples_per_mhz</code>	29499 at every anchor except 80 MHz, which has 10000

count would settle it]

A.5.3 Frequency anchors

The anchor list is the set of inspection frequencies at which PI-Distribution heatmaps exist. `datasets/data_multifreq_train/dataset_meta.json` lists 24 anchors, in MHz: 10.0, 63.0, 80.0, 100.0, 120.0, 150.0, 170.0, 180.0, 200.0, 230.0, 250.0, 270.0, 280.0, 300.0, 330.0, 350.0, 370.0, 390.0, 400.0, 420.0, 430.0, 450.0, 470.0, 500.0.

Anchor resolution at load time follows a four-step order, per `docs/dataset.md`: `dataset_meta.json` `pi_frequencies_mhz`, then `manifest.csv` `freq_mhz`, then `datasets/data_multifreq_train` then the legacy YAML defaults. That documentation statement was not verified against the implementation in `src_vae/others/multifreq_anchors.py` line by line.

Three sources give three different anchor counts. `docs/dataset.md` lines 96–101 describe a “Full 16-MHz set” and “Same 16 MHz in combinations sim batches”; the legacy fallback `configs/multifreq_anchors.yaml` lists 20 anchors (10, 63, 80, 100, 120, 150, 170, 200, 230, 250, 270, 300, 330, 350, 370, 390, 400, 420, 450, 500), omitting 180, 280, 430 and 470 MHz; `dataset_meta.json` lists the 24 above. Per the resolution order in `docs/dataset.md` lines 42–51, the `dataset_meta.json` list is what is actually loaded, and `configs/multifreq_anchors.yaml` is declared a fallback used only when no metadata or manifest is present. **[FILL IN: resolve the conflict between the “16-MHz set” description in `docs/dataset.md` lines 96–101, the**

20-anchor list in `configs/multifreq_anchors.yaml`, and the 24-anchor list in `datasets/data_multifreq_train/dataset_meta.json`; decide which is stale and correct the documentation]

[FILL IN: document the provenance of the 24-anchor selection — no artifact was located that records why these particular frequencies were chosen]

A.5.4 Normalized training set

Table A.14: `datasets/data_multifreq_train_norm_unbounded` (normalized; the exp059 training input).

Field	Value
<code>stage</code>	<code>normalized</code>
<code>source_script</code>	<code>pipelines/normalize/multifreq.py</code>
<code>generated_at_utc</code>	<code>2026-07-06T13:39:03.768907+00:00</code>
<code>mode</code>	<code>full</code>
<code>source_dataset</code>	<code>/home/ubuntu/genai_pdn/ datasets/data_multifreq_train</code>
<code>heatmap_norm</code>	<code>robust_log1p_per_mhz_unbounded</code>
<code>max_k_filter</code>	<code>30</code>
<code>manifest_rows</code>	<code>582997</code>
<code>unique_layouts</code>	<code>24979</code>
<code>heatmap_files</code>	<code>582997</code>
<code>pi_freq_files</code>	<code>582997</code>
<code>layout_directories</code>	<code>24979</code>
<code>total_mb</code>	<code>9344.12</code>
<code>heatmap</code>	<code>9180.49</code>
<code>layouts</code>	<code>33.06</code>
<code>PI_freq</code>	<code>75.61</code>
<code>samples_per_mhz</code>	<code>24979 at every anchor except 80 MHz, which has 8480</code>
<code>pi_frequencies_mhz</code>	<code>the same 24 anchors as the raw set</code>

The budget filter is applied by `pipelines/normalize/build_train_norm_unbounded.py`, which sets `DATA_DIR` to `datasets/data_multifreq_train`, `OUTPUT_DIR` to `datasets/data_multifreq_train_norm_unbounded`, and `MAX_K` to 30, so only layouts with $K \leq 30$ enter the normalized set. Note that this normalized set was generated on 2026-07-06, whereas the raw set it names as its source was last refreshed on 2026-08-05; the two artifacts are not snapshots of the same moment, and the $K \leq 30$ filter accounts for only part of the difference in counts.

A.5.5 $K \leq 5$ subset

The same report carries `k_hist_all`, the budget histogram of the full raw pool. Selected entries: $K = 2$: 1326; $K = 3$: 1483; $K = 4$: 1440; $K = 5$: 1401; $K = 20$:

Table A.15: datasets/data_multifreq_train_K5 and its dry-run report.

Field	Value
source_script	pipelines/data/copy_multifreq_k_subset.py
generated_at_utc	2026-08-02T17:03:40.514745+00:00
manifest_rows	131766
unique_layouts	5650
total_mb	8288.11
pi_frequencies_mhz	the same 24 anchors
samples_per_mhz	5650 at every anchor except 80 MHz, which has 1816
<i>Dry-run report (datasets/data_multifreq_train_K5_dry_run_report.json, 2026-08-02T17:03:40.514745+00:00)</i>	
max_k	5
designs_total / designs_kept	29499 / 5650
manifest_rows_total	/ 688477 / 131766
..._kept	

912; $K = 21$: 319; and 226 each for $K = 26$ through $K = 50$. **[FILL IN: reproduce the complete k_hist_all histogram as a table or bar chart from datasets/data_multifreq_train_K5_dry_run_report.json if the budget distribution is discussed in the main text]**

A.5.6 Active-learning overlay (exp059)

Table A.16: datasets/data_multifreq_al_overlay_exp059, read from disk on 2026-08-05.

Item	Value
manifest.csv	2001 lines (header plus 2000 rows)
Layout directories	2000
Subdirectories	PI_freq/, heatmap/, layouts/
Other files	normalization_stats.json, al_heatmap_only_design_ids.json
dataset_meta.json	absent
design_id form	e.g. al_al_exp059_gp_error_001_i0001_c009
source_folder	active_learning
Overlay	same key structure as the training set;
normalization_stats.json	background_value -3.1792, storage layout_store, max_k_filter 30

Because the overlay has no dataset_meta.json, its counts were read from the directory listing rather than from a written metadata artifact.

A.6 Normalization statistics

A.6.1 File structure

`normalization_stats.json` has seven top-level keys: `background_value`, `Heatmap`, `Impedance`, `PI_freq`, `multifreq_manifest`, `storage` and `max_k_filter`. It is assembled in `pipelines/normalize/multifreq.py`.

A.6.2 Heatmap normalization

Per-MHz heatmap statistics are computed on $\log(1 + x)$ of the foreground pixels of that band. The median and the interquartile range (IQR, $q_{75} - q_{25}$, floored at `ROBUST_IQR_EPS = 1e-4`) define a robust z -score $z = (\log(1 + x_{\text{fg}}) - \text{median})/\text{IQR}$. `clip_min` is the 0.5th percentile of z and `clip_max` the 99.5th; the background fill value is `round(clip_min - 1.5, 4)`. In *unbounded* mode the z -score is applied without clipping to `[clip_min, clip_max]`; the clip is applied only when the mode is not unbounded. The stored statistics still record the clip bounds.

Table A.17: Global heatmap statistics, `datasets/data_multifreq_train_norm_unbounded/normal` key `Heatmap`.

Key	Value
<code>norm_mode</code>	<code>robust_log1p_per_mhz_unbounded</code>
<code>unbounded</code>	<code>true</code>
<code>clip_min</code>	<code>-1.6791870594024658</code>
<code>clip_max</code>	<code>6.275787830352783</code>
<code>z_max</code>	<code>11.23223876953125</code>
<code>background_value</code>	<code>-3.1792</code>
<code>log_mean</code>	<code>0.0</code>
<code>log_std</code>	<code>1.0</code>
<code>fg_pixel_count</code>	<code>2000845704</code>
<code>anchors_mhz</code>	<code>length 24</code>
<code>by_mhz</code>	<code>24 keys</code>

Each `by_mhz` entry carries `median`, `iqr`, `clip_min`, `clip_max`, `background_value`, `fg_pixel_count`, `z_min` and `z_max`. The first four bands are reproduced below; the remaining 20 are in the same file under the same key.

Table A.18: Per-MHz heatmap statistics, first four anchors.

MHz	median	iqr	clip_min	clip_max
10.0	0.05887173116207123	0.03866304084658623	-0.7351016402244568	3.4719114303588867
63.0	0.41589492559432983	0.23903468251228333	-0.6570155620574951	3.969606876373291
80.0	0.5136411190032959	0.29707127809524536	-0.6531781554222107	6.275787830352783
100.0	0.6201565265655518	0.357897512614727	-1.2004997730255127	5.350724220275879

[FILL IN: if the full per-band table is wanted in the thesis, extract all 24 by_mhz entries from datasets/data_multifreq_train_norm_unbounded/normalization_statistics/]

A.6.3 Impedance and frequency normalization

Impedance is normalized by a natural-log z -score, $z = (\log(\max(Z, 1e-10)) - \log_mean) / \log_std$, and saved per layout as `imp.npy` of shape `(1, n_freq)` in the normalized store.

Table A.19: Impedance and `PI_freq` statistics blocks of the normalized training set.

Key	Value
<code>Impedance.layout_count</code>	24979
<code>Impedance.log_mean</code>	-1.039238452911377
<code>Impedance.log_std</code>	1.7831873893737793
<code>Impedance.z_min</code>	-2.7170498371124268
<code>Impedance.z_max</code>	2.474900722503662
<code>PI_freq.min_hz</code>	1000000.0
<code>PI_freq.max_hz</code>	600000000.0
<code>PI_freq.log10_min</code>	6.0
<code>PI_freq.log10_range</code>	2.778151250383644
<code>PI_freq.count</code>	688477
<code>PI_freq.anchor_mhz</code>	length 24
<code>PI_freq.unique_mhz_rounded</code>	length 24
<code>PI_freq.expected_anchors_mhz</code>	length 24
<code>multifreq_manifest.rows</code>	582997
<code>multifreq_manifest.unique_design_id</code>	24979
<code>multifreq_manifest.max_k_filters</code>	30
<code>multifreq_manifest</code>	per- 24979 at every anchor except 80.0 MHz with
MHz counts	8480

`PI_freq.count` (688477) equals the *raw* manifest row count, while `multifreq_manifest.rows` in the same file is 582997, the normalized manifest row count. This is recorded as an inconsistency in Section A.8 rather than resolved here.

A.7 Multi-type decap campaign (exp060) — in flux

The multi-type occupancy fork replaces the binary slot state with a per-slot type code. Its simulation campaign was still running when the evidence for this appendix was first collected and has since completed all 24 bands; the figures below were re-read on 2026-08-06 and are marked with their source. What has *not* happened is ingestion: no multi-type dataset directory was found under `datasets/`, so nothing in this section is a dataset specification, and exp060 still has no training input of its own.

A.7.1 Type catalog and combination list

Table A.20: Multi-type decap catalog (`scrap/generation/generate_peb_multitype.py`, `TYPE_CATALOG`).

Type	C	ESR	ESL
1	100e-9 F	8.9e-3 Ω	222e-12 H
2	47e-9 F	21.4e-3 Ω	154e-12 H
empty	false and 0	—	—

The combination list `data/heatmaps/combinations_multitype_combined_15000.csv` holds 15000 rows of 52 integer type codes (0 empty, 1 type-1, 2 type-2). Its sampler report (`data/heatmaps/combinations_multitype_sample_report.json`, seed 42, generated 2026-08-02T18:07:25.393879+00:00) records `per_k {1:104, 2:5304, 3:4797, 4:3197, 5:1598}` families `type1_only 4576 / type2_only 4576 / mixed 5848`, and `nonzero_type_counts {1:23015, 2:22866}`. `docs/multitype_peb_pipeline.md` contradicts itself on these numbers; see Section A.8.

A.7.2 Simulation pipeline configuration

`pipelines/dataset_sim/run_multitype_sim_pipeline.py` reads the 15000-row CSV from row 0 with `MAX_LAYOUTS None`, `USE_ANCHOR_MHZ False` and an explicit `SWEEP_MHZ` list of the same 24 anchors (10...500 MHz). `TRIGGER_APPEND_AFTER_MOVE` is `True`.

The append worker started by that pipeline (`pipelines/dataset_sim/run_append_worker.py`) runs `pipelines/data/append_merged_combinations_multifreq.py` with `OUTPUT_ROOT` set to `datasets/data_multifreq_train`, i.e. the single-type dataset root; that is the worker’s only output root. The worker log for the most recent jobs reports “Tasks to run: 0” and “Nothing to append”. This is stated as observed configuration, with no interpretation of intent. **[FILL IN: resolve the conflict between `TRIGGER_APPEND_AFTER_MOVE=True` in `run_multitype_sim_pipeline.py` line 120 and the single-type-only `OUTPUT_ROOT` in `run_append_worker.py` lines 44–46; decide where multi-type raw output is meant to be ingested and record the intended dataset root]**

Append jobs are queued per MHz in `logs/append_queue/pending.jsonl`, and completed jobs are recorded in `logs/append_queue/done.jsonl` with fields `mhz`, `append_tag`, `raw_root`, `enqueued_at`, `started_at`, `finished_at` and `returncode`. As read on 2026-08-06 the file holds 24 records, one per band, all with `raw_root C:\Users\muthusamy\Desktop\Raw`. The `merged_470` record is dated 2026-07-05; the other 23 fall between 2026-08-03 and 2026-08-05. The last record is `mhz 500`, `append_tag merged_500`, `enqueued_at 2026-08-05T19:27:53.160450+00:00`, `finished_at 2026-08-05T19:29:02.065218+00:00`,

Appendix A. Power Integrity Simulation Environment and Dataset Specification

returncode 0. Twenty-three of the 24 records carry returncode 0; merged_80 (finished_at 2026-08-03T17:38:40.736791+00:00) carries returncode 1.

Three timestamps sit close together and are recorded here without an inference about which process wrote what: merged_500 was enqueued at 2026-08-05T19:27:53.160450+00:00 the multi-type progress file was last updated at 2026-08-05T19:27:53.164372+00:00 (Table A.21), and datasets/data_multifreq_train/dataset_meta.json carries generated_at_utc 2026-08-05T19:28:42.489387+00:00 (Table A.13). **[FILL IN: establish whether any multi-type raw output was appended into datasets/data_multifreq_train by the append jobs of 2026-08-03 to 2026-08-05. The counts in Table A.13 are unchanged from the earlier read, which does not by itself settle it; the append worker’s per-job log for merged_500, and the append_tag values in datasets/data_multifreq_train/manifest.csv, would.]**

A.7.3 Campaign state

An earlier read of these artifacts caught the pipeline mid-run, with 23 of 24 bands complete. The values below replace that snapshot: the simulation process (`python -u pipelines/dataset_sim/run_multitype_sim_pipeline.py`) is no longer running, and the progress file records all 24 bands as done. This is the state of the *raw simulation output on the Windows host*, not of a dataset.

Table A.21: Multi-type simulation state. The progress-file rows were re-read on 2026-08-06; the folder counts were taken on 2026-08-05 after the final progress update.

Item	Value at the observation timestamp
Progress file	C:\Users\muthusamy\Desktop\Raw\sim_progress_multit
impedance_done	true
distribution_mhz_done	24 bands: 10, 63, 80, 100, 120, 150, 170, 180, 200, 230, 250, 270, 280, 300, 330, 350, 370, 390, 400, 420, 430, 450, 470, 500 — the same 24 anchors as the single-type set
updated_at	2026-08-05T19:27:53.164372+00:00
Band folders inspected	the Impedance folder and the heatmaps_450MHz and heatmaps_500MHz folders under the Raw root each contain 15000 PI folders, matching the 15000-row combination list
Multi-type dataset directory	none found under datasets/

[FILL IN: confirm the PI-folder count for the remaining 21 band folders under the Raw root; only Impedance, heatmaps_450MHz and heatmaps_500MHz were counted, and a completed band in distribution_mhz_done is not by itself proof of 15000 output folders]

[FILL IN: record the dataset root that the multi-type raw output is ingested into, once ingestion has happened, together with the resulting counts, anchor list and normalization statistics; until then exp060 has no dataset specification and no result from it can be reported]

A.7.4 exp060 configuration

experiments/exp060_multitype_occ/config.yaml sets `n_decap_types` to 2 (line 193) while still pointing `data_dir` at `datasets/data_multifreq_train_norm_unbounded` (line 134), which is the single-type normalized dataset. No multi-type normalized dataset directory was found under `datasets/` at the read times given in Table A.21, so the completion of the simulation campaign has not yet produced a training input for exp060. [FILL IN: update or explain the exp060 `data_dir`; it cannot supply the $52 \times T$ multi-type occupancy that `n_decap_types = 2` implies]

A.8 Recorded inconsistencies between artifacts

Five disagreements between repository artifacts were found while assembling this appendix. They are listed rather than silently resolved, because in each case the authoritative source is either ambiguous or the artifacts were written at different times.

[FILL IN: decide, for each row of Table A.22, which artifact is authoritative and either correct the stale one in the repository or state the resolution in the thesis text]

A.9 Reproducibility notes

Two constraints limit reproduction of the work in this appendix. First, the simulation half is bound to one Windows host: `engineer.exe`, the `.erf` design, the PEB staging directories and the Raw output root are all absolute `C:\Users\muthusamy\...` paths, and the AL loop cannot run its simulation steps without that machine. Second, every dataset directory is git-ignored, so the repository alone does not carry the data.

[FILL IN: record a wall-clock simulation cost per layout and per band; docs/ecadstar_headless_cli.md lines 113–121 gives an observed rate for one resumed band, but no systematic timing artifact exists. A per-band timing table from logs/append_queue/done.jsonl start/finish timestamps would settle it]

[FILL IN: state whether H-shape.erf and the derived datasets can be released, and under what terms, so the reproducibility statement is complete]

Table A.22: Conflicts between artifacts. Both sides are named; none is resolved here.

Subject	Source A	Source B
Anchor count	docs/dataset.md:96–101, “Full 16-MHz set”; configs/multifreq_anchors.yaml:20 anchors	datasets/data_multifreq_train/dataset_manifest.yaml:24 anchors (the list actually ordered, per the resolution order in docs/dataset.md:42–51)
PI_freq.count	PI_freq.count = 688477, the raw manifest row count	multifreq_manifest.rows = 582997 in the same file, datasets/data_multifreq_train_norm_unbo
Artifact staleness	normalized meta generated 2026-07-06T13:39:03; 24979 layouts / 582997 rows	raw meta generated 2026-08-05T19:28:42; 29499 layouts / 688477 rows. The $K \leq 30$ filter explains part of the gap, but the two are not from the same moment
Multi-type sampler	docs/multitype_peb_pipeline.yaml:77: per- K and the CSV on disk: per- K {2:3750, 3:3750, 4:3750, 5:3750}, families 5000 each, nonzero type counts {1:26258, 2:26242}	data/heatmaps/combinations_multitype_sampler.csv:1:104, 2:5304, 3:4797, 4:3197, 5:1598}, families 4576/4576/5848, nonzero {1:23015, 2:22866}. The same doc at lines 66–68 quotes the report’s numbers, so it contradicts itself
Merged path	CSV merged_combinations_report.json 29499-row merged key output_csv and CSV on disk is docs/dataset.md:78–81 point to which is what data/heatmaps/all_combinations_merged_combinations_sim_pipeline.py:50 which is not present there reads	
Append root	worker run_multitype_sim_pipeline.py:120 sets TRIGGER_APPEND_AFTER_MOVE=True for the multi-type campaign	py:44–46 has OUTPUT_ROOT = REPO_ROOT/datasets/data_multifreq_train the single-type dataset, as its only output root

Appendix B

Hyperparameter Configuration for Training and Optimisation

B.1 Scope and how to read these tables

This appendix records the configuration files that drive training and fine-tuning of the surrogate `exp059_capacity_freq`, the fine-tune override files that sit on top of the base configuration, and the `CONFIG` block of the Stage-2 latent-optimisation script. It is a transcription, not an analysis: every key name and every value is reproduced exactly as it appears in the source file, with no rounding, no unit conversion and no reformatting.

These are the current contents of the configuration files, not a run record. `config.yaml` has been edited since the capacity-phase training run — at least in the two keys `layout_train_prob` and `occ_only_encode_prob`, discussed as C5 in [section B.5](#) — and the fine-tune configuration that the active-learning loop actually launches is regenerated at every launch and overwritten (C1). Nothing in this appendix may be read as a per-run snapshot of the values that were in force when a particular checkpoint was produced.

The training configuration lives in `experiments/exp059_capacity_freq/config.yaml`. Despite the extension, the file is a JSON document with 183 top-level keys. It is read by `load_yaml_config()` (`experiments/exp059_capacity_freq/codes/exp059_common.py`, lines 17–25), which discards blank lines and every line whose first non-space character is `#`, then calls `json.loads` on the remainder. Consequently YAML syntax — bare keys, `-` lists, anchors — is not accepted. The base `config.yaml` contains no comment lines at all; the two override files do.

The 183 keys are grouped below into 21 tables. **The grouping is editorial:** keys belonging to one table are in general not contiguous in the file, and the file itself imposes no ordering or sectioning. The grouping was introduced here so that the tables

are readable; it should not be taken as structure present in the artifact.

None of the 183 keys carries a comment or any other in-file documentation, so this appendix cannot state why an individual value was chosen. **[FILL IN: for the values that were actually tuned rather than inherited from exp057/exp058, add the provenance — the sweep, ablation or hand-tuning step that fixed them. experiments/exp059_capacity_freq/notes.md would be the artifact to check; no sweep record was located during evidence extraction.]**

B.2 Surrogate training configuration: exp059_capacity_freq

B.2.1 Latent structure and architecture keys

Five of the architecture-level switches are set explicitly in `config.yaml`. The remaining architecture switches are not present in the file and fall back to code defaults; they are listed separately in [section B.3](#) so that this table does not imply they were configured.

Table B.1: Latent and architecture keys set in `config.yaml` (5 keys).

Key	Value
<code>latent_dim</code>	128
<code>heatmap_private_dim</code>	40
<code>cond_dim</code>	32
<code>freq_fourier_features</code>	16
<code>use_heatmap_film</code>	true

B.2.2 Training schedule, optimiser and learning-rate scheduler

[Table B.2](#) gives the schedule keys. The optimiser itself is not configurable from the file: `train_core.py` (lines 906–920) constructs `torch.optim.AdamW` with `weight_decay = 1e-4`, first attempting the fused implementation on CUDA, then the `foreach` implementation, and falling back to plain `AdamW`. The learning-rate schedule is `ReduceLROnPlateau(mode="min", factor=c.lr_factor, patience=c.lr_patience, min_lr=c.lr_min)` (`train_core.py`, lines 1431–1433), which with the values in [Table B.2](#) resolves to factor 0.5, patience 15 and minimum learning rate `1e-05`.

On resume with `reset_lr_on_resume` set, every parameter group’s learning rate is reset to `learning_rate`, except parameter group index 1 when the resume epoch is at or past `impedance_peak_focus_epoch`, which is set to `learning_rate × impedance_decoder_lr_mult`; the scheduler is then recreated from scratch (`train_core.py`, lines 1442–1451).

Table B.2: Training schedule and scheduler keys (13 keys).

Key	Value
num_epochs	510
batch_size	224
val_batch_size	320
learning_rate	1.5e-05
lr_min	1e-05
lr_patience	15
lr_factor	0.5
train_split	0.9
num_workers	4
reset_lr_on_resume	true
train_samples_per_epoch	64000
impedance_decoder_lr_mult	1.0
impedance_separate_lr	true

B.2.3 Data splitting and class balancing

Table B.3: Data split and balancing keys (7 keys).

Key	Value
balance_k	false
k_balance_power	0.4
k_balance_smoothing	0.001
stratify_by_k	true
split_by_design	true
balance_freq	false
freq_balance_power	1.4

Note that `balance_k` and `balance_freq` are both `false` while `k_balance_power`, `k_balance_smoothing` and `freq_balance_power` carry non-default-looking values, i.e. the two weighting exponents are present but inactive under this configuration. **[FILL IN: confirm from `train_core.py` whether `k_balance_power` and `freq_balance_power` are read at all when the corresponding `balance_*` flag is false, and state the sampler behaviour that is actually used.]**

B.2.4 Dataset, overlay and checkpoint paths

B.2.5 Logging and validation cadence

Table B.4: Dataset, overlay and checkpointing keys (8 keys).

Key	Value
data_dir	datasets/data_multifreq_train_norm_unbounded
experiment_dir	experiments/exp059_capacity_freq
al_overlay_data_dir	datasets/data_multifreq_al_overlay_exp059
al_overlay_sample_weight	128.0
resume_checkpoint	experiments/exp059_capacity_freq/checkpoints/last_model.pt
recalculate_curriculum_on_resume	false
checkpoint_interval	5
keep_last_n_checkpoints	0

Table B.5: Logging and validation cadence keys (3 keys).

Key	Value
val_on_checkpoint_only	true
epoch_print_interval	1
epoch_log_interval	2

B.2.6 Heatmap loss weights

The heatmap branch carries the largest number of individually weighted loss terms. [Table B.6](#) lists the general reconstruction and structure weights, [Table B.7](#) the peak- and valley-region terms, and [Table B.8](#) the keys governing the heatmap focus phase. **[FILL IN: describe, in Chapter 3 rather than here, what each heatmap loss term penalises and why the relative weights were set this way; the config file records no rationale.]**

Table B.6: Heatmap reconstruction and structure loss weights (15 keys).

Key	Value
heatmap_weight	5.0
heatmap_phys_p99_weight	2.0
heatmap_phys_p99_percentile	99.0
heatmap_peak_weight	3.0
heatmap_grad_weight	1.5
heatmap_lap_weight	2.0
heatmap_contrast_weight	1.5
heatmap_contrast_margin	0.5
heatmap_bg_weight	0.5
heatmap_dynrange_weight	2.0
heatmap_pearson_weight	0.75
heatmap_grad_vector_weight	2.0
heatmap_grad_direction_weight	1.0
heatmap_grad_direction_min_mag	0.08
heatmap_grad_huber_delta	0.5

Table B.7: Heatmap peak- and valley-region loss terms (11 keys).

Key	Value
heatmap_peak_centroid_top_q	0.95
heatmap_valley_centroid_bottom_q	0.05
heatmap_peak_topregion_weight	2.5
heatmap_valley_topregion_weight	2.5
heatmap_peak_topregion_q	0.95
heatmap_valley_topregion_q	0.05
heatmap_topregion_huber_delta	0.5
heatmap_peak_loc_weight	2.5
heatmap_peak_loc_temperature	0.035
heatmap_valley_loc_weight	0.0
heatmap_valley_loc_temperature	0.03

heatmap_valley_loc_weight is 0.0, so the valley-localisation term is switched off while its temperature is still specified.

Table B.8: Heatmap focus phase and low- K emphasis (9 keys).

Key	Value
heatmap_focus_start_frac	0.0
heatmap_focus_start_epoch	25
heatmap_focus_heatmap_weight	7.0
heatmap_focus_impedance_weight	4.0
heatmap_focus_cross_freq_weight	1.1
heatmap_focus_dynrange_weight	3.0
heatmap_focus_modality_dropout	0.03
hm_low_k_threshold	3
hm_low_k_multiplier	2.0

Table B.9: Heatmap value range and clipping (4 keys).

Key	Value
background_value	-3.1792
heatmap_z_clip_min	-1.6791870594024658
heatmap_z_clip_max	6.275787830352783
heatmap_clip_recon	false

heatmap_z_clip_min and physics_fg_clip_min (Table B.17) hold the same value, -1.6791870594024658 . **[FILL IN: state where this number comes from — presumably a percentile of the normalised heatmap distribution recorded in normalization_stats.json — and confirm that the two keys are meant to be tied. Do not keep the word "presumably": either cite the statistics file or mark the link as unverified.]**

B.2.7 Impedance loss weights and peak curriculum

The impedance branch predicts the spectrum $Z(f)$ over the 231 frequency bins spanning 1–600 MHz. Table B.10 lists its loss weights; Table B.11 lists the curriculum keys that stage the peak-alignment terms during training.

Table B.10: Impedance loss weights (11 keys).

Key	Value
impedance_weight	4.0
impedance_deriv_weight	1.4
impedance_topk_k	20
impedance_topk_weight	4.5
impedance_under_penalty	2.8
impedance_concavity_weight	2.5
impedance_freq_weight_alpha	2.0
impedance_dual_topk_weight	0.75
impedance_peak_index_weight	1.5
impedance_peak_mag_weight	2.0
impedance_num_peaks	8

Table B.11: Impedance peak curriculum (6 keys).

Key	Value
impedance_peak_start_frac	0.1
impedance_peak_ramp_frac	0.05
impedance_peak_focus_frac	0.1
impedance_peak_start_epoch	25
impedance_peak_ramp_epochs	22
impedance_peak_focus_epoch	25

Each curriculum stage is specified twice, once as a fraction of the run and once as an absolute epoch. **[FILL IN: state which of the two forms `train_core.py` actually uses — the `*_frac` keys or the `*_epoch` keys — and whether `recalculate_curriculum_on_resume` (false, Table B.4) selects between them on resume.]**

B.2.8 Occupancy loss

The occupancy head predicts the placement vector $\mathbf{b} \in \{0, 1\}^{52}$ under a budget K .

B.2.9 KL and latent regularisation

`use_beta_annealing` is false, so the eight β schedule keys describe a schedule that is not active; `beta_final` is 0.03. **[FILL IN: confirm from `train_core.py` which single β value is applied when `use_beta_annealing` is false.]**

Table B.12: Occupancy loss keys (11 keys).

Key	Value
occupancy_weight	7.0
occupancy_focal_gamma	1.5
focal_gamma_warmup_frac	0.0
focal_gamma_warmup_epochs	0
occ_k_consistency_weight	2.0
use_k_weighting	true
occ_mid_k_center	25.0
occ_mid_k_sigma	8.0
occ_mid_k_boost	0.75
occupancy_binary_decode	true
occupancy_binary_ste_train	false

Table B.13: KL, β -annealing and latent regularisation keys (17 keys).

Key	Value
free_bits	0.12
mu_hinge_threshold	4.0
mu_hinge_weight	0.1
mu_bias_weight	0.05
per_expert_kl_weight	0.02
sigma_reg_weight	1.5
sigma_reg_target	0.45
use_beta_annealing	false
beta_start_epoch	0
beta_end_frac	0.3
beta_initial	0.0
beta_final	0.03
beta_phase2_final	0.03
beta_phase2_end_frac	0.3
beta_end_epoch	25
beta_phase2_end_epoch	25
kan_spline_l1_weight	0.0001

B.2.10 Modality dropout and cross-modal terms

Modality dropout is what makes the occupancy-only encode path usable at inference, which is the only path available when a placement is proposed rather than reconstructed.

Table B.14: Modality dropout and cross-modal keys (8 keys).

Key	Value
modality_dropout	0.06
modality_dropout_start	0.06
modality_dropout_anneal_frac	0.04
modality_dropout_anneal_epochs	18
modality_dropout_protect_heatmap_frac	0.325
modality_dropout_protect_heatmap_epoch	25
cross_modal_weight	0.75
cross_modal_update_freq	4

B.2.11 Cross-frequency and layout path

Table B.15: Cross-frequency and layout-path keys (11 keys).

Key	Value
cross_freq_weight	1.2
cross_freq_start_frac	0.025
cross_freq_start_epoch	20
cross_freq_layout_z_only	false
cross_freq_layout_mix_prob	0.45
cross_freq_use_encode_z	true
layout_train_prob	0.65
occ_only_encode_prob	0.4
eval_use_occ_only_layout	true
freq_jitter_prob	0.6
freq_jitter_log10_sigma	0.1

The values of `layout_train_prob` and `occ_only_encode_prob` in this table are the current contents of the file, not a record of the capacity-phase run. `docs/cursor-handoff.md` states that the capacity phase was run with `layout_train_prob = 0` and `occ_only_encode_prob = 0`, whereas `config.yaml` now holds 0.65 and 0.4. The file therefore no longer describes the run that produced the capacity checkpoint, and a reader must not treat [Table B.15](#) as the capacity-phase record for these two keys. **[FILL IN: recover the two values that were in force when the capacity checkpoint was trained — the training log or the config snapshot stored alongside checkpoints/last_model.pt**

would settle it — and state whether any other key in section B.2 was also edited after that run.]

B.2.12 Distillation and physics terms

Table B.16: Latent- and output-distillation keys (4 keys).

Key	Value
latent_distill_weight	2.5
latent_distill_private_mult	3.0
latent_distill_logvar_weight	0.3
output_distill_weight	2.5

Table B.17: Physics-loss keys (10 keys).

Key	Value
physics_ri_weight	1.0
physics_critic_sup_weight	2.0
physics_ar_weight	0.5
physics_fg_clip_min	-1.6791870594024658
physics_critic_warmup_frac	0.025
physics_slope_anneal_frac	0.05
physics_critic_warmup_epochs	11
physics_slope_anneal_epochs	22
penalty_warmup_frac	0.025
penalty_warmup_epochs	11

B.2.13 Off-anchor evaluation and fine-tune early stopping

The `eval_off_anchor` keys define the frequencies at which the surrogate is scored outside the normal validation pass. The name is misleading and the set must not be read as a held-out frequency set. Of the three frequencies in `eval_off_anchor_mhz`, 155.0 and 265.0 MHz are not training anchors, but 250.0 MHz is one of the 24 anchors listed in `datasets/data_multifreq_train_norm_unbounded/dataset_meta.json` (`pi_frequencies_mhz`) and reproduced in [Appendix A](#). It also carries the largest weight of the three, 3.0 against 2.5 for each of the other two. One third of this evaluation set is therefore trained on, and it is the most heavily weighted third.

This matters beyond bookkeeping, because the metric is also a checkpoint-selection criterion: `experiments/exp059_capacity_freq/checkpoints/best_off_anchor_model.pt` exists, so a checkpoint is retained on the basis of a score computed partly at a trained frequency. Any statement in Chapter 4 or Chapter 5 about generalisation to unseen frequencies must be made against 155.0 and 265.0 MHz alone, not against the pooled off-anchor number. **[FILL IN: state why 250.0 MHz, an existing training anchor, is included in the off-anchor evaluation set, and whether the off-anchor metrics reported elsewhere in the thesis are per frequency or pooled across the three; the off-anchor evaluation code and the metrics CSVs under `experiments/exp059_capacity_freq/metrics/` would settle it.]**

Table B.18: Off-anchor evaluation keys (4 keys). 250.0 is a training anchor; see the text above.

Key	Value
<code>eval_off_anchor_mhz</code>	[155.0, 250.0, 265.0]
<code>eval_off_anchor_mhz_weights</code>	{ "155": 2.5, "250": 3.0, "265": 2.5 }
<code>eval_off_anchor_max_batches</code>	20
<code>eval_off_anchor_interval</code>	5

Table B.19: Active-learning fine-tune early-stopping keys present in the base `config.yaml` (4 keys).

Key	Value
<code>al_finetune_early_stop</code>	true
<code>al_finetune_early_stop_patience</code>	3
<code>al_finetune_early_stop_min_delta</code>	0.01
<code>al_finetune_early_stop_kind</code>	"layout_cross"

Table B.20: Numerical-stability keys (10 keys).

Key	Value
training_force_fp32	true
training_sanitizе_grads	false
training_skip_nonfinite_batches	true
training_grad_clip_norm	1.0
training_logvar_clamp_min	-10.0
training_logvar_clamp_max	10.0
training_recon_z_soft_floor	-8.0
training_recon_z_soft_cap	null
training_recon_z_cap_mult	1.05
training_loss_finite_cap	1000000.0

B.2.14 Numerical stability

B.2.15 Hardware and runtime

Table B.21: Hardware and runtime keys (12 keys).

Key	Value
amp	"off"
tf32	false
compile	false
compile_mode	"reduce-overhead"
cache_in_ram	false
use_ddp	true
ddp_base_batch_size	160
ddp_linear_lr_scale	true
persistent_workers	false
prefetch_factor	2
empty_cache_interval	0
device	"cuda:0"

Mixed precision is off and `training_force_fp32` is true, so training ran in single precision. `use_ddp` is true with `ddp_base_batch_size` = 160, `batch_size` = 224 and `ddp_linear_lr_scale` = true. **[FILL IN: state the effective per-GPU and global batch size and the learning rate that linear scaling actually produced. No artifact recording the scaled learning rate was found; a training log line from the DDP run would settle it.]**

B.3 Architecture switches taken from code defaults

Not every architecture switch is written into `config.yaml`. Keys absent from the file fall back to the defaults in `vae_model_kwargs()` (`experiments/exp059_capacity_freq/codes/exp059` lines 34–55). [Table B.22](#) lists those defaults and marks which of them `config.yaml` overrides. **The twelve rows marked “code default” were never configured for exp059**; they are recorded here because they determine the model that was built, not because a decision about them is documented anywhere.

Table B.22: Defaults in `vae_model_kwargs()` and whether `config.yaml` overrides them. “Code default” means the key does not appear in any exp059 configuration file.

Key	Code default	Status in exp059
<code>latent_dim</code>	42	overridden: 128
<code>cond_dim</code>	8	overridden: 32
<code>heatmap_private_dim</code>	8	overridden: 40
<code>modality_dropout</code>	0.0	overridden: 0.06
<code>freq_fourier_features</code>	8	overridden: 16
<code>use_heatmap_film</code>	True	overridden: true
<code>use_multiscale_film</code>	True	code default
<code>use_freq_poe_expert</code>	True	code default
<code>use_occ_spatial_decoder</code>	True	code default
<code>use_occ_spatial_tower</code>	True	code default
<code>occ_spatial_ch</code>	8	code default
<code>use_layout_private_head</code>	True	code default
<code>layout_private_hidden</code>	384	code default
<code>use_layout_private_freq_film</code>	True	code default
<code>graph_hidden_dim</code>	128	code default
<code>graph_num_layers</code>	3	code default
<code>graph_dropout</code>	0.1	code default
<code>imp_peak_dim</code>	8	code default

`use_heatmap_film` is set in `config.yaml` to the same value as the code default, so it is listed as overridden but does not change the model.

B.4 Fine-tune override configurations

Two override files sit beside the base configuration in `experiments/exp059_capacity_freq/`: `config_al_finetune.yaml` for the active-learning fine-tune and `config_occ_only_finetune.yaml` for the occupancy-only warm-up. Both are partial: they set a subset of keys and inherit the rest from `config.yaml`.

B.4.1 Active-learning fine-tune: file value versus value actually used

The fine-tune that runs is not driven directly by `config_al_finetune.yaml`. At launch, `write_runtime_finetune_config()` (`active_learning_pi/al/finetune_run.py`, lines 61–123) copies that file, overwrites `resume_checkpoint`, recomputes `num_epochs` as the checkpoint epoch plus `finetune.extra_epochs`, forces `eval_use_occ_only_layout` to `True`, and applies the overrides in the `finetune` block of the active-learning configuration. The result is written to `config_al_finetune.runtime.yaml`. Table B.23 therefore prints both columns side by side; where they differ, the runtime column is what ran.

The override generator also copies `beta_final`, `free_bits` and `use_beta_annealing` from the occupancy-only warm-up configuration into the runtime fine-tune configuration (`finetune_run.py`, lines 90–92). The keys that the active-learning configuration’s `finetune` block is allowed to override (`finetune_run.py`, lines 94–117) are: `al_overlay_sample_weight`, `layout_train_prob`, `occ_only_encode_prob`, `eval_use_occ_only_l`, `latent_distill_weight`, `output_distill_weight`, `latent_distill_logvar_weight`, `heatmap_phys_p99_weight`, `eval_off_anchor_mhz`, `eval_off_anchor_mhz_weights`, `eval_off_anchor_interval`, `eval_off_anchor_max_batches`, `al_finetune_early_stop`, `al_finetune_early_stop_patience`, `al_finetune_early_stop_min_delta`, `al_finetune_early_checkpoint_interval`, `beta_final`, `free_bits` and `use_beta_annealing`.

B.4.2 Occupancy-only warm-up configuration

The warm-up sets both `layout_train_prob` and `occ_only_encode_prob` to 1.0, i.e. it trains the occupancy-only encode path exclusively.

B.4.3 Active-learning configuration blocks that reach the fine-tune

The acquisition surrogate has its own settings in the `gp_error` block of the same file, transcribed in Table B.27. The primary active-learning configuration sets `acquisition_mode` to `"gp_error"` (line 139); within the block the GP type is `"svgp"`, the fitted target is `"p99_ae"` on the `"pred"` path, and `score_mode` is `"mu"`. **[FILL IN: state what `kappa`, `novelty_weight`, `novelty_k`, `joint_alpha_hm` and `joint_alpha_imp` do to the acquisition score, and in particular which of them are inactive when `score_mode` is `"mu"` and `novelty_weight` is 0.0; the scoring function in `active_learning_pi/al/gp_error_surrogate.py` would settle it.]**

Three settings that a sparse variational GP normally needs are genuinely absent

Table B.23: Active-learning fine-tune configuration: hand-written `config_al_finetune.yaml` (29 keys) against the generated `config_al_finetune.runtime.yaml` (29 keys). Rows in which the two disagree are discussed in [section B.5](#).

Key	<code>config_al_finetune.yaml</code>	<code>config_al_finetune.runtime.yaml</code>
<code>resume_checkpoint</code>	<code>experiments/</code>	<code>experiments/</code>
	<code>exp059_capacity_freq/</code>	<code>exp059_capacity_freq/</code>
	<code>checkpoints/</code>	<code>checkpoints/</code>
	<code>last_model.pt</code>	<code>last_model.pt</code>
<code>num_epochs</code>	1050	510
<code>learning_rate</code>	1.5e-05	1.5e-05
<code>reset_lr_on_resume</code>	true	true
<code>train_samples_per_epoch</code>	64000	64000
<code>al_overlay_data_dir</code>	<code>datasets/</code>	<code>datasets/</code>
	<code>data_multifreq_</code>	<code>data_multifreq_</code>
	<code>al_overlay_exp059</code>	<code>al_overlay_exp059</code>
<code>al_overlay_sample_weight</code>	128.0	128.0
<code>layout_train_prob</code>	0.9	0.65
<code>occ_only_encode_prob</code>	0.7	0.4
<code>eval_use_occ_only_layout</code>	true	true
<code>latent_distill_weight</code>	2.5	2.5
<code>output_distill_weight</code>	2.5	2.5
<code>use_beta_annealing</code>	false	false
<code>beta_final</code>	0.03	0.03
<code>free_bits</code>	0.12	0.12
<code>heatmap_weight</code>	5.0	5.0
<code>heatmap_focus_heatmap_weight</code>	7.0	7.0
<code>heatmap_focus_start_frac</code>	0.0	0.0
<code>heatmap_focus_start_epoch</code>	0	0
<code>cross_freq_weight</code>	1.2	1.2
<code>eval_off_anchor_mhz</code>	[155, 250, 265]	[155, 250, 265]
<code>eval_off_anchor_interval</code>	5	5
<code>eval_off_anchor_max_batches</code>	20	20
<code>al_finetune_early_stop</code>	true	true
<code>al_finetune_early_stop_patience</code>	3	3
<code>al_finetune_early_stop_min_delta</code>	0.01	0.01
<code>al_finetune_early_stop_kind</code>	"layout_cross"	"layout_cross"
<code>checkpoint_interval</code>	5	5
<code>epoch_print_interval</code>	1	1

Table B.24: `config_occ_only_finetune.yaml`, the occupancy-only warm-up override (28 keys).

Key	Value
<code>resume_checkpoint</code>	<code>experiments/ exp059_capacity_freq/ checkpoints/last_model.pt</code>
<code>num_epochs</code>	1100
<code>learning_rate</code>	1.2e-05
<code>reset_lr_on_resume</code>	true
<code>train_samples_per_epoch</code>	56000
<code>layout_train_prob</code>	1.0
<code>occ_only_encode_prob</code>	1.0
<code>eval_use_occ_only_layout</code>	true
<code>use_beta_annealing</code>	false
<code>beta_final</code>	0.03
<code>free_bits</code>	0.12
<code>latent_distill_weight</code>	2.5
<code>output_distill_weight</code>	2.5
<code>cross_freq_weight</code>	1.2
<code>cross_freq_layout_z_only</code>	true
<code>heatmap_weight</code>	5.0
<code>heatmap_focus_heatmap_weight</code>	6.0
<code>heatmap_focus_start_frac</code>	0.0
<code>heatmap_focus_start_epoch</code>	0
<code>eval_off_anchor_mhz</code>	[155, 250, 265]
<code>eval_off_anchor_interval</code>	5
<code>eval_off_anchor_max_batches</code>	20
<code>al_finetune_early_stop</code>	true
<code>al_finetune_early_stop_patience</code>	4
<code>al_finetune_early_stop_min_delta</code>	0.01
<code>al_finetune_early_stop_kind</code>	"layout_cross"
<code>checkpoint_interval</code>	5
<code>epoch_print_interval</code>	1

Table B.25: `finetune` block of the primary active-learning configuration `active_learning_pi/config/exp059_gp_error.json` (lines 111–122).

Key	Value
<code>enabled</code>	<code>true</code>
<code>run_after_cycle</code>	<code>false</code>
<code>experiment_dir</code>	<code>experiments/</code>
<code>config_path</code>	<code>exp059_capacity_freq</code> <code>experiments/</code> <code>exp059_capacity_freq/</code> <code>config_al_finetune.yaml</code>
<code>checkpoint_path</code>	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>checkpoints/last_model.pt</code>
<code>extra_epochs</code>	<code>45</code>
<code>overlay_data_dir</code>	<code>datasets/data_multifreq_</code> <code>al_overlay_exp059</code>
<code>al_overlay_sample_weight</code>	<code>128.0</code>
<code>occ_only_encode_prob</code>	<code>0.4</code>
<code>layout_train_prob</code>	<code>0.65</code>

Table B.26: `pre_al_occ_only_warmup` block of `active_learning_pi/config/exp059_gp_error.json` (lines 106–110).

Key	Value
<code>config_path</code>	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>config_occ_only_finetune.yaml</code>
<code>checkpoint_path</code>	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>checkpoints/last_model.pt</code>
<code>extra_epochs</code>	<code>100</code>

Table B.27: `gp_error` block of `active_learning_pi/config/exp059_gp_error.json` (lines 140–155), 14 keys.

Key	Value
<code>path</code>	"pred"
<code>target</code>	"p99_ae"
<code>gp</code>	"svgp"
<code>kappa</code>	0.5
<code>n_fit</code>	4000
<code>n_inducing</code>	256
<code>svgp_epochs</code>	80
<code>seed</code>	0
<code>verbose</code>	true
<code>novelty_weight</code>	0.0
<code>novelty_k</code>	10
<code>joint_alpha_hm</code>	1.0
<code>joint_alpha_imp</code>	1.0
<code>score_mode</code>	"mu"

from this block: the minibatch size used to fit the variational parameters, the learning rate, and the kernel. **[FILL IN: state the residual GP’s batch size, learning rate and kernel — they are not set in `active_learning_pi/config/exp059_gp_error.json`; if they are library defaults or hard-coded in `active_learning_pi/al/gp_error_surrogate.py` name the source, and for a library default name the library and version that supplies it.]**

B.5 Conflicts between configuration files

Four disagreements between configuration artifacts were found while assembling this appendix. They are recorded here rather than resolved, because in each case two files that are both in use state different things and no authoritative artifact settles which was intended.

C1: fine-tune layout probabilities

`experiments/exp059_capacity_freq/config_al_finetune.yaml` (lines 11–12) sets `layout_train_prob = 0.9` and `occ_only_encode_prob = 0.7`. The same pair appears in `docs/cursor-handoff.md` (lines 42 and 77), `docs/gp-error-surrogate.md` (line 86) and `CLAUDE.md` (line 17).

However, `active_learning_pi/config/exp059_gp_error.json` (lines 120–121) overrides the pair in its `finetune` block to `layout_train_prob = 0.65` and `occ_only_encode_prob = 0.4`. `write_runtime_finetune_config()` (`finetune_run.py`, lines 94–117) lets the active-learning JSON win over the hand-written fine-tune file for both keys, and the generated `config_al_finetune.runtime.yaml` (lines 9–10) accordingly carries 0.65 and 0.4. A fine-tune launched from the current configuration therefore runs at 0.65/0.4, not at the documented 0.9/0.7. Both values are stated in [Table B.23](#) and [Table B.25](#); neither is presented here as the correct one.

What the four completed active-learning cycles used is a separate question, and it cannot be answered from the repository. The runtime file is rewritten at every fine-tune launch, so it records only the most recent one, and no per-cycle configuration record survives: no iteration directory under `active_learning_pi/runs/al_exp059_gp_error_001/` records `layout_train_prob` at all, and neither `state.json` nor `decision_ledger.json` carries these keys. This is a reproducibility gap, not a detail: the two probabilities govern how much of the fine-tune trains the occupancy-only encode path, which is the path the active-learning loop scores with. **[FILL IN: pin down, from surviving training logs, which `layout_train_prob` and `occ_only_encode_prob` were in force in each of the four completed cycles of `al_exp059_gp_error_001`. The repository preserves no per-cycle configuration snapshot, so a fine-tune console log or a checkpoint-side metadata record is the only thing that can settle it.]** **[FILL IN: decide which pair was intended, correct whichever file is wrong, and — once the per-cycle values are recovered — state in Chapter 5 whether any reported active-learning result is affected by the value actually used.]** **[FILL IN: if the per-cycle values cannot be recovered, add a per-iteration configuration snapshot to the active-learning pipeline so that future cycles are reproducible, and say so in the limitations section.]**

C2: fine-tune epoch count

`config_al_finetune.yaml` (line 5) states `num_epochs = 1050`, while the generated `config_al_finetune.runtime.yaml` (line 3) states 510. `finetune_run.py` (lines 71–79) recomputes `num_epochs` as the checkpoint epoch plus `extra_epochs`, which is 45 in `exp059_gp_error.json` (line 117). The value in the hand-written file is therefore never the number of epochs that runs. The base `config.yaml` independently holds `num_epochs = 510`. **[FILL IN: state the checkpoint epoch that the runtime recomputation started from, from the checkpoint metadata or the fine-tune log, so the 510 in the runtime file can be traced.]**

C3: stale experiment name in the override headers

The header comments on line 1 of both `experiments/exp059_capacity_freq/config_al_finetune` and `experiments/exp059_capacity_freq/config_occ_only_finetune.yaml` read “for exp058”, while both files live under `exp059_capacity_freq` and set `resume_checkpoint` (line 4 in each) to `experiments/exp059_capacity_freq/checkpoints/last_model.pt`. The `config_al_finetune.yaml` header additionally reads “only after fresh train has produced last_model.pt / Do NOT point at exp057 checkpoints.” The comment lines are stripped by `load_yaml_config()` before parsing, so they do not affect execution, but they mislabel the file.

C4: normalisation statistics path

`experiments/exp059_capacity_freq/config.yaml` (line 134) points the training set at `datasets/data_multifreq_train_norm_unbounded`, whereas `pipelines/latent/optimize.py` (line 105) sets `NORMALIZATION_STATS_PATH` to `datasets/data_multifreq_norm/normalization_stats`. The two are different directories. **[FILL IN: state whether the two normalisation-statistics files contain the same per-MHz statistics; if they do not, any comparison between a Stage-2 result and an exp059 prediction is in different units and must be flagged in Chapter 4.]**

C5: the base configuration is not the capacity-phase record

Beyond the four file-to-file conflicts, `config.yaml` has been edited since the capacity-phase run. `docs/cursor-handoff.md` states that the capacity phase ran with `layout_train_prob = 0` and `occ_only_encode_prob = 0`, and the file now holds 0.65 and 0.4 (Table B.15). The current file therefore does not describe the run that produced the capacity checkpoint. This is stated plainly because the tables in section B.2 would otherwise be read as a run record, which they are not. **[FILL IN: check the remaining 181 keys**

against the training log or a config snapshot taken at the capacity run, and mark in section B.2 every key whose current value differs from the value used then.]

B.6 Stage-2 latent-optimisation configuration

The configuration in this section is not exp059's. `pipelines/latent/optimize.py` still selects `EXPERIMENT = "exp038_true_multi"` for the occupancy decoder and the impedance surrogate (lines 45–54), so no Stage-2 hyperparameter set wired to `exp059_capacity_freq` exists. The tables below record the `CONFIG` block as it stands: an exp038-targeted configuration. They must not be presented as the settings under which an exp059 model was inverted.

Like every runnable script in this repository, `optimize.py` takes no command-line arguments; the values are module-level constants edited before the run. Several are overridable through environment variables, which is noted per row.

Table B.28: `optimize.py` model and checkpoint selection (lines 45–54).

Constant	Value
<code>EXPERIMENT</code>	<code>"exp038_true_multi"</code>
<code>CHECKPOINT_PATH</code>	<code>f"experiments/{EXPERIMENT}/checkpoints/last_model.pt"</code>
<code>SURROGATE_CHECKPOINT_PATH</code>	<code>f"experiments/{EXPERIMENT}/checkpoints/surrogate_best.pt"</code>
<code>USE_SURROGATE</code>	<code>_env_flag("LATENT_OPT_USE_SURROGATE", "1")</code>

Table B.29: `optimize.py` search setup (lines 56–69).

Constant	Value
<code>K_LIST</code>	<code>list(range(1, 26))</code>
<code>NUM_CANDIDATE_SEEDS</code>	<code>32</code>
<code>SEED_MODE</code>	<code>os.getenv("LATENT_OPT_SEED_MODE", "random").strip().lower()</code>
<code>MASTER_SEED</code>	<code>_parse_master_seed()</code> (None when <code>LATENT_OPT_MASTER_SEED</code> is unset)
<code>OPTIMIZE_BATCH_PER_K</code>	<code>True</code>

The optimiser over the latent vector is `torch.optim.Adam([z], lr=LR)` (`optimize.py`, lines 504 and 514), with everything else in the model frozen. The discrete read-out from continuous occupancy probabilities to a hard top- K placement is `_ste_topk(occ_prob, K)` (`optimize.py`, line 259), the straight-through estimator that keeps the gradient path to $\mathbf{z}$ intact.

Table B.30: `optimize.py` latent-optimiser settings and objective (lines 71–77).

Constant	Value
NUM_STEPS	1200
LR	5e-2
GRAD_CLIP	5.0
INIT_SHARED_TEMP	0.8
OBJECTIVE_MODE	"gap_max" (alternative: "feasible_map")
LOSS_SPACE	"log" (alternative: "ohm")

Table B.31: `optimize.py` latent-prior and spectrum-shape regularisation weights (lines 79–86).

Constant	Value
Z_L2_WEIGHT	0.05
Z_PRIOR_MODE	"agg_posterior"
SHAPE_REG_WEIGHT	2.0
SHAPE_MIN_STD_DLOG	0.12
SHAPE_TRACK_WEIGHT	2.0
POSTERIOR_BOUNDARY_WEIGHT	10.0
POSTERIOR_SIGMA_LIMIT	2.5
PHYSICS_AR_WEIGHT	0.5

Table B.32: `optimize.py` peak-aware spectrum terms (lines 88–94). Each is read from the named environment variable with the default shown.

Constant	Environment variable	Default
PEAK_LOSS_WEIGHT	LATENT_OPT_PEAK_LOSS_WEIGHT	"1.0"
PEAK_INDEX_WEIGHT	LATENT_OPT_PEAK_INDEX_WEIGHT	"3.0"
PEAK_MAG_WEIGHT	LATENT_OPT_PEAK_MAG_WEIGHT	"2.0"
DUAL_TOPK_WEIGHT	LATENT_OPT_DUAL_TOPK_WEIGHT	"2.5"
IMPEDANCE_NUM_PEAKS	LATENT_OPT_IMPEDANCE_NUM_PEAKS	"8"
IMPEDANCE_TOPK_K	LATENT_OPT_IMPEDANCE_TOPK_K	"20"

Appendix B. Hyperparameter Configuration for Training and Optimisation

The source comment at line 88 records that these terms are “aligned with exp038 impedance_spectrum_loss”, which is consistent with the exp038 target of the whole block.

Table B.33: `optimize.py` diversity, occupancy-confidence and feasibility terms (lines 96–103).

Constant	Value
DIVERSITY_WEIGHT	0.35
OCC_CONFIDENCE_WEIGHT	0.5
BOUNDARY_MARGIN	0.1
GAP_REWARD_WEIGHT	1.0
EXCEED_WEIGHT	25.0
EXCEED_POWER	2.0
SELECT_METRIC	"max_ohm"

Table B.34: `optimize.py` paths, device and output naming (lines 105–120).

Constant	Value
NORMALIZATION_STATS_PATH	datasets/data_multifreq_norm/ normalization_stats.json
TARGET_IMPEDANCE_PATH	configs/target_impedance.npy
OUTPUT_ROOT	data/latent_runs
DEVICE	"cuda" if torch.cuda.is_available() else "cpu"
DTYPE	torch.float32
HIST_KEYS	("total", "exceed", "gap_mean", "best_score")
SOLUTION_PREFIX	"best_"
SOLUTION_LATENT_NAME	f"{SOLUTION_PREFIX}latent.npy"
SOLUTION_METRICS_NAME	f"{SOLUTION_PREFIX}metrics.json"

NORMALIZATION_STATS_PATH is the second half of conflict C4 in [section B.5](#): it does not point at the exp059 training set named in `config.yaml`.

B.7 Values this appendix cannot supply

For completeness, the following are known gaps rather than omissions.

- No per-key rationale exists for any of the 183 keys in `config.yaml`; the file contains no comment lines.
- No sweep record or hyperparameter table was located that distinguishes exp059 values that were tuned from values inherited from exp057 or exp058.
- Twelve architecture switches ([Table B.22](#)) have no documented value in any exp059 configuration file and exist only as code defaults.
- The effective per-GPU and global batch size under DDP, and the learning rate produced by `ddp_linear_lr_scale`, are not recorded in any artifact.
- No Stage-2 hyperparameter set targeting exp059 exists; [section B.6](#) documents an exp038-targeted configuration.
- The residual-GP acquisition surrogate’s minibatch size, learning rate and kernel are not set in the `gp_error` block ([Table B.27](#)); the remaining fourteen keys of that block are transcribed there.
- The `layout_train_prob` and `occ_only_encode_prob` values in force during each completed active-learning cycle are not recoverable from the repository: the run-time fine-tune configuration is overwritten at every launch and no per-iteration snapshot is written (C1, [section B.5](#)).

Appendix C

Supplementary Experimental Results

Appendix D

Implementation Index

Chapters 2 and 3 describe the method and the system in the usual narrative form, without interrupting the text with source-file references. This appendix restores that traceability. Each row names a statement made in the running text and the artifact in the project repository it was taken from, grouped by the section in which the statement appears. Line numbers are omitted deliberately: they move as the code changes, whereas file and symbol names are stable. Paths are relative to the repository root.

[FILL IN: state the repository URL and the commit hash the thesis was written against, so a reader can resolve these paths to an exact revision]

D.1 Chapter 2 — Theoretical Background

Impedance Behavior and Decoupling Strategies

Statement in the text	Source artifact
Anti-resonance enters the surrogate as a shape prior: every impedance peak must be preceded by a resonance dip, scored as $\text{mean}(\text{peak_score} * \exp(-10 * \text{prior_dip}))$ on the raw log-impedance channel.	<code>experiments/ exp059_capacity_freq/ codes/physics_loss.py-233</code>
From bin index 190 upwards the spectrum is monotonically increasing by construction, so the anti-resonance term acts mainly at low frequency.	<code>experiments/ exp059_capacity_freq/ codes/physics_loss.py-233 (docstring)</code>

Appendix D. Implementation Index

Statement in the text	Source artifact
The target mask is a fixed array of 231 values, minimum 0.8 and maximum 49.99999999999999, stored with shape (231,1) and squeezed to (231,); Stage 2 aborts if the squeezed shape is not (231,).	<code>configs/target_impedance.npy;</code> <code>pipelines/latent/optimize.py</code> -548

Power Integrity Simulation and Multi-modal Design Data

Statement in the text	Source artifact
Six-step data flow: sample/merge combination CSVs, build PEB project in ECADSTAR script order, simulate via headless engineer.exe -batch on Windows, append to the raw multi-frequency dataset, robust log1p per-MHz normalization with optional $K \leq 30$ filter, then train/evaluate/active-learn.	<code>docs/data-pipeline.md</code> , section 1
Row i of a combination CSV maps to the ECAD-STAR object $PI-(i+1)$ in output order.	<code>docs/data-pipeline.md</code> , section 1
The conventional manual design loop the framework replaces: place, simulate spectrum, on mask violation run a spatial PI distribution, move capacitors toward hotspots, repeat.	<code>docs/framework-overview.md</code> , section 1
Board carries 52 decap slots on a 7x8 grid with four invalid cells at (0,3), (0,4), (6,3), (6,4); ordered labels C1-C52, index i of b maps to <code>LABELS_ORDERED[i]</code> .	<code>libs/data_creation/occupancy.py</code> -31
Slot count is derived from the label list: <code>NUM_SLOTS = len(LABELS_ORDERED)</code> .	<code>experiments/exp059_capacity_freq/codes/graph_occ.py</code>
Occupancy tensors are loaded from the layout store, flattened, clamped to $[0,1]$, and the budget recovered as $K = \sum_i b_i$.	<code>src_vae/others/dataloader.py</code> -201
The spectrum has exactly 231 points; the raw CSV reader rejects any file of different length (<code>EXPECTED_IMP_LENGTH = 231</code>).	<code>libs/data_creation/impedance.py</code> ,33-36
Spectrum batches of shape $(B,231)$ are unsqueezed to $(B,1,231)$ inside the model.	<code>experiments/exp059_capacity_freq/codes/train_core.py</code> -736

Appendix D. Implementation Index

Statement in the text	Source artifact
The impedance decoder returns shape (B,1,231).	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py
The frequency grid is a stored array of shape (231,) running 1000000.0 Hz, 1100000.0 Hz, ... 600000000.0 Hz; first log10 step 0.0414, last 0.0073; no specification document defines it.	configs/ Frequency_data_hz.npy
The PI-distribution heatmap is interpolated onto a 64x64 grid (HEATMAP_GRID_SIZE = 64); raw creation returns the two-channel stack [Zi * mask_board, mask_board].	libs/data_creation/ heatmap.py,74
Normalized heatmaps that reach training are single-channel float32 of shape (1,64,64).	datasets/ data_multifreq_train_norm_unbounded/ heatmap/ sample_1000000.npy
Heatmap encoder first layer is nn.Conv2d(1, 16, ...), consistent with single-channel input.	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py
Heatmap decoder ends in nn.Conv2d(16, 1, kernel_size=3, padding=1), i.e. one channel at 64x64.	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py
The training set declares 24 PI frequency anchors in MHz (10.0 ... 500.0).	datasets/ data_multifreq_train_norm_unbounded/ dataset_meta.json, key pi_frequencies_mhz
The same 24-anchor list appears in the normalization statistics under Heatmap.anchors_mhz and PI_freq.anchor_mhz.	datasets/ data_multifreq_train_norm_unbounded/ normalization_stats.json

Appendix D. Implementation Index

Statement in the text	Source artifact
Anchors are resolved in order from dataset_meta.json, the environment data directory, a legacy manifest, configs/multifreq_anchors.yaml, then a built-in default tuple (10,63,80,130,150,200,250,270,330,400,500); the YAML file lists 20 entries. Both fall-backs disagree with the live 24-anchor dataset and are marked legacy.	src_vae/others/ multifreq_anchors.py-21,49-81; configs/ multifreq_anchors.yaml
PI_freq normalization constants for the current dataset: log10_min = 6.0, log10_range = 2.778151250383644.	datasets/ data_multifreq_train_norm_unbounded/ normalization_stats.json, key PI_freq
When a batch carries no frequency the training code substitutes the normalized value for 200.0 MHz, the default_mhz of pi_freq_norm_for_model.	src_vae/others/ multifreq_anchors.py (pi_freq_norm_for_model)
Normalized training dataset: 582997 manifest rows over 24979 unique layouts, layout store, max_k_filter 30; per-anchor counts 24979 except 80 MHz with 8480.	datasets/ data_multifreq_train_norm_unbounded/ dataset_meta.json
The $K \leq 30$ filter drops layouts with $K > 30$ before statistics and output are computed.	docs/dataset.md, section 3
Heatmap normalization robust_log1p_per_mhz_unbounded with per-MHz median/IQR, background -3.1792, clip_min -1.6791870594024658, clip_max 6.275787830352783; the same three values appear in the exp059 config.	datasets/ data_multifreq_train_norm_unbounded/ normalization_stats.json; experiments/ exp059_capacity_freq/ config.yaml-142
The unbounded loader path applies no clip.	src_vae/others/ dataloader.py-217
Impedance normalization is a log z-score with mu_log = -1.039238452911377 and sigma_log = 1.7831873893737793, fitted over 24979 layouts.	datasets/ data_multifreq_train_norm_unbounded/ normalization_stats.json

Appendix D. Implementation Index

Statement in the text	Source artifact
Layout-level holdout: all frequencies of one design_id go to train or validation but never both; train_split 0.9, seed 42.	experiments/ exp059_capacity_freq/ config.yaml,19; experiments/ exp059_capacity_freq/ codes/ dataloader_multifreq.py-80

Formulation of the Decoupling Capacitor Placement Problem

Statement in the text	Source artifact
Engineering problem statement: keep $Z(f)$ below the target mask over 1-600 MHz at 231 bins with b in $\{0,1\}^{52}$; exhaustive search over 2^{52} placements is intractable.	docs/ framework-overview.md, section 1
Feasibility is tested against the mask shifted inward by $\text{BOUNDARY_MARGIN} = 0.1$.	pipelines/latent/ optimize.py,357-363
The default comparison space is $\text{LOSS_SPACE} = \text{'log'}$, i.e. z-scored log-impedance, so the margin 0.1 is in normalized units and not 0.1 ohm.	pipelines/latent/ optimize.py
Per-bin violation exponent $\text{EXCEED_POWER} = 2.0$; base objective weights $w_{\text{gap}} = 1.0$ and $w_{\text{exc}} = 25.0$; default $\text{OBJECTIVE_MODE} = \text{'gap_max'}$, alternative 'feasible_map' keeps only the exceed term.	pipelines/latent/ optimize.py,100-102,313-329
Selection metric defaults to $\text{SELECT_METRIC} = \text{'max_ohm'}$; the best candidate is only updated for feasible seeds, and a no_solution.json is written per K when no seed is feasible.	pipelines/latent/ optimize.py,366-369,466-471,644-646
Stage 2 loads the exp038_true_multi VAE and impedance surrogate ($\text{EXPERIMENT} = \text{'exp038_true_multi'}$); no Stage-2 entry point is wired to exp059_capacity_freq.	pipelines/latent/ optimize.py,45-47

Appendix D. Implementation Index

Statement in the text	Source artifact
NORMALIZATION_STATS_PATH points at a directory absent from the current checkout; the live dataset is datasets/data_multifreq_train_norm_unbounded.	pipelines/latent/ optionize.py

Variational Autoencoders and Multi-modal Learning

Statement in the text	Source artifact
exp059 KL settings: free_bits 0.12, beta annealing off with beta_final 0.03, per-expert KL weight 0.02, sigma-regularization weight 1.5 toward target 0.45.	experiments/ exp059_capacity_freq/ config.yaml-118
PoE fusion: precision-weighted product of Gaussian experts, a unit-Gaussian prior expert always appended, eps = 1e-8 added to each variance and to the combined precision.	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py-449
Expert log-variances clamped to [-6.0, 1.0] at each head; fused log-variance clamped to [-4.0, 2.0] before reparameterization.	experiments/ exp059_capacity_freq/ codes/ vae_poe_freq.py,160-162
Latent partition arithmetic: d_shared = d_latent - d_hm_priv - d_imp_peak and d_layout = d_shared + d_imp_peak.	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py-133,171-174
exp059 capacity: latent_dim 128, heatmap_private_dim 40, cond_dim 32.	experiments/ exp059_capacity_freq/ config.yaml-4
imp_peak_dim is unset in the exp059 config and takes the builder default 8, alongside graph_hidden_dim 128, graph_num_layers 3, graph_dropout 0.1.	experiments/ exp059_capacity_freq/ codes/exp059_common.py-54
d_shared = 80 is independently stated in the experiment notes (d_layout = 88 follows from the partition arithmetic).	experiments/ exp059_capacity_freq/ notes.md

Appendix D. Implementation Index

Statement in the text	Source artifact
Expert-to-latent wiring: heatmap expert writes the full latent, occupancy expert the shared dims, impedance expert shared+peak; unowned dims padded with the prior. Heatmap encoder input width $64 + 2 \cdot \text{cond_dim}$; occupancy/impedance heads $64 + \text{cond_dim}$.	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/</code> <code>vae_multi_input_simple.py</code> -258,394-413
A dedicated frequency expert maps the frequency embedding to the heatmap-private dims only; <code>use_freq_poe_expert</code> defaults to True and is not overridden in the exp059 config.	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/vae_poe_freq.py</code> -84; <code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/exp059_common.py</code>
Occupancy slot graph: 52 nodes, four-neighbour edges on the 7x8 grid, self-loops, symmetric normalization $D^{-1/2} A D^{-1/2}$; node features occupancy (1) + slot embedding (hidden/2) + normalized row/col (2), positions normalized by 6.0 and 7.0; mean+max pooling projected to width 64.	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/graph_occ.py</code> -47
Spectrum chain graph over 231 bins with optional self-loops and the same symmetric normalization; node features bin value (1) + bin embedding (hidden/2) + normalized position (1) from <code>torch.linspace(0.0, 1.0, 231)</code> , pooled to width 64.	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/graph_imp.py</code> ,13-24
Graph convolution layer computes <code>lin_self(x) + lin_neigh(Ax)</code> followed by LayerNorm, LeakyReLU(0.1) and optional dropout, in plain PyTorch with no <code>torch_geometric</code> dependency.	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/graph_occ.py</code> -75 (imported by <code>graph_imp.py</code>)
The two 64-wide graph features are concatenated and projected by <code>Linear(128, 64)</code> with LayerNorm and LeakyReLU into the joint feature feeding the occupancy and impedance expert heads.	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/</code> <code>vae_multi_input_simple.py</code> -247,422-428

Appendix D. Implementation Index

Statement in the text	Source artifact
Graph decoders broadcast latent+conditioning to every node and read out 52 occupancy logits / 231 spectrum values.	experiments/ exp059_capacity_freq/ codes/graph_occ.py-178; experiments/ exp059_capacity_freq/ codes/graph_imp.py-123
Budget embedding table has 53 entries covering K in $\{0, \dots, 52\}$; the heatmap conditioning vector is the concatenation of the K and frequency embeddings, width $2 \cdot \text{cond_dim}$.	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py-193,457-468
Frequency embedding is a Fourier-feature MLP over $[f, \sin(2 \cdot \pi \cdot f \cdot b), \cos(2 \cdot \pi \cdot f \cdot b)]$ for $b = 1..B$, input width $1 + 2B$, mapped Linear-SiLU-Linear-SiLU to cond_dim ; exp059 sets <code>freq_fourier_features</code> 16 (exp057/exp058 set 8). The exp059 model imports this module from exp043.	experiments/exp043/codes/ freq_conditioning.py-42
FiLM applies $x \rightarrow x \cdot (1 + \gamma) + \beta$ with γ/β from a Linear layer whose weights and bias are zero-initialized except that the bias entries of the γ half are set to 1.0.	experiments/exp043/codes/ freq_conditioning.py-59
Multi-scale FiLM at four decoder scales: 8x8 on 128 channels, 16x16 on 64, 32x32 on 32, 64x64 on 16, each conditioned on the $2 \cdot \text{cond_dim}$ vector; a further <code>FiLM2d(32, 2 * cond_dim)</code> sits in the heatmap encoder.	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py-375,505-544
<code>use_multiscale_film</code> is absent from the exp059 config and defaults to True in the model-kwarg builder; the exp057 builder has no such key.	experiments/ exp059_capacity_freq/ codes/exp059_common.py
The exp059 heatmap decoder has no U-Net skip connections: latent plus FiLM only, to match Stage 2 and generation. exp057's builder still exposes <code>use_heatmap_unet_skips</code> defaulting to True.	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py

Appendix D. Implementation Index

Statement in the text	Source artifact
Modality dropout: inside <code>encode()</code> , each of the three data experts is kept with probability <code>1 - modality_dropout</code> via <code>torch.bernoulli(torch.full((3,), 1.0 - modality_dropout))</code> ; if all three drop, one is forced back on; the frequency expert is appended afterwards and never dropped; <code>modality_dropout_protect_heatmap</code> forces the heatmap expert on.	<code>experiments/exp059_capacity_freq/codes/vae_poe_freq.py</code> -158
exp059 modality-dropout values: 0.06 with the same start value, heatmap protection from epoch 25 / fraction 0.325, focus-phase 0.03.	<code>experiments/exp059_capacity_freq/config.yaml</code> -33,46,118-119
<code>encode_occupancy_latent_full</code> builds a zero impedance batch, runs the joint layout encoder and forms the posterior from the occupancy expert plus the PoE prior alone; its docstring records that this matches the AL inference path and the occupancy-only fine-tune.	<code>experiments/exp059_capacity_freq/codes/vae_multi_input_simple.py</code> -905
With probability <code>layout_train_prob</code> the decode latent comes from the layout branch, and within it with probability <code>occ_only_encode_prob</code> from the occupancy-only encode; the full encode is always run for the KL term.	<code>experiments/exp059_capacity_freq/codes/train_core.py</code> -715
Config-vs-notes disagreement: live config sets <code>layout_train_prob</code> 0.65 and <code>occ_only_encode_prob</code> 0.4 and distillation weights 2.5/2.5; the AL fine-tune override sets 0.9 and 0.7; the notes record 0/0 and 0/0 plus different <code>eval_use_occ_only_layout</code> , <code>al_overlay_data_dir</code> , <code>learning_rate</code> , <code>num_epochs</code> and <code>resume_checkpoint</code> .	<code>experiments/exp059_capacity_freq/config.yaml</code> ,48,159,162; <code>experiments/exp059_capacity_freq/config_al_finetune.yaml</code> ; <code>experiments/exp059_capacity_freq/notes.md</code> ; CLAUDE.md

Appendix D. Implementation Index

Statement in the text	Source artifact
Layout-to-private head: two-hidden-layer MLP over [occ_feat(64), imp_feat(64), cond(2*cond_dim)] predicting the heatmap private dims, optional frequency FiLM, squashed as $4 \cdot \tanh(x/4)$; acts as distillation student for the heatmap encode teacher. use_layout_private_head and use_layout_private_freq_film default True and are unset in the exp059 config. Weights: latent 2.5, private multiplier 3.0, logvar 0.3, output 2.5.	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py-291; experiments/ exp059_capacity_freq/ codes/vae_poe_freq.py-264

Physics-Informed and Surrogate Modeling

Statement in the text	Source artifact
PhysicsCritic CNN maps occ_spatial (B,1,7,8) plus PI_freq to an effectiveness map (B,1,64,64) in [0,1], supervised by inverting and min-max normalizing the heatmap foreground, supplying a radius-influence penalty; detached during the VAE step.	experiments/ exp059_capacity_freq/ codes/ physics_loss.py-52,76,164-217
Physics weights: radius-influence 1.0, critic supervision 2.0, anti-resonance 0.5.	experiments/ exp059_capacity_freq/ config.yaml-126
Occupancy is binarized to a hard top-K vector before the heatmap decode via occupancy_for_heatmap_decode(occupancy, K, force_binary=True, ste=self.occupancy_binary_ste), vectorized through a descending-sort threshold so per-row variable K needs no Python loop.	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py-646; experiments/ exp059_capacity_freq/ codes/ occupancy_binary.py-65
exp059 sets occupancy_binary_decode = true and occupancy_binary_ste_train = false.	experiments/ exp059_capacity_freq/ config.yaml-165

Appendix D. Implementation Index

Statement in the text	Source artifact
Decoder input widths: heatmap decoder takes full latent + 2*cond_dim; occupancy decoder z_shared + K embedding; impedance decoder z_layout + K embedding + 64-wide occupancy graph context (passed occupancy, else sigmoid(occ_recon)).	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py-296,625-654
The frequency-weighted 2D-FFT heatmap loss (L1 between log1p magnitudes of rfft2, optional Gaussian MHz bump) is implemented but disabled: heatmap_spectral_weight is absent from the config and from the Config dataclass, so the lookup falls back to 0.0 and the function returns None. The exp059 notes claim weight 1.5, mid-boost 1.5, centre 200 MHz, width 80 MHz.	experiments/ exp059_capacity_freq/ codes/ train_vae_simple.py-222,199-201; experiments/ exp059_capacity_freq/ notes.md
eval_off_anchor_mhz = [155.0, 250.0, 265.0] with weights 2.5, 3.0, 2.5; 250.0 MHz is itself one of the 24 training anchors.	experiments/ exp059_capacity_freq/ config.yaml-60; datasets/ data_multifreq_train_norm_unbounded/ dataset_meta.json
decode_heatmap_blended log-blends the decodes at the two bracketing anchors with (lo, hi, w) from bracket_anchors_mhz(mhz); the training and evaluation path decodes directly at the requested PI_freq.	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py-694
AL candidate generation settings: K from 3 to 10, 1600 candidates per K, 80 worst per K, 640 ECAD simulations per cycle, heatmap grid 64, evaluation MHz grid [120,150,155,170,180,200,230,250,265,270,280,300].	active_learning_pi/ config/ exp059_gp_error.json

Optimization in Latent Representation Spaces

Appendix D. Implementation Index

Statement in the text	Source artifact
Straight-through top-K read-out implemented as hard + occ_prob - occ_prob.detach(); the docstring records that without it the top-K argsort severs the gradient and z never moves in response to the spectrum target.	<code>pipelines/latent/optimize.py</code> -268
Stage-2 search settings: K_LIST = 1..25, NUM_CANDIDATE_SEEDS 32, NUM_STEPS 1200, LR 5e-2, GRAD_CLIP 5.0.	<code>pipelines/latent/optimize.py</code> -57,71-73
Total Stage-2 objective weights (z-L2 0.05, shape 2.0, posterior 10.0, track 2.0, anti-resonance 0.5, peak w_peak, diversity 0.35, occupancy-confidence 0.5) and POSTERIOR_SIGMA_LIMIT = 2.5.	<code>pipelines/latent/optimize.py</code> -97,372-416
The impedance path defaults to a separate occupancy-to-spectrum surrogate: USE_SURROGATE = <code>_env_flag('LATENT_OPT_USE_SURROGATE', '1')</code> .	<code>pipelines/latent/optimize.py</code> ,271-282
That surrogate maps occupancy (52,) to a log-z spectrum (1,231) with hidden widths (256, 512, 512, 512, 256); its docstring states PI_freq is not used.	<code>experiments/exp038_true_multi/codes/surrogate_impedance.py</code> -8,36-53

Research Gap and Motivation for the Proposed Framework

Statement in the text	Source artifact
Retargeting to a new Z_target(f) by re-running the latent search with the decoder frozen is a property of the implemented Stage-2 script, which is wired to <code>exp038_true_multi</code> .	<code>pipelines/latent/optimize.py</code> ,45-47

D.2 Chapter 3 — Design and Implementation

Raw Power-Integrity Simulation Outputs and File Structure

Appendix D. Implementation Index

Statement in the text	Source artifact
The raw ECADSTAR export is converted into a layout store of layouts/, heatmap/ and PI_freq/trees plus a manifest.csv index.	<code>pipelines/data/processing_multifreq.py</code>
The processing stage follows the repository convention of a module-level CONFIGURATION block instead of command-line arguments.	<code>pipelines/data/processing_multifreq.py</code>
Per design the store keeps <code>occ.npy</code> of shape (52,) and <code>imp.npy</code> of shape (231,); the occupancy encoder is sized by <code>NUM_SLOTS = len(LABELS_ORDERED)</code> and the impedance encoder by <code>NUM_BINS = 231</code> .	<code>pipelines/data/processing_multifreq.py</code>
Only the active-learning batch invocation of the simulator is documented (<code>engineer.exe <design.erf> -batch <file.peb> -batch-auto-exit</code>); bulk data generation is not described anywhere.	<code>docs/data-pipeline.md</code> <code>sec.5</code>
The 64x64 heatmap resolution is inferred from the AL config <code>heatmap_grid_size = 64</code> and the decoder's final FiLM stage, not read from dataset metadata.	<code>active_learning_pi/config/exp059_gp_error.json</code>

Multi-frequency Dataset Construction and Normalization

Statement in the text	Source artifact
Dataset stage 'normalized', source script <code>pipelines/normalize/multifreq.py</code> , timestamp <code>2026-07-06T13:39:03.768907+00:00</code> ; 582997 manifest rows, 24979 unique layouts, 24979 layout dirs, 582997 heatmap and 582997 PI_freq files; 24 anchor frequencies; <code>samples_per_mhz</code> 24979 for all anchors except 80 MHz at 8480.	<code>datasets/data_multifreq_train_norm_unbounded/dataset_meta.json</code>

Appendix D. Implementation Index

Statement in the text	Source artifact
Heatmap normalization mode robust with unbounded = true; per-MHz medians and IQRs (10.0 MHz: 0.05887173116207123 / 0.03866304084658623; 500.0 MHz: 1.3578120470046997 / 0.6529430150985718); clip_min -1.6791870594024658, clip_max 6.275787830352783, z_max 11.23223876953125, background_value -3.1792. Impedance log_mean -1.039238452911377, log_std 1.7831873893737793, layout_count 24979, z_min -2.7170498371124268, z_max 2.474900722503662. Frequency log10_min 6.0, log10_range 2.778151250383644, min_hz 1000000.0, max_hz 600000000.0.	datasets/ data_multifreq_train_norm_unbounded/ normalization_stats.json
Foreground/background threshold BG_OHM = 0.05; robust clip percentiles 0.5 and 99.5 apply only in bounded mode; env switches NORM_ROBUST_PER_MHZ (default '1') and NORM_UNBOUNDED_Z (default '0').	pipelines/normalize/ multifreq.py
The unbounded build filters layouts at MAX_K = 30, with K recovered per design as (occ > 0.5).sum(), dropping them from both the statistics fit and the written output.	pipelines/normalize/ build_train_norm_unbounded.py
The experiment config mirrors the heatmap normalization constants (background_value -3.1792, heatmap_z_clip_min -1.6791870594024658, heatmap_z_clip_max 6.275787830352783) and sets heatmap_clip_recon = false.	experiments/ exp059_capacity_freq/ config.yaml
Documentation conflict on the anchor set: a 'Full 16-MHz set' is claimed here.	docs/dataset.md sec.4
Documentation conflict on the anchor set: anchors named as (10, 70, 120, 200, 270, 400) MHz here.	docs/ normalization-and-losses.md sec.1.4

Data Splitting, Sampling, and Batch Construction

Statement in the text	Source artifact
Layout-level holdout: <code>split_by_design = true</code> , <code>train_split = 0.9</code> ; <code>_split_indices_by_design</code> assigns all rows of a <code>design_id</code> to one side via <code>torch.randperm</code> ; default split seed 42. The docstring records that an earlier frequency-level split 'leaked layout identity into val and is unsuitable for GP / AL residual evaluation'.	<code>experiments/exp059_capacity_freq/codes/dataloader_base.py</code>
<code>stratify_by_k</code> only enables <code>precompute_k</code> on the dataset; it does not stratify the split.	<code>experiments/exp059_capacity_freq/codes/dataloader_base.py</code>
The weighted sampler is constructed only when <code>balance_k</code> or <code>balance_freq</code> is true; both are false in exp059, so no K-stratification or K/frequency balancing is active.	<code>experiments/exp059_capacity_freq/codes/dataloader_base.py-362</code>
Config sets <code>stratify_by_k = true</code> (conflicting with the dataloader behaviour), <code>balance_k = false</code> , <code>balance_freq = false</code> .	<code>experiments/exp059_capacity_freq/config.yaml</code>
Layout holdout dated 2026-07-20; seed 42 and <code>train_split 0.9</code> give approximately 22481 train and 2498 validation layouts (stated with a tilde); required a fresh retrain with <code>resume_checkpoint null</code> , prior artifacts archived under <code>archive_pre_layout_holdout_*</code> .	<code>experiments/exp059_capacity_freq/notes.md-62</code>
<code>train_samples_per_epoch = 64000</code> with <code>RandomSampler</code> with replacement; <code>batch_size 224</code> , <code>val_batch_size 320</code> , <code>num_workers 4</code> , <code>prefetch_factor 2</code> , <code>persistent_workers false</code> , <code>cache_in_ram false</code> .	<code>experiments/exp059_capacity_freq/config.yaml</code>
Cross-frequency pair lookup maps each training index to alternate-MHz rows of the same design; <code>make_multifreq_collate_with_pairs</code> attaches the paired row to the batch.	<code>experiments/exp059_capacity_freq/codes/dataloader_base.py</code>

Architectural Design

Appendix D. Implementation Index

Statement in the text	Source artifact
Decoder input widths: <code>dec_in_occ = shared_latent_dim + cond_dim; dec_in_imp = layout_latent_dim + cond_dim + 64</code> (<code>'z_layout + K + occ graph ctx'</code>); heatmap decoder takes full <code>z</code> plus <code>2*cond_dim</code> .	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/</code> <code>vae_multi_input_simple.py</code> -296
<code>layout_latent_dim = shared_latent_dim + imp_peak_dim</code> .	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/</code> <code>vae_multi_input_simple.py</code>
The shared latent block is 80 dimensions for the exp059 configuration.	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>notes.md</code>
FiLM2d and FreqConditioner are imported from an earlier fork, so the exp059 stack is not self-contained.	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/</code> <code>vae_multi_input_simple.py</code> -> <code>experiments/</code> <code>exp043/codes/</code> <code>freq_conditioning.py</code>
U-Net skip modules (OptionalSkipFuse, <code>_last_heatmap_skips</code> , <code>heatmap_skips</code>) were deleted; decoder is latent+FiLM only.	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>notes.md</code> -51
<code>use_multiscale_film</code> is absent from <code>config.yaml</code> and defaults to True in <code>vae_model_kwargs</code> , so the headline architectural change of exp059 is governed by a code default.	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/exp059_common.py</code>
<code>latent_dim 128</code> , <code>heatmap_private_dim 40</code> , <code>cond_dim 32</code> , <code>freq_fourier_features 16</code> , <code>use_heatmap_film true</code> ; <code>imp_peak_dim</code> unset (constructor default 8); graph hyperparameters unset (defaults <code>graph_hidden_dim 128</code> , <code>graph_num_layers 3</code> , <code>graph_dropout 0.1</code>).	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>config.yaml</code>

Appendix D. Implementation Index

Statement in the text	Source artifact
Modality dropout schedule: modality_dropout 0.06, modality_dropout_start 0.06, modality_dropout_anneal_epochs 18, modality_dropout_protect_heatmap_epoch 25, heatmap_focus_modality_dropout 0.03; occupancy_binary_decode true, occupancy_binary_ste_train false.	experiments/ exp059_capacity_freq/ config.yaml
Model class MultiInputVAEPoeFreq: PoE fusion subclass adding a PI_freq expert on the private dims and removing U-Net skips; expert logvar clamp [-6.0, 1.0], fused PoE logvar clamp [-4.0, 2.0]; encoders (52-slot 7x8 graph, 231-bin chain graph, CNN with HM_BOTTLENECK 8, HM_DROPOUT 0.05); layout_joint_fusion Linear(128,64)+LayerNorm+LeakyReLU(0.1); nn.Embedding(53, cond_dim); layout_private_hidden 384 with 4.0*tanh(mu/4.0) squash; topk_occ_binary vectorised read-out; encode_paths encode, encode_cross_modal, encode_layout_latent_full, encode_occupancy_latent.	experiments/ exp059_capacity_freq/ codes/ vae_multi_input_simple.py

Objective Functions and Physics-Informed Regularization

Statement in the text	Source artifact
The 2D-FFT log-magnitude L1 anti-blur loss (_heatmap_spectral_loss) is implemented, with an optional Gaussian MHz bump.	experiments/ exp059_capacity_freq/ codes/ train_vae_simple.py-221
The heatmap_spectral_weight lookup returns the default 0.0 because the key is absent from config.yaml, and the function returns None for any non-positive weight, so the term never enters the total.	experiments/ exp059_capacity_freq/ codes/train_vae_simple.py

Appendix D. Implementation Index

Statement in the text	Source artifact
The experiment notes present the frequency-weighted spectral loss as one of three headline exp059 changes, with heatmap_spectral_weight 1.5, mid_boost 1.5, centre 200 MHz, width 80 MHz – keys that do not exist in config.yaml.	experiments/ exp059_capacity_freq/ notes.md-23
PhysicsLoss is instantiated in the training core, and its parameters reach the optimizer and the checkpoints.	experiments/ exp059_capacity_freq/ codes/train_core.py
The trainer monkey-patches the live epoch function (<code>_tr._run_epoch = _run_epoch_encode</code>).	experiments/ exp059_capacity_freq/ codes/train_vae_simple.py
The replacement epoch function drops the physics and pw arguments from its signature.	experiments/ exp059_capacity_freq/ codes/run_epoch_encode.py
The replacement epoch function calls the loss with physics=None.	experiments/ exp059_capacity_freq/ codes/run_epoch_encode.py
<code>_vae_loss_exp055</code> likewise forwards physics=None.	experiments/ exp059_capacity_freq/ codes/train_vae_simple.py
<code>train_core.vae_loss</code> guards the radius-influence, critic-supervision and anti-resonance terms with 'if physics is not None', so none of them is added in exp059.	experiments/ exp059_capacity_freq/ codes/train_core.py-613
Physics module implements radius influence, critic supervision (PhysicsCritic, dual-branch, freq_emb_dim 16, detached inside the radius term) and the anti-resonance formulation of eq:anti_resonance.	experiments/ exp059_capacity_freq/ codes/physics_loss.py
Physics weights configured non-zero: physics_ri_weight 1.0, physics_critic_sup_weight 2.0, physics_ar_weight 0.5, physics_critic_warmup_epochs 11, physics_slope_anneal_epochs 22, physics_fg_clip_min -1.6791870594024658.	experiments/ exp059_capacity_freq/ config.yaml

Appendix D. Implementation Index

Statement in the text	Source artifact
Reconstruction weights heatmap 5.0, occupancy 7.0, impedance 4.0, occ_k_consistency 2.0, with heatmap-focus overrides from epoch 25 (heatmap 7.0, impedance 4.0, cross-freq 1.1, dynamic range 3.0); occupancy focal gamma 1.5 with pos-weight (52-K)/K clamped [0.5, 10.0]; impedance composite weights (deriv 1.4, topk_k 20, topk 4.5, under 2.8, concavity 2.5, freq alpha 2.0, dual topk 0.75, peak index 1.5, peak mag 2.0, num_peaks 8, peak start 25 / ramp 22 / focus 25); heatmap shape terms (pearson 0.75, grad vector 2.0, grad direction 1.0, min grad magnitude 0.08, grad huber delta 0.5, top-region 2.5 at q=0.95 and 2.5 at q=0.05 with delta 0.5, peak-loc 2.5 at temperature 0.035, valley-loc 0.0); latent terms (free_bits 0.12, use_beta_annealing false, beta_final 0.03, mu_hinge_weight 0.1 threshold 4.0, mu_bias_weight 0.05, per_expert_kl_weight 0.02, sigma_reg_target 0.45, sigma_reg_weight 1.5, upper band 1.3x target); cross-frequency (weight 1.2, start epoch 20, layout_mix_prob 0.45, use_encode_z true); distillation (latent 2.5 with private x3.0 and logvar 0.3, output 2.5 Huber vs detached teacher); frequency jitter (prob 0.6, log10 sigma 0.1).	experiments/ exp059_capacity_freq/ config.yaml

Training, Inference

Appendix D. Implementation Index

Statement in the text	Source artifact
Training is launched as a module: <code>'python -m experiments.exp059_capacity_freq.codes.train_vae_simple</code> with <code>VAE_EXPERIMENT_DIR</code> set and <code>VAE_CONFIG_PATH</code> unset; two-GPU runs use <code>'torchrun -standalone -nproc_per_node=2 -m experiments.exp059_capacity_freq.codes.train_vae_simple'</code> . (Invocation mechanics; removed from the body.)	<code>experiments/exp059_capacity_freq/codes/train_vae_simple.py</code>
<code>load_yaml_config</code> strips <code>'#'</code> comment lines and calls <code>json.loads</code> , so <code>config.yaml</code> is JSON despite its extension; an optional <code>VAE_CONFIG_PATH</code> environment override is layered on top of <code>config.yaml</code> . (File-format mechanics; removed from the body, override mechanism retained.)	<code>experiments/exp059_capacity_freq/codes/exp059_common.py</code>
The exp059 trainer monkey-patches <code>train_core</code> : <code>Config</code> , <code>build_vae_model</code> , <code>_run_epoch</code> , <code>heatmap_loss</code> , <code>vae_loss</code> , <code>_on_stats_loaded</code> , the dataloader factory, the YAML loader and the impedance loss.	<code>experiments/exp059_capacity_freq/codes/train_vae_simple.py</code>
AdamW with <code>weight_decay 1e-4</code> (fused, then foreach, then plain fallback on CUDA); <code>ReduceLROnPlateau min mode</code> with <code>lr_factor 0.5</code> , <code>lr_patience 15</code> , <code>lr_min 1e-5</code> ; <code>impedance_separate_lr true</code> , <code>impedance_decoder_lr_mult 1.0</code> , <code>impedance_peak_focus_epoch 25</code> .	<code>experiments/exp059_capacity_freq/config.yaml</code>
The experiment notes state the exp059 fresh run used <code>learning_rate 2e-4</code> , <code>num_epochs 500</code> and <code>resume_checkpoint null</code> .	<code>experiments/exp059_capacity_freq/notes.md</code>
The current config states <code>learning_rate 1.5e-05</code> , <code>num_epochs 510</code> , <code>reset_lr_on_resume true</code> and <code>resume_checkpoint experiments/exp059_capacity_freq/checkpoints/last_model.pt</code> – no longer matching the notes.	<code>experiments/exp059_capacity_freq/config.yaml</code> , 8, 138

Appendix D. Implementation Index

Statement in the text	Source artifact
Numerical-stability policy: amp 'off', tf32 false, compile false, training_force_fp32 true, training_skip_nonfinite_batches true, training_grad_clip_norm 1.0, logvar clamp [-10.0, 10.0], recon z soft floor -8.0, finite loss cap 1000000.0; clipping applied as torch.nn.utils.clip_grad_norm_(params, 1.0).	experiments/ exp059_capacity_freq/ config.yaml, experiments/ exp059_capacity_freq/ codes/train_core.py
Curriculum milestones:	experiments/
cross_freq_start_epoch 20,	exp059_capacity_freq/
heatmap_focus_start_epoch 25, modality_dropout_protect_heatmap_epoch 25,	config.yaml
impedance_peak_start_epoch 25,	
beta_end_epoch 25,	
physics_critic_warmup_epochs 11,	
penalty_warmup_epochs 11.	
Validation on epoch 1, each checkpoint interval and the final epoch (val_on_checkpoint_only true), checkpoint_interval 5, keep_last_n_checkpoints 0; off-anchor evaluation every 5 epochs at 155.0/250.0/265.0 MHz with weights 2.5/3.0/2.5, max 20 batches, eval_use_occ_only_layout true.	experiments/ exp059_capacity_freq/ config.yaml
The off-anchor metric doubles as a checkpoint-selection criterion.	experiments/ exp059_capacity_freq/ checkpoints/ best_off_anchor_model.pt
Each run writes loss.csv, heatmap_peak_split.csv, impedance_split.csv, off_anchor_eval.csv, epoch_timing.csv and timing.json.	experiments/ exp059_capacity_freq/ metrics/

Appendix D. Implementation Index

Statement in the text	Source artifact
DDP enabled with <code>ddp_base_batch_size 160</code> and <code>ddp_linear_lr_scale true</code> ; <code>nccl</code> backend with a default 30-minute timeout overridable by <code>NCCL_TIMEOUT_MINUTES</code> ; model wrapped with <code>find_unused_parameters=True</code> .	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/</code> <code>distributed_train.py</code> , <code>experiments/</code> <code>exp059_capacity_freq/</code> <code>config.yaml</code>
Encode-path mixing: <code>layout_train_prob 0.65</code> and <code>occ_only_encode_prob 0.4</code> .	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>config.yaml-48</code>
The notes describe the <code>exp059</code> capacity phase as full-encode only, with both mixing probabilities at 0.	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>notes.md-14</code>
The handover document likewise describes the capacity phase as full-encode only (probabilities 0).	<code>docs/cursor-handoff.md-37</code>
The model parameter count is printed at runtime but stored in no artifact.	<code>experiments/</code> <code>exp059_capacity_freq/</code> <code>codes/train_core.py</code>

Auxiliary Guidance Network for Latent-Space Optimization

Statement in the text	Source artifact
The nearest artifact to an 'auxiliary guidance network' is a feed-forward occupancy-to-impedance surrogate: <code>occupancy (52,)</code> to <code>impedance log-z (1, 231)</code> , <code>PI_freq</code> unused; <code>hidden_dims (256, 512, 512, 512, 256)</code> , <code>dropout 0.1</code> , <code>num_epochs 200</code> , <code>batch_size 128</code> , <code>learning_rate 3e-4</code> , <code>lr_patience 15</code> , <code>lr_factor 0.5</code> , <code>lr_min 1e-5</code> , <code>weight_decay 1e-4</code> , <code>topk_k 20</code> , <code>topk_weight 7.0</code> , <code>under_penalty 2.8</code> . Present only in <code>exp037_lat_change</code> , <code>exp038_true_multi</code> , <code>exp039_improved_heatmap</code> , <code>exp040</code> and <code>exp041</code> ; absent from <code>exp057</code> , <code>exp058</code> , <code>exp059</code> , <code>exp060</code> .	<code>experiments/</code> <code>exp038_true_multi/codes/</code> <code>surrogate_impedance.py</code>

Appendix D. Implementation Index

Statement in the text	Source artifact
Stage 2 loads the impedance surrogate checkpoint when USE_SURROGATE is on (the default), and otherwise falls back to the VAE impedance decoder head.	experiments/ exp038_true_multi/ checkpoints/ surrogate_best.pt

Problem Formulation

Statement in the text	Source artifact
Stage 2 defaults to EXPERIMENT = 'exp038_true_multi' for both the VAE occupancy decoder and the impedance surrogate checkpoint, and imports MultiInputVAE from that fork; feasibility margin BOUNDARY_MARGIN 0.1, objective mode 'gap_max', LOSS_SPACE 'log', EXCEED_POWER 2.0, gap weight 1.0, exceed weight 25.0; target mask loaded from configs/target_impedance.npy, required shape (231,) after squeeze, converted by $(\log Z - \mu_{\log})/\sigma_{\log}$.	pipelines/latent/ optimize.py
Exact form of the exceed maximum-excursion component and of the gap term (not extracted; subject of a \{\}fillin).	pipelines/latent/ optimize.py-329
Stage-2 default normalization statistics path is a directory absent from the current checkout.	pipelines/latent/ datasets/data_multifreq_norm/normalization_statistics/optimize.py
The exp059 training set is which is the path Stage 2 should use.	experiments/ datasets/data_multifreq_train_norm_unbounded/exp059_capacity_freq/ config.yaml
The limitation that Stage 2 is not wired to exp059 is also recorded in the project documentation.	docs/ latent-optimization.md sec.7
The stored target mask has shape (231, 1), dtype float64, minimum 0.8 and maximum 49.99999999999999, with no unit metadata and no located generating script.	configs/ target_impedance.npy

Optimization Procedure and Feasibility Assessment

Statement in the text	Source artifact
Search settings: K_LIST 1..25, NUM_CANDIDATE_SEEDS 32, NUM_STEPS 1200, LR 5e-2, GRAD_CLIP 5.0, INIT_SHARED_TEMP 0.8, OPTIMIZE_BATCH_PER_K True; seeds random per run (saved to run_config.json) or fixed 0..N-1 via LATENT_OPT_SEED_MODE, default 'random'.	pipelines/latent/ optimize.py
Straight-through top-K (<code>_ste_topk</code>) with the in-code justification 'Without this the top-K argsort severs the gradient and z never moves on the spectrum target'; applied only on the surrogate branch, <code>surrogate(_ste_topk(occ, K))</code> , while the fallback model.decode_impedance(z, K) branch keeps the read-out out of the gradient path. USE_SURROGATE defaults to True.	pipelines/latent/ optimize.py
Search regularizers: Z_L2_WEIGHT 0.05 with Z_PRIOR_MODE 'agg_posterior', POSTERIOR_BOUNDARY_WEIGHT 10.0 beyond POSTERIOR_SIGMA_LIMIT 2.5, SHAPE_REG_WEIGHT 2.0 with SHAPE_MIN_STD_DLOG 0.12, SHAPE_TRACK_WEIGHT 2.0, PEAK_LOSS_WEIGHT 1.0, PEAK_INDEX_WEIGHT 3.0, PEAK_MAG_WEIGHT 2.0, DUAL_TOPK_WEIGHT 2.5, IMPEDANCE_NUM_PEAKS 8, IMPEDANCE_TOPK_K 20 (all overridable via LATENT_OPT_* environment variables), DIVERSITY_WEIGHT 0.35, OCC_CONFIDENCE_WEIGHT 0.5, PHYSICS_AR_WEIGHT 0.5 overridable by the checkpoint config's physics_ar_weight.	pipelines/latent/ optimize.py

Appendix D. Implementation Index

Statement in the text	Source artifact
Selection metric <code>SELECT_METRIC = 'max_ohm'</code> (alternative <code>'mean'</code>); infeasible budgets emit an explicit <code>no_solution.json</code> rather than a best-effort placement.	<code>pipelines/latent/optimize.py</code>

Experimental Setup

Statement in the text	Source artifact
ECADSTAR host settings: <code>erf_path</code> <code>C:\Users\muthusamy\Desktop\design\H-shape.emc\H-shape.erf</code> , <code>powerbus</code> <code>Power_GND</code> , <code>peb_components</code> <code>IC1_Port1</code> , <code>heatmap_only_peb</code> <code>true</code> , <code>wait_timeout_sec</code> <code>18000</code> . (Absolute Windows path removed from the body.)	<code>active_learning_pi/Hconfig/exp059_gp_error.json</code>
ECADSTAR is driven from WSL with Windows path conversion; the simulation steps cannot run without that host.	<code>active_learning_pi/al/ecadstar.py</code>
The residual GP is implemented with GPyTorch as an ApproximateGP trained with a VariationalELBO.	<code>active_learning_pi/al/gp_error_surrogate.py</code>
Stale reference document: dated 2026-06-22, describes the exp050 Tier-A configuration, lists six anchors and a heatmap weight of 4.0, neither matching the exp059 dataset nor its config. Usable for method description, not for numbers.	<code>docs/normalization-and-losses.md</code>
Stale reference document: written for the exp054-to-exp057 lineage, with its own caveat that exact epoch counts, weights and off-anchor MHz lists are config-specific and should be taken from the experiment's config.yaml.	<code>docs/training-procedure.md</code>
Configuration files of record: capacity phase and AL fine-tune, the latter injected at runtime by <code>VAE_CONFIG_PATH</code> .	<code>experiments/exp059_capacity_freq/config.yaml</code> , <code>experiments/exp059_capacity_freq/config_al_finetune.yaml</code>

Active Learning for scaling the model

Statement in the text	Source artifact
Entry point with a CONFIG block: COMMAND = 'full' runs the whole cycle, PROPOSE_ONLY = True runs generation and inference only. (Entry-point mechanics removed from the body.)	pipelines/ active_learning/run.py
The AL pipeline module refuses to run directly and instructs the user to edit the CONFIG block in run.py.	active_learning_pi/al/ pipeline.py
The cycle step count is computed at runtime and is 8 only when fine-tuning and post-fine-tune evaluation are enabled, otherwise 7 or 6.	pipelines/ active_learning/run.py
Primary configuration (residual-GP acquisition) with k_min 3, k_max 10, k_sweep_per_pool true, candidates_per_k 1600, worst_per_k 80, total_ecad_per_cycle 640, candidate_seed 42 with seed = candidate_seed + iteration*10000 + K, MHz grid and per-K strata quotas, explore_candidates_per_k 160 over 120-320 MHz quantized to 5 MHz with band edges [120, 180, 230, 280, 320], min_uncertainty_percentile 40, mhz_strata_per_k set with worst_mhz_strata_per_k null (auto-scaled to worst_n).	active_learning_pi/ config/ exp059_gp_error.json
Alternative configurations: Monte-Carlo acquisition and equal-budget random control.	active_learning_pi/ config/exp059.json, active_learning_pi/ config/exp059_random.json
Exp057-era documented budgets conflict with the exp059 configuration: candidates_per_k 400, worst_per_k 10, 80 ECAD simulations per cycle, off-anchor frequencies 90/265/435/510 MHz.	docs/active-learning.md sec.3-4

Appendix D. Implementation Index

Statement in the text	Source artifact
GP settings: feature path 'pred', target 'p99_ae', regressor 'svgp', kappa 0.5, n_fit 4000, n_inducing 256, svgp_epochs 80, seed 0, novelty_weight 0.0, score_mode 'mu'. Alternative residual targets peak_px, peak_soft_px, peak_sharp_px, struct, mae, mse, imp_mse.	active_learning_pi/ config/ exp059_gp_error.json, active_learning_pi/al/ gp_error_surrogate.py
predict_candidates dispatches between gp_error, random and mc, defaulting to mc when unset; the GP feature path encodes occupancy (encode_occupancy_latent), decodes, re-encodes the model's own predicted heatmap and uses that posterior mean.	active_learning_pi/al/ inference_pool.py
Docstring conflict: states that gp_error candidates are ranked 'by UCB $\mu_e + \kappa \sigma_e$ '.	active_learning_pi/al/ inference_pool.py-179
score_mode 'mu' resolves to the posterior mean only, with sigma unused; the novelty term enters only under 'ucb_novelty'.	active_learning_pi/al/ gp_error_surrogate.py-352
Documentation states the mean-only default explicitly.	docs/ gp-error-surrogate.md sec.5
The fitted GP artifact is cleared at the start of every per-K acquisition, so each cycle refits.	active_learning_pi/al/ per_k_acquire.py-69
Random control assigns uniform scores from np.random.default_rng(seed + iteration), preserving the ECAD budget exactly.	active_learning_pi/ al/inference_pool.py, active_learning_pi/ config/exp059_random.json
MC-mode parameters retained in the exp059 config: mc_passes 4, mc_latent_passes 4, mc_decoder_passes 8, mc_decoder_dropout true, score_metric 'auto_bad', weights_decoder_dropout_rce 1.0, latent_heatmap_mse 1.0, p99_var 1.0, p99_range 0.5, peak_loc_spread 0.5, peak_cv 0.25, prior_tension 0.35.	active_learning_pi/ config/ exp059_gp_error.json

Appendix D. Implementation Index

Statement in the text	Source artifact
The exp059 acquisition A/B (validation_acquisition_ab.py) is still an open task and the claim ledger records acq_ab_gp_vs_mc as UNCERTAIN.	docs/cursor-handoff.md sec.3D
Scoring settings: inference_mode 'layout', pi_ref_mhz 200.0, shared_temp 1.5, heatmap_grid_size 64, normalization_stats_json pointing at the unbounded training set.	active_learning_pi/ config/ exp059_gp_error.json
Board mask used at scoring time is a 64x64 boolean array.	configs/binary_mask.npy
Ingested labels are merged into the AL overlay dataset mixed into training with al_overlay_sample_weight 128.0; fine-tune resumes from checkpoints/last_model.pt with num_epochs = checkpoint_epoch + extra_epochs, extra_epochs 45; the merged runtime config is written to config_al_finetune.runtime.yaml and passed via VAE_CONFIG_PATH.	datasets/ data_multifreq_al_overlay_exp059, active_learning_pi/ config/ exp059_gp_error.json
Fine-tune configuration sets layout_train_prob 0.9 and occ_only_encode_prob 0.7, with latent_distill_weight 2.5, output_distill_weight 2.5, eval_use_occ_only_layout true and early stopping (patience 3, min delta 0.01, kind 'layout_cross').	experiments/ exp059_capacity_freq/ config_al_finetune.yaml-12
write_runtime_finetune_config copies the AL JSON keys over the values in config_al_finetune.yaml, so the executed fine-tune values are 0.65 and 0.4.	active_learning_pi/al/ finetune_run.py-117
Project documentation states the fine-tune mixing probabilities as 0.9 / 0.7, which is not what runs.	docs/cursor-handoff.md sec.2, CLAUDE.md

Appendix D. Implementation Index

Statement in the text	Source artifact
Reporting gates: <code>decision_min_rank_n 30</code> , <code>decision_min_ecad_n 20</code> , <code>infer_scope 'ingested'</code> .	<code>active_learning_pi/config/</code>
Per-cycle artifacts: <code>scored_candidates.json</code> , <code>acquisition_rank_quality.json</code> , <code>eval_off_anchor_pre_finetune.json</code> , <code>eval_off_anchor_post_finetune.json</code> , <code>eval_cycle_summary.json</code> , <code>CY-CLE_EVAL_REPORT.md</code> , <code>DECI-SION_REPORT.md</code> , <code>decision_ledger.json</code> .	<code>exp059_gp_error.json</code> , <code>active_learning_pi/al/decision_report.py</code>
The loop has run on <code>exp059</code> for four iterations; the fourth records acquisition mode <code>gp_error</code> , <code>K</code> values <code>[3,4,5,6,7,8,9,10]</code> , <code>candidates_per_k 1600</code> , <code>worst_per_k 80</code> , <code>total_candidates 12800</code> , <code>total_selected 640</code> .	<code>active_learning_pi/runs/al_exp059_gp_error_001/iter_0004/k_sweep_summary.json</code>